HYBRID AI-BASED DETECTION OF FILELESS MALWARE IN IoMT HEALTHCARE SYSTEMS

INDIVIDUAL THESIS

SHAKYA W.A.R

IT22210142

Component 04: Cross Layer Lateral Movement Correlation Detection Engine and XAI Dashboard.

Group ID: R26-CS-006

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Information Technology

Specializing in Cyber security

Department of Computer systems and Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

[October 2026]

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: SHAKYA W.A.R.

Student ID: IT22210142

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Kanishka Yapa		
Co-supervisor: Mr. Dinith Primal		

Abstract

This thesis examines Component 4 of the project Hybrid AI-based Detection of Fileless Malware in IoMT Healthcare Systems: a cross-layer lateral-movement correlation detection engine and explainable artificial intelligence dashboard. The component addresses the difficulty of relating heterogeneous security observations into an inspectable incident while distinguishing observed activity, model output and forecast support. The supplied prototype integrates event normalisation, compatible machine-learning inference, graph and temporal correlation, memory–network association, weighted fusion, SQLite persistence and a SOC-oriented dashboard. The study combines source inspection, analysis of archived metadata and an isolated database-resilience verification. Thirty-three Python files parsed successfully, and twenty database assertions passed across nine test sections. Historical metadata reports XGBoost and Random Forest F1 values of 0.9162 and 0.9231 respectively; these are classifier summaries and are not interpreted as incident-level detection performance. An archived staged integration test failed because a final incident appeared earlier than expected. Separate database and runtime snapshots are reported without assuming that they represent the same run. The contribution is an implemented, evidence-linked incident pipeline and a reproducible framework for its evaluation. The available evidence does not establish hospital deployment, incident-level superiority, explanation faithfulness or measured stakeholder usability. Future work should resolve the staged failure, evaluate provenance-aware corroboration, test restart and delivery behaviour, and conduct controlled detection and analyst studies.

Keywords: cross-layer correlation; lateral movement; IoMT security; explainable AI; SOC dashboard.

ACKNOWLEDGEMENTS

This research uses the collective project context and supplied integration artefacts while limiting its claims to the individual Component 4 contribution. The final author should confirm and acknowledge the specific assistance of supervisors, collaborators and any participating organisations before submission.

The thesis acknowledges the researchers and maintainers whose publications, datasets and software underpin the technical discussion. Their work is cited in the references. The supplied university formatting and structure documents guided the organisation of this manuscript.

Generative-AI assistance in preparing this draft is disclosed in Appendix B. Responsibility for the final submitted work, the accuracy of its claims and compliance with institutional rules remains with the student.

Table of Contents

Abstract ii

ACKNOWLEDGEMENTS iii

LIST OF FIGURES x

LIST OF TABLES xi

LIST OF ABBREVIATIONS xii

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 2

1.3 Problem Statement 2

1.4 Research Gap 3

1.5 Aim and Objectives 4

1.6 Research Questions or Hypotheses 4

1.7 Scope, Limitations and Assumptions 5

1.8 Individual Contribution 6

1.9 Thesis Structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review Method and Scope 7

2.2 Domain and Theoretical Foundations 7

2.2.1 Events, observations and incidents 7

2.2.2 Lateral movement and remote services 8

2.2.3 Host and memory evidence 8

2.2.4 Missingness and dependence 9

2.3 Related Research 9

2.3.1 Provenance-based correlation 9

2.3.2 Supervised and unsupervised models 10
2.3.3 Model explanations 11
2.4 Existing Systems, Products and Techniques 11
2.5 Gap Synthesis and Conceptual Framework 12
3 RESEARCH METHODOLOGY 14
3.1 Research Design 14
3.2 Data, Participants or Study Materials 14
3.2.1 Project artefacts 14
3.2.2 Benchmark data descriptions 15
3.2.3 Participants and ethics boundary 15
3.3 Experimental or Study Procedure 16
3.3.1 Completed inspection procedure 16
3.3.2 Prospective detection experiment 16
3.3.3 Robustness and integration procedure 17
3.4 Measures and Evaluation Metrics 17
3.4.1 Detection metrics 17
3.4.2 Reliability and explanation metrics 18
3.5 Data Analysis 19
3.6 Ethics, Privacy, Security and Responsible AI 19
3.7 Reproducibility and Research Artefacts 20
4 PRODUCT AND SOLUTION DESIGN 21
4.1 Requirements 21
4.1.1 Functional requirements 21
4.1.2 Non-functional requirements 21
4.2 Component Architecture within the Complete Solution 22
4.3 Design Alternatives and Decisions 23

4.3.1 Central incident service versus tightly coupled models 23
4.3.2 Weighted fusion versus learned incident fusion 23
4.3.3 SQLite and bounded in-memory state 24
4.4 Detailed Design 24
4.4.1 Unified event and provenance 24
4.4.2 Model eligibility and feature spaces 25
4.4.3 Graph correlation 26
4.4.4 Temporal correlation 26
4.4.5 Memory–network association 27
4.4.6 Fusion and evidence gates 27
4.4.7 Incident lifecycle and persistence 28
4.4.8 SOC and restricted clinical presentation 30
4.4.9 Explanation contract 30
4.5 Standards and Professional Practice 31
5 IMPLEMENTATION, INTEGRATION AND TESTING 32
5.1 Development Environment and Technology Stack 32
5.2 Component Implementation 32
5.2.1 Ingestion and normalisation 32
5.2.2 Orchestration and signal state 33
5.2.3 API retrieval and feedback 33
5.2.4 Dashboard implementation boundary 34
5.3 Integration with Other Components 34
5.3.1 Observed producer evidence 34
5.3.2 Forecast support 34
5.3.3 Delivery and identity integration 35
5.4 Verification and Software Testing 35

5.4.1 Completed checks 35
5.4.2 Archived integration failure 36
5.4.3 Test coverage limitations 36
5.5 Deployment and Operation 36
6 RESULTS AND VALIDATION 38
6.1 Research Results 38
6.1.1 Archived supervised-model summaries 38
6.1.2 Archived data and anomaly summaries 38
6.2 Product and Technical Results 39
6.2.1 Implemented capabilities 39
6.2.2 Persistence results 39
6.2.3 Evidence completeness 40
6.3 User or Stakeholder Validation 40
6.4 Integration Results 41
6.5 Summary of Findings 41
7 DISCUSSION 43
7.1 Interpretation of Findings 43
7.2 Comparison with Literature, Baselines and Existing Solutions 43
7.3 Research Contribution 44
7.4 Product Contribution 44
7.5 Limitations and Threats to Validity 45
7.5.1 Internal validity 45
7.5.2 Construct validity 45
7.5.3 External validity 46
7.5.4 Reliability and reproducibility 46
7.6 Implications 46

8 CONCLUSION AND FUTURE DIRECTIONS 48
8.1 Conclusion 48
8.2 Achievement of Objectives 48
8.3 Individual Research and Product Contributions 49
8.4 Future Directions 49
REFERENCES 50
GLOSSARY 52
APPENDIX A: RESEARCH INSTRUMENTS AND EXTENDED RESULTS 53
A.1 Evidence Register and Reproducibility Manifest 53
A.2 Configuration and Feature Contracts 54
A.3 Persistent Schema Audit 57
A.4 Evaluation Scenario Catalogue 61
A.5 Trial Record and Analysis Worksheet 62
A.6 Proposed Stakeholder Validation Instrument 63
A.7 Completed Database Verification Record 64
A.8 Archived Staged Integration Failure 66
A.9 Typed API Request and Response Contracts 67
A.10 Fusion, Scoring and Incident Update Logic 71
A.11 Orchestrator and Model Eligibility 83
A.12 Graph Construction and Path Evaluation 91
A.13 Temporal Pattern Correlation 100
A.14 Memory–network Correlation 106
A.15 Database Manager and Transaction Boundary 114
A.16 SHAP Model-explanation Implementation 116
APPENDIX B: ACADEMIC INTEGRITY AND AI-USE DISCLOSURE 119
B.1 Assistance Used 119

B.2 Evidence and Authorship Boundaries 119

B.3 Required Author Review Before Submission 120

LIST OF FIGURES

Figure 4.1. Component 4 architecture and evidence boundaries. 24

Figure 4.2. Eligibility and fusion decision boundaries. 30

Figure 4.3. Persistent incident, evidence and feedback relationships. 31

LIST OF TABLES

Table 1.1. Research objectives and evidence requirements. 4
Table 2.1. Qualitative comparison of relevant approaches. 13
Table 3.1. Proposed evaluation measures and their boundaries. 19
Table 4.1. Functional requirements and verification approaches. 22
Table 5.1. Component 4 analytical API routes. 35
Table 6.1. Archived supervised-model metrics, not reproduced. 40
Table 6.2. Separate persistence evidence snapshots. 41
Table 6.3. Objective achievement and remaining validation. 44
Table A.1. Evidence register. 55
Table A.2. Selected configuration values. 56
Table A.3. Ordered 37-feature network-model input. 56
Table A.4. Six-feature memory-model input. 59
Table A.5. Stored events columns. 59
Table A.6. Stored alerts columns. 61
Table A.7. Stored alert_evidence columns. 62
Table A.8. Stored xai_explanations columns. 62
Table A.9. Stored analyst_feedback columns. 62
Table A.10. Prospective functional and robustness scenarios. 63
Table A.11. Prospective trial record. 64
Table A.12. Prospective stakeholder tasks. 66
Table B.1. Final author-review responsibilities. 121

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
C4	Component 4
CORS	Cross-origin resource sharing
DICOM	Digital Imaging and Communications in Medicine
F1	Harmonic mean of precision and recall
FN / FP	False negative / false positive
IoMT	Internet of Medical Things
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
ML	Machine learning
RF	Random Forest
RQ	Research question
SHAP	SHapley Additive exPlanations
SMB	Server Message Block
SMOTE	Synthetic Minority Over-sampling Technique
SOC	Security operations centre
TP	True positive
UTC	Coordinated Universal Time
WAL	Write-ahead logging
XAI	Explainable artificial intelligence

1 INTRODUCTION

1.1 Overall Project Context

The overarching research project, Hybrid AI-based Detection of Fileless Malware in IoMT Healthcare Systems, investigates a layered approach to identifying malicious behaviour in a healthcare computing environment. Its setting includes endpoints, clinical workstations, networked services and connected medical infrastructure. Within this environment, a suspicious process, an unusual connection and an application anomaly may be recorded by different monitoring mechanisms. The project therefore requires a way to examine relationships among observations rather than treating every component output as a complete incident.

This individual thesis concerns Component 4: the Cross-Layer Lateral Movement Correlation Detection Engine and XAI Dashboard. Other components appear only as external evidence producers or consumers. Their algorithms, datasets, model training and individual research claims are outside the contribution evaluated here. The purpose of describing their interfaces is to establish what Component 4 can legitimately infer from incoming evidence and what remains unknown when a producer is unavailable.

The term fileless is used operationally to describe behaviour involving memory-resident activity, script interpreters or legitimate administration mechanisms that may not yield a conventional executable-file signature. It does not imply that every incident leaves no disk artefacts. Lateral movement means activity connecting an initial foothold to another asset or execution context. Neither a suspicious interpreter nor a connection to a remote-administration port independently proves that this movement occurred. An explanation must identify the observations supporting the relationship and distinguish those observations from hypotheses.

Clinical continuity is a design concern because a security interface may be used near systems that support patient care. Component 4 is an analyst decision-support prototype, not an autonomous medical-device controller. Its output should support investigation and appropriately authorised response. The evidence available for this

thesis does not demonstrate deployment in a production hospital or establish patient-safety outcomes.

1.2 Individual Component Context

Component 4 receives heterogeneous events, normalises them, evaluates compatible machine-learning inputs, correlates related activity, fuses candidate signals and exposes incidents with explanations. The supplied implementation contains graph, temporal and memory–network correlators, a FastAPI service, SQLite persistence and a Dash-based dashboard [1]. This is the implementation examined in this thesis. A separate landing page, doctor portal or later frontend revision must not be assumed to be part of this inspected snapshot.

The central challenge is semantic: a producer's confidence score, a graph pattern and a forecast do not describe the same thing. Component 4 must preserve source identity, timestamps, target assets and evidence lineage before assigning an incident-level score. It must also represent an unavailable model differently from a model that evaluated an event and found little evidence of malicious activity.

The dashboard is the human-facing part of this research contribution. It should let a security operations centre (SOC) analyst move from a prioritised incident to its evidence, path, chronology, signal contributions, model explanation and investigation status. A clinician-facing summary is considered only as a restricted presentation requirement for Component 4 output. It is not a separate clinical application developed or evaluated by this thesis.

1.3 Problem Statement

Independent alerts can obscure the relationships needed to recognise movement across assets. An endpoint detector may report suspicious execution without a destination; a network detector may identify an anomalous flow without the originating process; and a forecasting component may estimate future risk without observing an attack. Simply summing these outputs can create misleading certainty, particularly when several outputs derive from the same original observation.

The research problem is to construct an inspectable correlation mechanism that connects compatible observations across layers and presents a defensible incident narrative. The mechanism must preserve evidence provenance, apply explicit eligibility and correlation rules, avoid treating forecasts as observations, and expose uncertainty to an analyst. A useful incident is therefore more than a high score: it is a bounded claim about assets, time, behaviour and supporting records.

The available project materials demonstrate implementation work and selected verification evidence, but they do not constitute a complete controlled evaluation of lateral-movement detection effectiveness. Accordingly, this thesis separates implementation findings, archived model measurements, a newly executed database verification and an unexecuted evaluation protocol. This separation is necessary to avoid presenting software existence as proof of research effectiveness.

1.4 Research Gap

Prior provenance-based work demonstrates the value of connecting security observations into larger attack representations [2], [3], [4], [5]. Explainability methods provide mechanisms for examining model predictions [6], [7]. However, applying these ideas within this project's heterogeneous IoMT-oriented prototype introduces an integration gap: model-level explanations must coexist with graph, temporal and memory evidence, while the analyst must be able to see where each contribution originated.

The gap addressed is not a claim that cross-layer correlation or explainable intrusion detection has never been studied. It is the narrower design and evaluation problem of producing an evidence-linked incident representation for this specific component architecture. A second gap concerns evidence quality: outputs may share an origin, arrive late, use different feature spaces, or describe a forecast rather than an observation. These conditions require explicit treatment within the correlation and dashboard contract.

The thesis therefore positions the contribution as an implemented integration artefact and a critical evaluation of its evidence handling. Claims of algorithmic superiority,

clinical generalisation or a novel universally optimal fusion rule are not supported by the supplied materials and are not made.

1.5 Aim and Objectives

The aim is to design, implement and critically evaluate a Component 4 engine that correlates cross-layer evidence of potential lateral movement and communicates its reasoning through an explainable SOC dashboard.

Table 1.1. Research objectives and evidence requirements.

ID	Objective	Evidence required
O1	Establish a common event representation with provenance and asset context.	Normalisation logic, interface contracts and validation cases.
O2	Combine graph, temporal and memory–network evidence with compatible ML outputs.	Inspected processing paths and controlled correlation scenarios.
O3	Produce an incident explanation that separates observed evidence, forecast support and model attribution.	Stored explanation schema and analyst-facing traceability.
O4	Persist incidents, evidence links and analyst feedback reliably.	Schema inspection and isolated database verification.
O5	Evaluate detection quality, robustness and usability with reproducible methods.	Incident-labelled trials, ablations and stakeholder assessment; these remain incomplete.

Table 1.1 defines the objectives and the kinds of evidence needed to evaluate them. Implementation coverage and empirical effectiveness are assessed separately throughout the thesis. A feature may be present in source code while its operational benefit remains unmeasured.

1.6 Research Questions or Hypotheses

RQ1 asks how heterogeneous observations can be transformed into a common representation without losing the identity and temporal relationships needed for lateral-movement correlation. RQ2 asks how graph, temporal, memory and compatible model signals can be combined while distinguishing missing evidence and forecast support. RQ3 asks what explanation content enables an analyst to trace a fused incident back to its originating observations. RQ4 asks how persistence, delayed delivery and degraded dependencies affect the reliability of the component. RQ5 asks whether the

complete component improves incident detection and analyst task performance over simpler baselines.

The present evidence permits architectural and implementation answers to RQ1–RQ4, with limited verification of database behaviour. It does not permit a statistically defensible positive answer to RQ5. A prospective hypothesis is that evidence-linked correlation reduces disconnected alerts while maintaining incident recall compared with independently thresholded source alerts. A second prospective hypothesis is that the explanation interface reduces investigation time without reducing decision accuracy. Both require the experiments specified in Chapter 3; they are not reported as confirmed findings.

1.7 Scope, Limitations and Assumptions

The technical scope includes ingestion, normalisation, model eligibility, graph and sequence correlation, memory–network association, incident fusion, evidence persistence, explanation generation, API retrieval and SOC presentation. The scope excludes training or evaluating the other project components, developing malware, autonomous containment of clinical devices, patient diagnosis and certification for operational hospital use.

The evidence boundary is the supplied Component 04 archive, the integration guidance document and the two university templates. The archive is treated as a source snapshot rather than a verified representation of the latest running deployment. No commit identifier or complete experiment ledger establishes a chronological sequence among every file. Conflicting snapshots are therefore retained as separate observations rather than merged into a synthetic history.

The design assumes authorised collection, sufficiently stable asset identifiers, meaningful event timestamps and access to the evidence needed for an analyst to verify an alert. These assumptions are not guaranteed in real deployments. Shared addresses, address reassignment, missing clocks, incomplete process telemetry and producer outages can reduce the confidence of a correlation even when its numerical score is high.

1.8 Individual Contribution

The component's individual contribution is the integration of evidence into an incident lifecycle: a shared event representation; multiple correlation mechanisms; a controlled fusion stage; durable links between incident and evidence; and an explanation-oriented dashboard. The inspected code provides concrete artefacts for these responsibilities [1]. Responsibility for upstream collection or classification is described only at the interface boundary.

This manuscript also contributes an evidence audit of the supplied implementation. It identifies the distinction between 37-feature network models, a six-feature memory model and twelve contextual correlation features; the distinction between archived training metrics and incident-level detection performance; and the distinction between persistent database records and recovered in-memory correlation state. These observations are important for interpreting the research honestly.

Authorship and contribution ownership must be confirmed by the student against project records before submission. Source availability does not independently establish who wrote each line, and this thesis does not attribute upstream work to the Component 4 author.

1.9 Thesis Structure

Chapter 2 reviews the technical foundations and related research. Chapter 3 defines the methodology, available evidence and prospective evaluation. Chapter 4 specifies the product architecture and design decisions. Chapter 5 examines implementation, integration and testing. Chapter 6 reports supported results. Chapter 7 interprets the findings and limitations. Chapter 8 concludes and identifies future work. The appendices provide reproducibility material, interface details, test instruments, selected implementation listings and the academic-integrity disclosure.

The supplied structure document contains a mismatch between its illustrative contents list and its chapter body. This thesis follows the eight-chapter body. Commercialisation considerations are included within product design and future directions, without adding a ninth research chapter or expanding the individual contribution beyond Component 4.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review is a focused technical review rather than a systematic review with a registered protocol. Primary research and official technical documentation were selected for direct relevance to provenance correlation, anomaly detection, explainability, benchmark data and incident handling. Searches and verification were performed during manuscript preparation on 9 October 2026. No exhaustive database coverage, screening count or inter-reviewer agreement is claimed.

The review asks three practical questions. First, what forms of relational evidence can support an incident-level explanation? Second, which modelling approaches are relevant to the inspected component's feature spaces? Third, how should an interface communicate the boundary between an observed event, a model attribution and an analyst conclusion? Sources that address unrelated clinical prediction tasks or the internal algorithms of other project components are excluded.

The comparison is qualitative because the cited systems use different datasets, telemetry assumptions and evaluation units. A reported result for an endpoint provenance system cannot be transferred directly to an IoMT-oriented, multi-producer prototype. The literature informs requirements and experimental design; it does not supply substitute measurements for Component 4.

2.2 Domain and Theoretical Foundations

2.2.1 Events, observations and incidents

An event is a recorded item of activity; an observation is an event interpreted within a collection context; and an incident is an investigation-level grouping with an explicit security claim. This distinction prevents the component from equating the number of ingested records with the number of attacks. A repeated delivery can create several messages but still refer to one observation. Conversely, one incident may contain many relevant and irrelevant records.

For this thesis, cross-layer correlation means associating evidence from different behavioural views through shared entities and a bounded time relationship. It does not require every candidate to contain every layer. A missing layer must remain visible as a coverage limitation. The design is strongest when records share stable identifiers, such as an asset, process or authenticated session, rather than merely occurring near one another.

2.2.2 Lateral movement and remote services

MITRE ATT&CK's Remote Services technique describes adversarial use of mechanisms that also support legitimate administration [8]. Its relevance is the need to examine remote access together with subsequent activity. Component 4 consequently treats a remote-service port as contextual evidence, not an attack label. A plausible movement claim should identify a source, a destination, the relationship connecting them and the evidence supporting activity at the destination.

Healthcare examples make this distinction concrete. A workstation transferring an image to an imaging service may be expected behaviour. A maintenance account accessing several systems may also be authorised. The correlator should consider expected asset roles, services and timing, while allowing an analyst to inspect unusual execution or authentication that changes the interpretation. Baselines support this assessment but cannot guarantee that familiar communication is harmless.

2.2.3 Host and memory evidence

Microsoft's Sysmon documentation identifies event types for process creation, network connections and other process-related activity [9]. Such telemetry can strengthen a relational hypothesis when it can be joined to network and identity context. However, the existence of a process-access or remote-thread observation is not automatically proof of malicious injection. Interpretation depends on the collection configuration, participating processes and surrounding behaviour.

The inspected component includes memory-oriented indicators and a memory–network correlator. This should be understood as reasoning over supplied memory/host evidence rather than proof that the thesis deployed comprehensive

memory forensics on medical devices. A manually entered script or synthetic event must retain that provenance; it must not become a claim of execution on a monitored host.

2.2.4 Missingness and dependence

Missing evidence and negative evidence have different meanings. A network model that did not run because features were unavailable supplies no evaluated score. Replacing that state with zero would imply evidence of normality. The implementation instead normalises weights over available scores, which preserves this distinction but can increase the relative influence of a small available subset. An evidence gate is therefore needed in addition to a numerical threshold.

Dependence is a separate issue. Graph, temporal and model signals may all be triggered by the same underlying event. Counting three mechanisms does not establish three independent observations. Component 4's signal-family count is useful for describing processing diversity, but stronger research claims require provenance-aware overlap analysis. This thesis uses the term corroboration cautiously when event lineage has not been independently verified.

2.3 Related Research

2.3.1 Provenance-based correlation

HOLMES correlates suspicious information flows into higher-level representations of advanced attacks [2]. Its relevance to Component 4 is the movement from isolated low-level alerts to a connected account of behaviour. The present component differs in its use of heterogeneous producer messages, explicit score fusion and a dashboard contract. HOLMES does not provide direct evidence that the local fusion weights are optimal.

SLEUTH uses audit-derived dependency relationships to support attack detection and reconstruction [3]. It motivates retaining the path from a conclusion to the observations that support it. Component 4 has a more limited evidence boundary when a producer sends an alert summary instead of raw provenance. In that case, the dashboard must

identify the summary as a producer assertion and offer a reference to the original record where available.

UNICORN represents evolving provenance through graph sketches for anomaly detection [4]. This is relevant to the challenge of retaining longer behavioural history without treating every event as an independent sample. The inspected Component 4 instead uses bounded correlation windows and explicit graph/path logic. The choice simplifies inspection but leaves slow movement outside the active window as an acknowledged limitation.

KAIROS addresses practical intrusion detection and investigation over whole-system provenance graphs [5]. It provides a useful comparison for the combined detection-and-investigation objective. Component 4 is not evaluated on an equivalent provenance corpus, so a performance ranking would be invalid. Its contribution is assessed in relation to its own interfaces, evidence links and bounded deployment assumptions.

2.3.2 Supervised and unsupervised models

XGBoost is a tree-boosting framework [10]. Random forests combine tree predictors [11]. These methods are relevant because both appear in the supplied network-model artefacts. Their algorithmic identities do not establish input compatibility: a deployed model must receive the feature names, ordering and preprocessing used by its fitted pipeline. Component 4 therefore needs an eligibility check before inference.

Isolation Forest detects anomalies through isolation rather than a supervised attack-class boundary [12]. In the archive it is associated with six memory/host features. An anomaly score describes unusualness under the fitted representation, not a verified lateral-movement incident. It becomes useful to Component 4 when combined with a specific asset, timestamp and behavioural relationship.

SMOTE synthesises samples to address class imbalance [13]. Its use must be confined to the training partition to avoid contaminating evaluation. The archived metadata records resampling counts, but the metadata alone does not prove the split procedure or absence of leakage. The thesis therefore reports those counts as archival facts and

requires a rerunnable training pipeline before stronger conclusions about generalisation.

2.3.3 Model explanations

SHAP provides an additive attribution approach for explaining model predictions [6]. For Component 4, the useful question is which compatible input features contributed to a particular model output. Attribution values depend on the model, reference data and output space. They should not be displayed as causal effects or as proof that a feature caused an attack.

LIME explains a prediction through a locally fitted interpretable surrogate [7]. Its presence in the source supports an implementation claim, but the archive does not provide a completed explanation-fidelity or stability study. A plausible-looking explanation must still be checked for consistency with the actual instance and model version.

Model explanation and incident explanation are separate products. A feature attribution can explain a network classifier while leaving unanswered why two devices were connected into one incident. The incident explanation must add entity joins, chronology, matched patterns, evidence identifiers, signal availability and score construction. This distinction is central to the dashboard design.

2.4 Existing Systems, Products and Techniques

The relevant alternatives include independent alert lists, fixed correlation rules, graph-based investigation systems and integrated explainable dashboards. Independent alert lists are simple to implement but require the analyst to reconstruct relationships manually. Fixed rules are inspectable but can be brittle under changing telemetry. Graph methods express relationships clearly but depend on reliable identity and adequate collection coverage. Explanation interfaces improve transparency only when the displayed explanation is faithful to the actual computation.

Approach	Useful capability	Limitation for this component
Independent alerts	Preserves producer decisions with low integration complexity.	Does not establish an incident path or resolve duplicate evidence.
Rule-only correlation	Makes matching conditions explicit.	Requires maintained patterns and may miss novel sequences.
Provenance graph	Represents dependencies and supports investigation [2], [3], [4], [5].	Fine-grained provenance is not guaranteed by every producer.
Model attribution	Explains a compatible model output [6], [7].	Does not explain the complete fusion and entity-linking decision.
Component 4 hybrid design	Combines bounded correlation, model evidence and an incident interface.	Effectiveness and usability remain incompletely evaluated.

Table 2.1. Qualitative comparison of relevant approaches.

Table 2.1 compares capabilities at the level of design, not benchmark performance. No vendor-specific effectiveness claim is inferred. The appropriate baseline for a future experiment is an implementation that uses the same event stream and ground truth, so improvements can be attributed to correlation rather than to different telemetry.

NIST's incident-response guidance treats incident handling as part of broader cybersecurity risk management [14]. For this prototype, the practical implication is to support investigation state and evidence retention rather than equating alert generation with completed response. The implementation does not constitute a complete organisational incident-response programme.

2.5 Gap Synthesis and Conceptual Framework

The conceptual framework links five concepts: trustworthy event representation, bounded relational correlation, explicit fusion, traceable explanation and analyst review. A failure in any earlier concept constrains later conclusions. A polished explanation cannot correct an incorrect asset join, and a high fusion score cannot replace missing evidence of movement.

The proposed unit of research evaluation is the incident, with event-level results retained as supporting measures. This choice aligns the experiment with the

component's purpose. It also requires explicit matching rules between a predicted incident and a ground-truth scenario. Otherwise, repeated alerts about the same attack could artificially increase apparent detection success.

NIST's explainability principles emphasise explanation, meaningfulness, explanation accuracy and knowledge limits [15]. In this component, those ideas translate into showing the evidence used, adapting detail to the analyst's task, preserving the actual scoring path and identifying absent or incompatible inputs. This is a design interpretation, not a claim of certified compliance.

The resulting research gap is measurable: whether an evidence-linked fusion and dashboard pipeline offers better incident-level outcomes than simpler baselines under equivalent telemetry. The current implementation establishes an artefact suitable for that evaluation; it does not itself establish the outcome.

3 RESEARCH METHODOLOGY

3.1 Research Design

The study follows an artefact-centred engineering approach: define the problem, specify requirements, inspect and construct the solution, verify implementation properties and evaluate the resulting behaviour. The thesis combines documentary analysis of the supplied source with limited reproducible verification. It also specifies the experiments needed to complete the empirical research claim.

Three evidence categories are used. Category A is direct inspection of source, configuration, schemas and stored artefacts. Category B is an archived measurement or test output supplied with the project. Category C is a verification executed during thesis preparation in an isolated environment. Proposed evaluations are marked P and are never merged with completed results. These categories make the provenance of each conclusion explicit.

The method is retrospective with respect to the supplied implementation. It cannot reconstruct unrecorded development decisions or assign reliable completion dates to every file. Where design rationale is inferred from the code, the thesis states the rationale as an interpretation rather than an author interview. The overall project is not reimplemented or evaluated as a four-component system within this study.

3.2 Data, Participants or Study Materials

3.2.1 Project artefacts

The primary project material is the supplied Component 04 archive [1]. It contains Python modules, configuration, model metadata, audit snapshots, test scripts, a SQLite database and demonstration material. The integration guidance document [16] is secondary project documentation: it describes integration recommendations and reported context, but explicitly does not establish that a current four-component deployment passed all tests.

The required formatting PDF and individual-thesis structure document govern presentation [17], [18]. They are not experimental evidence. Selected source hashes

and exact inspected paths appear in Appendix A. This allows a future reader to distinguish the examined snapshot from later edits.

3.2.2 Benchmark data descriptions

UNSW-NB15 is an intrusion-detection dataset published by UNSW [19]. The project metadata records 175,341 training rows and 82,332 test rows. These figures describe the archived training materials; the raw datasets were not reprocessed for this thesis. The dataset's network orientation makes it relevant to the component's 37-feature network models, but it does not by itself represent a deployed IoMT hospital.

The archive also refers to Mordor/Security Datasets material, with 210,110 memory-oriented rows and 222,862 lateral-movement rows. Security Datasets provides security-event datasets for research and analysis [20]. These records must not be described as patient data or as evidence collected from a production hospital. Combining records from different sources requires careful schema and provenance treatment.

3.2.3 Participants and ethics boundary

No completed participant study, recruitment record, consent form or institutional ethics approval was supplied. Consequently, Chapter 6 reports no clinician satisfaction scores, analyst task-time results or qualitative interview themes. A prospective usability instrument is included in Appendix A for future approved use. It must not be mistaken for a completed study.

The newly executed verification used temporary SQLite databases and inspected source files. It did not connect to clinical systems, publish events to a live service or access patient records. The archived database was opened read-only for aggregate inspection. This narrow boundary limits both the operational risk and the claims supported by the verification.

3.3 Experimental or Study Procedure

3.3.1 Completed inspection procedure

The archive was extracted into an isolated working directory. Python source files were parsed to identify syntax validity and API routes. Model metadata and audit outputs were read as data rather than loading untrusted serialized estimators. The database schema and aggregate row counts were inspected, and a read-only integrity check was performed. This procedure establishes what the snapshot contains, not whether all runtime dependencies are compatible.

The existing database-resilience script was then executed against temporary databases using Python 3.12.14. Its test cases cover initialisation, SQLite settings, transaction commit and rollback, a simulated path failure, reopening, schema reinitialisation and fresh recovery. The original runtime database was absent at the time of that execution and the test confirmed that it was not created. The archived database was extracted separately afterwards for read-only inspection.

3.3.2 Prospective detection experiment

The proposed experiment begins with scenario-level ground truth prepared independently of detector outputs. Each scenario specifies participating assets, event identities, timestamps, legitimate activity and the target movement relationship. Positive scenarios should vary the sequence and visibility of suspicious activity. Negative scenarios should include legitimate administration, imaging-service transfers, unrelated simultaneous events and repeated deliveries.

Every system variant must receive the same immutable stream. The baseline variants are independent source alerts, graph-only correlation, temporal-only correlation, memory–network-only correlation and complete fusion. A separate complete-fusion variant without forecast support isolates the effect of forecast context. The threshold and time windows should be chosen on development scenarios and frozen before evaluation.

An incident match requires agreement on the relevant asset relationship and an allowed temporal overlap with ground truth. Each ground-truth incident can contribute at most

one true positive. Additional duplicate incidents are measured separately. This avoids rewarding a system for repeatedly alerting on the same sequence. The matching policy and any adjudication must be recorded before inspecting final results.

3.3.3 Robustness and integration procedure

Robustness trials should replay valid events with delayed arrival, duplication, missing optional fields, malformed timestamps, unavailable models and service interruptions. Each condition tests a different claim. Delivery retry tests examine idempotency; restart tests examine retained state; model incompatibility tests examine eligibility; and timestamp perturbations examine correlation sensitivity. Combining them into one success percentage would obscure the reason for a failure.

The staged API test supplied in the archive contains a network hop, a second network stage, memory evidence, temporal evidence and a teammate stage. Its archived output fails at the first stage because a final incident was produced earlier than expected. Reproduction should begin from an empty temporary database and empty in-memory state, preserve the exact request sequence and log all signal families after each stage. Only then can the failure be attributed to gating, fixture state or an outdated expectation.

3.4 Measures and Evaluation Metrics

3.4.1 Detection metrics

At incident level, precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is 2PR/(P+R), with undefined denominators reported explicitly. Accuracy may be reported only when the negative unit is clearly defined. For continuous monitoring, arbitrary segmentation of benign time into samples can change accuracy without changing operational usefulness.

False alerts per asset-hour, duplicate incidents per ground-truth incident and time to first supported detection are therefore important supplementary measures. Detection latency should be measured from the relevant event-time boundary and also from ingestion time, because collection delay is not the same as processing delay. The result table must state which latency is being reported.

3.4.2 Reliability and explanation metrics

Reliability measures include accepted-event count, unique persisted-event count, evidence-link integrity, successful retrieval after reopen, failed requests, recovery time and unresolved delivery attempts. Explanation completeness measures whether the incident includes the source records, time range, entity relationship, available signals, score construction and model eligibility state. Completeness does not measure faithfulness by itself.

Explanation faithfulness requires a separate check against the computation. For additive model explanations, the reported attribution and baseline should be reconciled with the relevant model output within a stated numerical tolerance. For incident explanations, every asserted relationship should map to evidence or be marked as an inference. Analyst task performance should include correct interpretation, completion time and confidence calibration rather than visual preference alone.

Measure	Unit	Interpretation boundary
Incident precision/recall/F1	Matched incidents	Requires independently labelled scenarios.
False-alert rate	Alerts per asset-hour	Requires a documented benign observation period.
Detection latency	Seconds	Must separate event delay from processing time.
Duplicate ratio	Extra incidents per true incident	Requires stable incident matching.
Explanation completeness	Required fields present	Does not prove explanation correctness.
Task accuracy and time	Correct decisions and seconds	Requires a completed participant protocol.

Table 3.1. Proposed evaluation measures and their boundaries.

Table 3.1 defines the proposed metrics. The archived network-model scores in Chapter 6 use a different evaluation unit and are not substituted for these incident-level measures.

3.5 Data Analysis

Completed analysis is descriptive. It compares source configuration, saved model metadata, stored database content and test outputs while preserving their separate origins. Numerical differences between archived model summaries are arithmetic comparisons, not statistical significance tests. No confidence interval is manufactured from rounded metrics without predictions, sample-level outcomes and a known evaluation design.

For the prospective experiment, scenario-level bootstrap intervals can quantify uncertainty where enough independent scenarios are available. Repeated events from one scenario should not be treated as independent trials. Paired comparisons should use the same scenario set for each variant. Results should be stratified by attack visibility, legitimate-administration activity, delayed delivery and missing telemetry rather than reported only as one aggregate number.

Error analysis should inspect false positives and false negatives at the evidence level. A false positive caused by a shared IP address requires an identity correction, whereas one caused by a short threshold may require calibration. These are different engineering problems. The evaluation log should therefore record the failed assumption and the evidence available at decision time.

3.6 Ethics, Privacy, Security and Responsible AI

The component handles security telemetry that can contain usernames, hostnames, command lines and operational relationships. Even without patient records, this information can be sensitive. The research design should minimise collected fields, pseudonymise identifiers in exported reports and restrict access to raw evidence. Retention should be justified by investigation and reproducibility needs rather than indefinite collection.

The dashboard must communicate uncertainty and avoid a misleading appearance of medical authority. A security risk score is not a patient-risk score, and a classifier probability is not necessarily calibrated. Automatic isolation of a clinical device would require operational authorisation and a separate safety assessment; it is not established by this prototype.

Responsible use also requires disclosure of limitations. Missing telemetry should be displayed as incomplete coverage, not as a healthy state. An explanation should distinguish observations from hypotheses and avoid attributing malicious intent to an individual solely from account activity. Analyst feedback should remain auditable, and changes to model training based on feedback should be reviewed rather than applied silently.

3.7 Reproducibility and Research Artefacts

Reproducibility requires more than a requirements file. A complete package should identify source revision, dependency resolution, model hashes, dataset versions, preprocessing, split manifests, random seeds, configuration and scenario streams. The archive provides some of these items, but not a complete experiment manifest. Appendix A records the inspected artefacts and describes the remaining requirements.

The supplied dependency file names Python 3.13, while the isolated verification used Python 3.12.14. Only the database script and syntax inspection were executed in the latter environment. The full API, dashboard, trained models and SHAP/LIME runtime were not rerun. This distinction prevents a local verification result from becoming an unsupported compatibility claim.

The source listings in the appendix are selected from the supplied implementation and reformatted for readability using Python's abstract syntax tree. Comments and blank lines are omitted; original file and function names remain identified. The listings support inspection and are not a replacement distribution of the runnable project. The original source snapshot remains the authoritative implementation evidence.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Functional requirements

Component 4 must accept events from supported sources, preserve their origin, normalise asset and time fields, and decide which analytical paths are eligible. It must retain enough context to associate activity without inventing missing evidence. When an incident is created, its stored representation must identify the evidence and signal contributions used at that time.

ID	Requirement	Verification approach
FR1	Preserve source identity and original event reference.	Compare input, normalised event and stored record.
FR2	Route only compatible inputs to each saved model.	Missing-feature and wrong-feature-order cases.
FR3	Correlate asset relationships within explicit windows.	Positive and unrelated-event scenarios.
FR4	Keep forecasts distinct from observed evidence.	Forecast-only and mixed-input cases.
FR5	Create or update a traceable incident.	Stable ID, evidence links and occurrence checks.
FR6	Expose evidence, score construction and explanations.	Incident-detail and XAI retrieval checks.
FR7	Record analyst feedback with history.	Status transition and feedback persistence checks.
FR8	Report health and unavailable dependencies.	Controlled degradation and recovery cases.

Table 4.1. Functional requirements and verification approaches.

Table 4.1 is a requirements traceability baseline. It is not a claim that every acceptance test has passed. Chapter 6 identifies the portions supported by completed evidence.

4.1.2 Non-functional requirements

The component should be understandable, recoverable and maintainable on the available research hardware. Performance requirements must be measured under a declared event rate and resource budget. The archive does not provide a validated

hospital-scale throughput target, so no numerical service-level guarantee is invented here.

Security requirements include authenticated publishers, authenticated analysts, least-privilege access, transport protection when traffic leaves a trusted local boundary and auditability of feedback. The inspected API file does not establish an authentication or role-authorisation implementation. CORS configuration controls browser-origin behaviour; it is not a substitute for authentication. These requirements remain deployment-hardening work.

4.2 Component Architecture within the Complete Solution

The architecture contains an ingestion boundary, a normalisation and eligibility stage, parallel analytical mechanisms, a fusion stage, a persistence layer and an API/dashboard presentation layer. Figure 4.1 shows these relationships. Upstream components provide observations or forecast context, but Component 4 owns the incident representation and its explanation within the scope of this thesis.

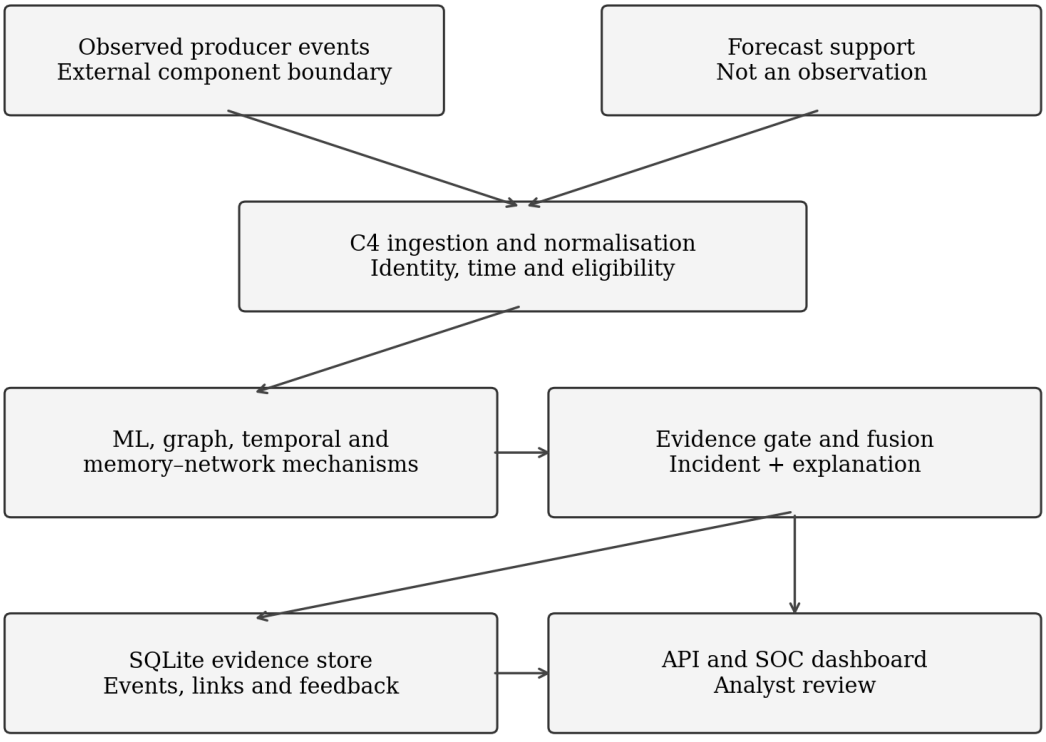


Figure 4.1. Component 4 architecture and evidence boundaries.

The orchestrator coordinates event processing and recent signal state. It is the appropriate place to apply common policy because otherwise each API route might implement a different interpretation of sufficient evidence. The fusion engine calculates incident-level contributions and deduplication behaviour. Persistence stores events, alerts, evidence links, explanations and feedback, allowing later retrieval without requiring the browser to maintain the incident history.

The architecture should avoid feeding a fused Component 4 incident back into its own observed-evidence input. Such a loop would allow a conclusion to become support for itself. Similarly, a forecast derived from upstream observations must retain its origin so that the same data are not counted again as independent confirmation.

4.3 Design Alternatives and Decisions

4.3.1 Central incident service versus tightly coupled models

A central incident service allows each producer to retain its own inference lifecycle and resource requirements. Component 4 consumes a documented result rather than importing every producer's model into its dashboard. This reduces coupling and makes producer failures visible at a boundary. It also requires a reliable event contract and delivery policy, which must be evaluated rather than assumed.

An alternative is one monolithic process containing all models and interfaces. That design can simplify a small demonstration but makes memory use, failure isolation and dependency compatibility harder to reason about. The integration guidance recommends keeping services independent [16]. The inspected Component 4 source is consistent with an API boundary, but a fully reliable producer outbox and delivery ledger are not demonstrated by the available evidence.

4.3.2 Weighted fusion versus learned incident fusion

The inspected fusion rule is a weighted combination over available signals plus an agreement bonus. Its advantage is inspectability: the dashboard can show the active weights and contributions. Its limitation is that the weights and bonus are not demonstrated to be statistically calibrated. A learned fusion model could capture

interactions but would require a suitable incident-labelled dataset and additional explanation work.

The present thesis retains the implemented rule as a baseline artefact. It does not reinterpret the output as a calibrated attack probability. A future comparison should evaluate fixed weights, calibrated weights and simple logical rules on identical scenarios. The evidence gate must remain conceptually separate from the score because a high score with insufficient relational evidence should not establish a movement incident.

4.3.3 SQLite and bounded in-memory state

SQLite provides a practical single-node store for a research prototype. The supplied database manager enables foreign-key enforcement and write-ahead logging, as verified by the isolated test. These settings support integrity and transactional behaviour, but they do not by themselves establish high-availability service operation or multi-node consistency. SQLite's official documentation describes foreign-key enforcement and WAL behaviour [21].

The active correlators also use recent in-memory state. This supports bounded processing but creates a recovery question: reopening persistent incident records is different from rebuilding the recent event windows needed to continue correlation. The design should either rehydrate an appropriate event interval on startup or explicitly declare a cold-start coverage gap.

4.4 Detailed Design

4.4.1 Unified event and provenance

A unified event should contain an event identifier, producer identity, layer, event time, source and destination, event type, process or session context where available, indicators and raw evidence references. The inspected schema also provides feature and detail containers. These fields permit different producers to share a common envelope without pretending that their internal feature vectors are interchangeable.

Canonicalisation must be conservative. Trimming identifiers and normalising known aliases can improve joins, but merging two devices solely because they share a current IP can create false paths. A robust asset mapping should retain historical address intervals and the reason for each mapping. This is a design requirement beyond a simple string comparison.

Time handling must distinguish event time, ingestion time and forecast generation time. A naive local timestamp must not be labelled UTC without conversion. When the original timezone is unknown, the uncertainty should be preserved or the input rejected under a documented policy. The source contains timestamp parsing helpers, but broad clock-skew tolerance has not been experimentally characterised.

4.4.2 Model eligibility and feature spaces

The archive distinguishes three feature spaces. XGBoost and Random Forest use 37 UNSW-NB15-style network features. Isolation Forest uses six memory/host features. A separate FeatureExtractor creates twelve contextual features for correlation logic. These representations serve different purposes and must not be substituted for one another merely because they are numeric.

The orchestrator includes compatibility checks before ML inference. If a live event lacks the complete network vector, it may still contribute to graph or temporal reasoning. This is preferable to fabricating default feature values that make the model execute but change the meaning of its input. The dashboard should expose the reason a model was unavailable so that an analyst does not interpret the missing score as benign evidence.

The training metadata mentions a broader Mordor feature-selection stage, while the fitted-model audit identifies six memory inputs. These statements can describe different pipeline stages, but the supplied summary is insufficient to establish their exact relationship. The fitted input contract is the immediate runtime constraint; the training lineage requires further documentation.

4.4.3 Graph correlation

The graph mechanism represents devices and observed relationships while retaining a hospital baseline for context. A candidate path is evaluated within a bounded temporal window and maximum hop count. The supplied defaults are a ten-minute correlation window and four hops. Baseline nodes and edges are context, not attack observations, and must not inflate live-event counts.

Path interpretation requires more than topological connectivity. A chain A→B→C may join unrelated sessions unless event order, time, service and identity are considered. The implementation provides path evaluation and scoring methods, reproduced selectively in Appendix A. Their presence supports an implementation claim; their false-positive behaviour requires controlled negative scenarios.

The graph display should label observed edges separately from baseline relationships and inferred paths. It should also let an analyst inspect the event identifiers behind a selected relationship. Where the source lacks destination execution or authentication evidence, the interface should describe a suspected movement path rather than confirmed compromise of every node.

4.4.4 Temporal correlation

Temporal correlation groups events by relevant entities and seeks patterns within a configured window. It can connect individually ambiguous actions into a more meaningful sequence. However, temporal adjacency alone does not establish causality. The matching logic should require an explicit relationship and preserve the matched records so that an analyst can assess alternatives.

Late and out-of-order events complicate this design. If the component uses arrival order as a substitute for event order, a delayed record may produce a different result from the same scenario delivered normally. The prospective protocol therefore replays identical scenarios with controlled delay. A production design should define its allowed lateness, watermark or re-evaluation policy explicitly.

The archived runtime snapshots report no active temporal patterns in that recorded state. This does not prove that the source has no pattern definitions; it describes one

runtime output. Configuration, runtime activation and observed matches must remain separate in the result interpretation.

4.4.5 Memory–network association

The memory correlator associates suspicious memory or host activity with network behaviour on related entities. Its default window is 300 seconds. It has separate paths for a network event arriving after memory evidence and memory evidence arriving after network activity. This is important because collection order can differ from event order.

A defensible association should identify the asset and, where available, the process or session responsible for the network action. A host-wide anomaly associated with every outgoing connection would overstate precision. The incident explanation should therefore state the granularity of the match and distinguish a host-level temporal association from a process-level relationship.

The source includes duplicate suppression and event summaries. These improve presentation only if they retain the underlying evidence identities. Suppressing repeated notifications should not delete the fact that an incident accumulated additional observations. The stored incident should preserve occurrence information and first/last observation times.

4.4.6 Fusion and evidence gates

The configured weights are 0.30 for ML, 0.25 for graph, 0.25 for temporal, 0.15 for memory and 0.05 for teammate evidence. For available signals A, the base score is the sum of weight times score divided by the sum of available weights. Missing signals are excluded from both numerator and denominator. A real evaluated zero remains included.

The agreement bonus is 0.03 for two observed families, 0.07 for three, and 0.10 for four or more; otherwise it is zero. The resulting score is capped at one. The default alert threshold is 0.45. These are implementation constants, not experimentally established optimal values. Their influence should be evaluated using sensitivity analysis and ablation.

Figure 4.2 separates eligibility, evidence gating and scoring. The orchestrator normally requires at least three observed signal families before calling fusion. The fusion class has an additional internal sufficiency check that can accept a structured graph, temporal or memory correlation result. These checks operate at different boundaries. Calling the fusion class directly is therefore not equivalent to following the normal orchestrator path.

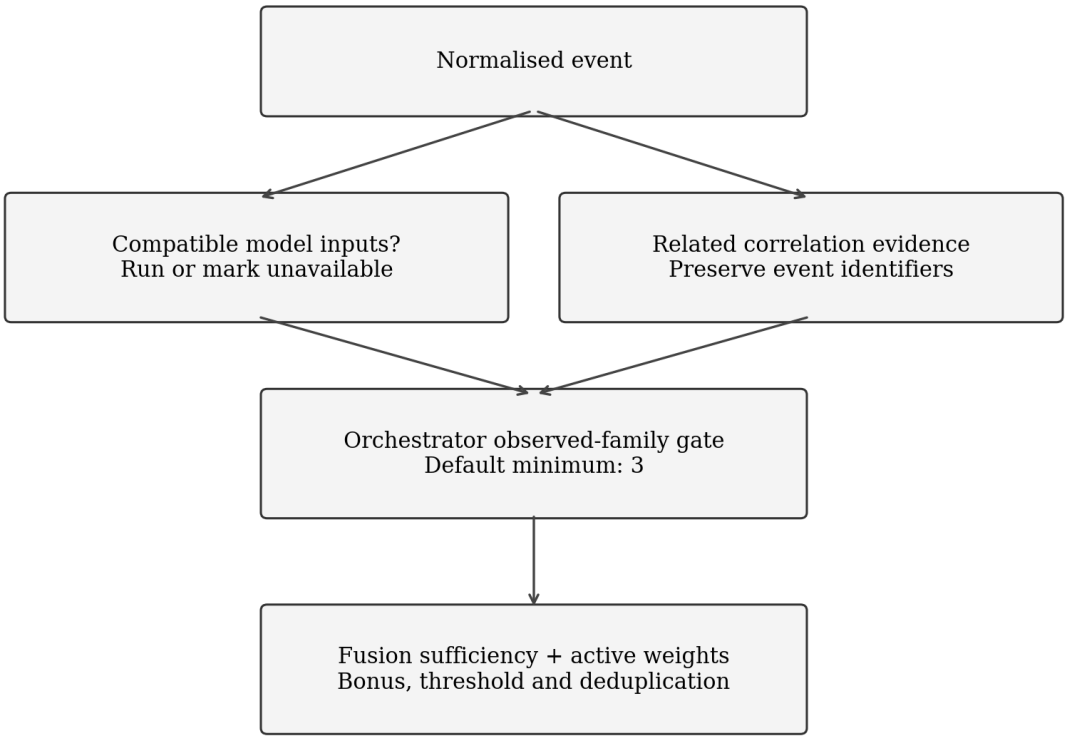


Figure 4.2. Eligibility and fusion decision boundaries.

Forecast support is recognised separately from observed teammate evidence. A forecast alone should not generate an observed lateral-movement incident. Nevertheless, signal families can share evidence, so the existing gate should not be described as a proof of statistical independence. A future gate should count distinct supporting observations and document overlap between mechanisms.

4.4.7 Incident lifecycle and persistence

The schema separates events, alerts, alert-evidence links, XAI explanations and analyst feedback. This supports a one-to-many relationship between an incident and

its evidence, while feedback can record changes over time. Unique event identifiers and unique alert–event associations are useful foundations for idempotency, although the complete ingestion path still needs duplicate-delivery testing.

Figure 4.3 depicts the persistence relationships. An explanation should be tied to the incident version or evidence state that produced it. Otherwise, an updated incident could display an old explanation without making the mismatch visible. The current schema and update logic provide a starting point, but a versioned explanation contract is a recommended extension.

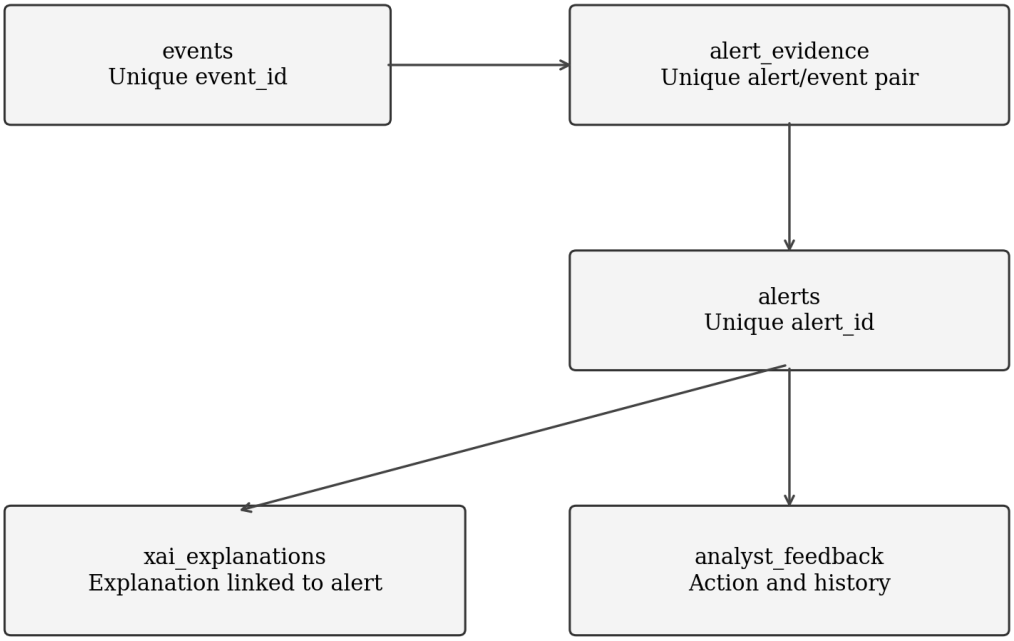


Figure 4.3. Persistent incident, evidence and feedback relationships.

An analyst's false-positive decision should not erase source evidence. It changes the investigation interpretation and may inform later model governance. Keeping the original score, evidence links and feedback history permits review of both the detector and the analyst decision.

4.4.8 SOC and restricted clinical presentation

The SOC view should prioritise incident severity, age, affected assets, status, evidence coverage and ownership. An incident detail view should contain the reconstructed path, event timeline, raw evidence references, signal scores and effective weights, model eligibility, explanation provenance, ATT&CK mappings and analyst notes. Unavailable or stale data should be visible alongside the result.

A clinician-facing summary should use plain language, show only the relevant service status and provide an escalation route. It should not expose command lines, attack graphs or unreviewed technical conclusions by default. This is a presentation requirement for a future restricted view, not evidence that a complete doctor portal or role-based access system has been implemented.

The dashboard should not equate a successful page refresh with healthy data ingestion. It needs separate timestamps for the latest event, latest producer contact and latest UI update. A cached incident can remain useful during an outage, but the interface must state that it is cached. This distinction is particularly important when a graph still displays baseline assets despite an empty live event window.

4.4.9 Explanation contract

Every explanation should answer: what triggered the incident, which assets are involved, which observations support the relationship, how the score was constructed, what was unavailable and what remains uncertain. Model attribution is a subordinate explanation attached only to the model and instance that generated it. Incident-level prose must not imply that SHAP explains the graph or temporal rule unless that relationship is explicitly implemented.

ATT&CK technique labels are analyst aids. They should be accompanied by the matched evidence and the mapping rule, not displayed as authoritative proof. A mapping to remote services can guide investigation while leaving open whether the activity was authorised. The same principle applies to process-injection or credential-related labels derived from incomplete telemetry.

An explanation should favour testable statements. For example, it can state that two records share an asset and occur within the configured window. It should not silently upgrade that observation into a causal claim. Where an inference is useful, it should be labelled and linked to the missing evidence that would confirm or weaken it.

4.5 Standards and Professional Practice

The design uses structured API schemas, explicit timestamps, durable identifiers and traceable evidence as professional engineering practices. It draws on ATT&CK terminology [8], NIST incident-response guidance [14] and explainability principles [15] without claiming formal compliance or certification. The project remains a research prototype whose deployment controls require further work.

Commercialisation would require more than packaging the dashboard. A viable Component 4 offering would need supported integrations, identity and access management, retention controls, upgrade and rollback procedures, observability, support ownership and an evidence-based operating envelope. No market survey, cost-benefit study, intellectual-property assessment or customer validation was supplied. These remain future product activities rather than completed research results.

The likely value proposition is a traceable incident workspace that reduces the effort of relating heterogeneous alerts. That proposition should be tested with analyst tasks and integration maintenance costs before making financial claims. A component-level licence or service would also need clear boundaries concerning upstream data quality and clinical response authority.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The source is organised into API, collection, configuration, preprocessing, engine, database, dashboard and XAI modules. FastAPI and Pydantic define the service boundary; Dash, Plotly and Cytoscape support the interface; NetworkX supports graph processing; and SQLite provides persistence. The machine-learning stack includes XGBoost, scikit-learn, SHAP, LIME, NumPy, pandas and joblib [1].

The supplied requirements file pins versions including FastAPI 0.136.0, Dash 4.1.0, XGBoost 3.2.0, scikit-learn 1.8.0 and SHAP 0.51.0. These are recorded requirements, not a verified installation report. Model binaries can depend on library versions, so reproducibility requires a resolved environment and successful load/inference checks in addition to version strings.

The inspection environment used Python 3.12.14. Thirty-three Python files parsed successfully. Parsing detects syntax problems but does not execute imports, validate dependency compatibility or test behaviour. The database test used only its relevant local modules and temporary storage. No end-to-end performance claim follows from these checks.

5.2 Component Implementation

5.2.1 Ingestion and normalisation

The API exposes separate network, memory and host ingestion routes, plus a teammate-alert route. Pydantic models validate required text and numeric ranges for selected fields. The normaliser converts supported source formats into a shared representation while retaining details and raw data. This design provides a common processing path while allowing source-specific extraction.

The request model's acceptance criteria should not be confused with full semantic validation. A non-empty address string, for example, is not necessarily a verified asset identity. A timestamp accepted as text may still require careful parsing. Appendix A provides a field-level contract audit and proposed negative tests for these distinctions.

5.2.2 Orchestration and signal state

The orchestrator processes the normalised event, evaluates relevant engines, associates recent signals with event entities and counts observed signal families before fusion. It maintains recent results for different mechanisms and cleans state according to configured windows. The selected implementation listings show the actual compatibility, association and gating functions.

The central coordination is useful because it keeps model eligibility separate from event acceptance. An event can be useful for graph reasoning even when it lacks the 37 network features required for the supervised models. This partial-processing behaviour should be visible in the ingestion response and incident explanation.

5.2.3 API retrieval and feedback

Method	Route	Component 4 purpose
GET	/api/health	Reports service and dependency health.
GET	/api/stats	Returns component summary statistics.
GET	/api/events	Retrieves event records.
GET	/api/alerts	Retrieves incident summaries.
GET	/api/alerts/{alert_id}	Retrieves a selected incident.
GET	/api/xai/explain/{alert_id}	Retrieves explanation material.
GET	/api/graph	Retrieves graph representation.
POST	/api/events/network	Accepts a network event.
POST	/api/events/memory	Accepts memory-oriented evidence.
POST	/api/events/host	Accepts host activity.
POST	/api/integration/teammate-alert	Accepts upstream component output.
POST	/api/alerts/{alert_id}/feedback	Records analyst action or feedback.

Table 5.1. Component 4 analytical API routes.

Table 5.1 is derived from route decorators in the supplied source. The root route is omitted because it is not an analytical interface. The existence of a route does not establish authentication, production availability or successful producer integration.

5.2.4 Dashboard implementation boundary

The inspected dashboard is a Dash application with an API client and simulation support. Its source includes incident and explanation presentation logic. The thesis evaluates this architecture and its data contract; it does not claim that a new animated login, redesigned landing page or clinician portal was implemented during manuscript preparation.

The dashboard should request incident details through stable identifiers and avoid rerunning models merely to refresh a view. Displayed counts should state whether they describe current in-memory activity, persistent history or baseline assets. Mixing these sources can create apparent contradictions, especially immediately after a restart.

5.3 Integration with Other Components

5.3.1 Observed producer evidence

The integration document describes Component 1 and Component 2 as sources of observed detections and Component 3 as forecast context [16]. Component 4 must preserve their different semantics. An upstream score should include its scale and evaluated unit, such as a flow, capture window or script. A capture-level score must not be copied onto every connection as if each had been independently classified.

Adapters should translate producer fields into the existing ingestion schema and retain original metadata. If an upstream producer labels application activity as a network layer, the adapter needs a documented mapping rather than an implicit reinterpretation. The same principle applies to confidence values expressed on different scales.

5.3.2 Forecast support

A forecast should identify its target, generation time, input interval, horizon and completeness. Its value is contextual: it can prioritise attention or accompany an observed incident. It does not establish that the predicted behaviour has occurred. The source distinguishes forecast alerts by layer, component identity or event type.

Forecast freshness must be checked independently of incident age. A recent incident can be accompanied by an old forecast, and a fresh forecast can be based on incomplete

observations. The dashboard should expose these facts. A future evaluation should compare complete fusion with and without forecast support while preserving the same observed inputs.

5.3.3 Delivery and identity integration

Cross-machine integration requires a reachable address, an appropriate listener, controlled network access and authenticated publishers. The default API host in the source is loopback. That is suitable for a local process boundary but not evidence that another machine can reach the service. Changing the bind address is an operational step that must be accompanied by access controls.

A reliable delivery design should preserve the producer's event identifier across retries and record pending, acknowledged and failed deliveries. Such a contract enables idempotency and replay. The source's database uniqueness constraints help, but the archive does not establish a complete durable outbox implementation across all producers.

Asset identity must follow the actual deployment. A single physical laptop hosting several services should not be displayed as several independent devices merely because examples use multiple role names. Conversely, a shared address must not collapse several assets without a justified mapping. Integration testing should include these ambiguity cases.

5.4 Verification and Software Testing

5.4.1 Completed checks

The isolated database-resilience script passed all twenty printed assertions across nine test sections. It verified temporary database creation, required tables, foreign-key and WAL settings, commit behaviour, rollback after an intentional failure, health detection and recovery for an invalid path, reopen persistence, idempotent initialisation and fresh schema recovery. The safety assertion confirmed that the test did not create the real runtime database.

After separate extraction, the archived SQLite database returned “ok” for the integrity check and contained 18 events, one alert, five alert-evidence links, one XAI explanation and one feedback record. This is a consistency observation about one file. It does not establish that all stored evidence is correct, that every intended event was captured or that the incident is a true positive.

5.4.2 Archived integration failure

The supplied integration log records a reachable API followed by a failure at Stage 1, the SMB lateral network hop. The assertion expected no final incident, but the actual response indicated that a final incident had been created. This failure is relevant to the sufficient-evidence requirement and must remain visible in the thesis.

The available log does not include enough state to isolate the cause. Possible explanations include residual state, a difference between test expectations and current policy, or a defect in signal counting or correlation. These are hypotheses, not findings. A clean-state replay with per-stage evidence IDs and family counts is necessary before selecting a correction.

5.4.3 Test coverage limitations

The API resilience and model resilience scripts were inspected but not executed during thesis preparation. The full service stack was not started, and no live producer was connected. Database verification therefore cannot be used to claim successful end-to-end detection, explainability fidelity, UI usability, concurrency robustness or authentication enforcement.

The test catalogue in Appendix A separates completed, archived and planned cases. This makes it possible to extend verification without overwriting the existing evidence. A future report should preserve both failed and successful runs with their configuration and source revision.

5.5 Deployment and Operation

The recommended research deployment is an isolated, authorised environment with a documented asset inventory, pinned dependencies, model hashes and explicit producer

endpoints. Before replay, the operator should record configuration and decide whether the run starts from empty state or a restored window. That choice directly affects incident creation and duplicate handling.

Health reporting should distinguish API reachability, model availability, database availability, producer freshness and dashboard connectivity. A single green status can conceal a missing analytical dependency. The source's health endpoint provides a basis for differentiated health, but the operational meaning of each status should be validated under controlled failures.

A restart procedure should preserve persistent records and explicitly handle recent correlation state. If rehydration is not implemented, the dashboard should show a cold-start interval. Backups should be tested by restoring to a separate location rather than overwriting the working database. Operational changes should retain the previous configuration so that changed results can be explained.

6 RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Archived supervised-model summaries

Model	Accuracy	Precision	Recall	F1
XGBoost, 37 network features	0.9018	0.8633	0.9761	0.9162
Random Forest, 37 network features	0.9112	0.8830	0.9669	0.9231

Table 6.1. Archived supervised-model metrics, not reproduced.

Table 6.1 transcribes the supplied training metadata dated 4 May 2026 [1]. These are historical model summaries, not newly reproduced results. The metadata labels the Random Forest as “combined”, but the fitted-model audit identifies 37 network features. It must not be described as a trained cross-layer incident classifier solely from that label.

The Random Forest's recorded accuracy is 0.0094 higher and its F1 is 0.0069 higher than the XGBoost summary, while XGBoost's recorded recall is 0.0092 higher. These arithmetic differences do not establish statistical significance or superior deployment performance. Sample predictions, confidence intervals and a verified evaluation manifest were not available for this analysis.

Most importantly, neither row measures the complete lateral-movement correlation engine. The unit is the underlying classifier evaluation, not a matched multi-event incident. The thesis therefore does not claim that Component 4 detects lateral movement with 91.12% accuracy or 92.31% F1.

6.1.2 Archived data and anomaly summaries

The metadata records 119,341 malicious and 56,000 normal training samples before resampling, followed by 119,341 samples in each class and 238,682 in total. This indicates that the normal class was the smaller class in the recorded training subset. It would be misleading to state generically that the procedure oversampled malicious examples in this run.

The Isolation Forest summary records 432,972 training rows, contamination 0.10 and 7,109 anomalies, described as approximately 1.6%. The arithmetic ratio is about 1.64%. The difference between the contamination parameter and the reported anomaly proportion cannot be resolved from the summary alone; they may refer to different stages or evaluated subsets. No anomaly-detection precision or recall can be derived without labels and predictions.

The SHAP metadata includes 500 explained samples and a reported top feature, ct_dst_sport_ltm, with value 2.4473. Without the complete aggregation definition and output-space context, that number should not be interpreted as a universal causal importance or a percentage contribution to every incident. The thesis records the existence of explanation metadata but does not claim validated global explainability.

6.2 Product and Technical Results

6.2.1 Implemented capabilities

Source inspection confirms modules for event normalisation, feature eligibility, graph correlation, temporal matching, memory–network association, score fusion, persistent evidence links, XAI support and a Dash interface. Thirty-three Python files parsed successfully. These findings establish implementation coverage in the supplied snapshot, subject to runtime testing.

The fusion implementation exposes available-signal normalisation and effective weights, allowing an explanation to describe the actual active combination. The orchestrator applies a three-family gate in the normal path. This separation is a useful architectural property, although the archived integration failure prevents an unconditional claim that premature alerts have been eliminated.

6.2.2 Persistence results

Evidence source	Events	Alerts	Interpretation
Archived SQLite file inspected read-only	18	1	One stored snapshot; five evidence links, one explanation and one feedback row.

Evidence source	Events	Alerts	Interpretation
before_restart.json	68	14	Saved runtime report; all alerts reported critical.
after_restart.json	68	14	Same recorded persistent totals as the preceding report.

Table 6.2. Separate persistence evidence snapshots.

Table 6.2 keeps the database file and runtime reports separate. The snapshots also report zero processed events, zero in-memory events and zero active graph connections in the runtime portion, while retaining baseline graph context. Identical before/after totals are consistent with retained records, but they do not independently demonstrate recovery of active correlation windows.

The new database verification passed twenty assertions across nine sections. This provides reproducible support for selected database-manager properties in the inspection environment. It does not reconcile the separate snapshot counts, and it does not prove that the API's entire transaction sequence is atomic under concurrent ingestion.

6.2.3 Evidence completeness

The stored alert–evidence and explanation tables demonstrate that the implementation has a persistence mechanism for investigation context. Row existence is weaker than semantic completeness. A future audit must check that every displayed path edge and narrative assertion maps to the correct records and that explanation updates follow incident updates.

The current result is therefore an implemented evidence structure with limited verified persistence behaviour. It is not a completed validation of explanation faithfulness, attack-path correctness or analyst comprehension.

6.3 User or Stakeholder Validation

No completed stakeholder validation is available in the supplied materials. There are no verified participant counts, task-time measurements, questionnaires or interview transcripts supporting a usability result. Claims that doctors found the portal easy to

use or that SOC analysts investigated faster would therefore be invented and are excluded.

The proposed validation in Appendix A uses realistic analyst tasks: identify the initiating asset, distinguish forecast support from observation, inspect the evidence behind a path, recognise an unavailable model and record a justified disposition. A restricted clinical-view task would ask users to identify service status and the escalation route without requiring security terminology. These tasks remain future work subject to the appropriate institutional process.

Until that evaluation is completed, “user friendly” is a design objective rather than a measured outcome. The thesis can assess whether information is present and logically organised, but it cannot infer human performance from source code alone.

6.4 Integration Results

The source exposes a teammate-alert ingestion route and normalisation logic for upstream component outputs. This supports the claim that an integration boundary exists. The integration guidance describes how observed producer evidence and forecast context should be connected [16], but it is not a complete execution record of the current four-component deployment.

The strongest supplied automated API result is mixed: the health pre-check passed, then the staged scenario failed because an incident appeared too early. No later successful rerun with the same source revision and clean-state manifest was established during this study. The appropriate conclusion is that integration mechanisms are implemented but end-to-end acceptance remains unresolved.

This result does not invalidate every module. It identifies a specific gap between expected staged behaviour and an observed response. Resolving that gap requires preserving evidence at each processing stage, not merely changing a threshold until a demonstration appears successful.

6.5 Summary of Findings

Objective	Supported finding	Remaining validation
O1	Unified normalisation and typed ingestion exist.	Full producer contract and identity ambiguity tests.
O2	Multiple correlators and explicit fusion are implemented.	Clean-state incident trials and ablations.
O3	Model/XAI modules and incident explanation storage exist.	Faithfulness, completeness and analyst interpretation study.
O4	Database manager passed isolated resilience checks.	Full-service restart, concurrency and replay testing.
O5	Metrics and evaluation instruments are specified.	Incident-labelled detection and stakeholder experiments.

Table 6.3. Objective achievement and remaining validation.

Table 6.3 summarises achievement without converting incomplete evaluation into success. The evidence supports a substantial implemented prototype and selected technical verification. It supports only preliminary research validation of the complete correlation-and-dashboard contribution.

7 DISCUSSION

7.1 Interpretation of Findings

The principal finding is architectural: the component provides a path from heterogeneous observations to a persistent incident with explanation material. This is more informative than a disconnected alert list because it can preserve relationships, score contributions and analyst feedback. The practical benefit remains a hypothesis until measured against an equivalent baseline.

The strongest executed evidence concerns database behaviour. The weakest evidence concerns incident-level effectiveness and human interpretation. This imbalance is common in implementation-heavy projects but must not be hidden. A reliable database can preserve an incorrect incident faithfully; therefore persistence success and detection validity are complementary rather than interchangeable properties.

The archived premature-alert failure is particularly important because it tests the meaning of cross-layer sufficiency. A system may contain several analytical mechanisms yet still overcount evidence or carry state from an earlier run. The next research step should examine evidence lineage and state isolation before tuning model weights.

7.2 Comparison with Literature, Baselines and Existing Solutions

The component shares the literature's emphasis on relational context but operates with a more heterogeneous and sometimes coarser evidence boundary than whole-system provenance research. This affects the strength of its conclusions. When a producer supplies only an alert summary, Component 4 can correlate that assertion with other records, but it cannot reconstruct unobserved process dependencies.

Compared with an independent-alert baseline, the proposed advantage is an explicit incident representation and investigation trail. Compared with a single correlation mechanism, the proposed advantage is complementary evidence. Neither advantage has been quantified with incident-labelled controlled trials. The literature comparison therefore remains conceptual and does not present a benchmark league table.

Explainability methods also need contextualisation. A SHAP plot attached to an incident may be valid for one model while saying little about the entity joins that made the incident possible. The component's explanation design must make this distinction visible. A useful evaluation would test whether analysts can correctly identify which parts of the incident are model-supported, rule-supported or uncertain.

7.3 Research Contribution

The research contribution is the explicit integration of model eligibility, bounded relational correlation, evidence gating and an incident explanation contract within Component 4. It offers a concrete artefact that can be inspected and evaluated. The thesis also provides a reproducibility-oriented framework for separating implementation evidence from archival model scores and prospective research claims.

The contribution is bounded. The available work does not establish a new general theory of fileless malware, a universally optimal fusion algorithm or validated clinical deployment. Its academic value lies in the precise problem formulation, traceable design and critical analysis of evidence quality, together with an implemented prototype suitable for further experimentation.

The evidence audit identifies a useful future research direction: provenance-aware corroboration. Counting mechanisms is simpler than counting independent observations, but the distinction matters when several mechanisms consume the same records. A future model of evidence overlap could improve both gating and explanation without requiring every producer to share its internal implementation.

7.4 Product Contribution

The product contribution is a central SOC-oriented incident workspace backed by an ingestion and persistence service. It provides the structural basis for viewing alerts, paths, evidence, explanations and feedback in one place. This can reduce the integration burden for a research demonstration and create a clearer boundary between analytical services and presentation.

The product is not yet established as an operational hospital platform. Authentication, role enforcement, durable cross-service delivery, correlation-state recovery and

measured capacity require further work. A clinician-facing summary can be designed around the same incident service, but it should not expose the technical SOC workflow or imply that security scores are medical assessments.

The commercial value remains an untested proposition. Before deployment or commercialisation, the team should establish who maintains integrations, who authorises response, how telemetry is governed and what service conditions are supported. These responsibilities are part of product readiness, not optional details after model evaluation.

7.5 Limitations and Threats to Validity

7.5.1 Internal validity

The archive lacks a complete experimental chain linking raw data, preprocessing, split generation, training, predictions and final metrics. Leakage, repeated scenarios and inconsistent preprocessing cannot be ruled out from metadata alone. The historical classifier scores are therefore treated as reported values rather than independently reproduced estimates.

The integration failure has insufficient state to determine causation. A residual event window, a changed policy or a defect could produce the same visible symptom. The thesis avoids selecting one explanation without evidence. A clean-state experiment is required to distinguish them.

7.5.2 Construct validity

Several quantities can be mistaken for the intended research construct. Model accuracy is not incident detection effectiveness; an incident risk score is not a calibrated probability; explanation completeness is not faithfulness; and page responsiveness is not analyst usability. The proposed metrics separate these constructs to reduce misleading conclusions.

Likewise, a count of signal families is not a measure of independent corroboration. Shared evidence can increase multiple mechanism outputs. A more defensible

construct would measure distinct observations, entity relationships and supporting layers while documenting dependence.

7.5.3 External validity

The supplied datasets and demonstration artefacts do not establish representation of all IoMT healthcare environments. Asset types, network segmentation, clinical workflows and collection permissions vary. A model trained on benchmark network features may encounter a different distribution in a hospital. The thesis does not claim generalisation beyond the inspected prototype without further field evaluation.

The bounded windows also limit coverage of slow or interrupted movement. A ten-minute graph window and five-minute memory association may be appropriate for selected scenarios but can miss relationships outside those bounds. Expanding windows may increase memory use and false joins, so the trade-off must be measured.

7.5.4 Reliability and reproducibility

Different archived snapshots contain different record totals, and no complete revision manifest establishes their sequence. Treating them as one run would undermine reliability. This thesis instead reports them separately and records hashes for selected artefacts.

The newly executed tests cover only a narrow subsystem under Python 3.12.14. The pinned full stack was not installed or exercised. The thesis therefore cannot guarantee that every supplied dependency combination, model artefact or dashboard callback works in the student's current environment.

7.6 Implications

For research, the implication is that evidence quality should be evaluated before numerical fusion is optimised. A well-calibrated score cannot correct an incorrect asset join or duplicate observation. The next experiment should therefore combine negative identity cases, delayed events and evidence-overlap analysis with conventional detection metrics.

For engineering, the implication is that the incident explanation must be generated from the same stored evidence state used by the decision. Explanation text assembled from current dashboard state can drift from the original alert. Versioned explanations and immutable evidence references would make retrospective investigation more defensible.

For healthcare-oriented deployment, the implication is to preserve a human decision boundary. Technical analysts need detailed evidence and uncertainty, while clinical users need concise operational status and a safe escalation route. Neither audience should be asked to interpret an unexplained security score as a direct statement about patient safety.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This thesis examined Component 4 of the Hybrid AI-based Detection of Fileless Malware in IoMT Healthcare Systems project: a cross-layer lateral-movement correlation engine and XAI dashboard. The supplied implementation provides ingestion, normalisation, compatible model routing, graph and temporal correlation, memory–network association, fusion, persistence and explanation-oriented presentation.

The evidence supports the existence of a substantial prototype and selected reliable database behaviours. It does not support a claim of completed incident-level detection validation or demonstrated stakeholder benefit. Historical classifier metrics are reported with their provenance, an archived staged integration failure is retained, and conflicting snapshots are not combined into a fictitious result.

The central conclusion is that traceability is a necessary part of cross-layer detection. An incident should expose the observations, relationships and computation that support it, together with what is missing. The component establishes a foundation for this approach, while its empirical effectiveness remains an explicit target for further research.

8.2 Achievement of Objectives

O1 is supported by the implemented normalisation and API schemas, subject to complete producer-contract testing. O2 is supported at implementation level by the correlators and fusion path, but the premature-alert failure and absence of controlled incident trials limit effectiveness claims. O3 is supported by XAI modules and explanation storage, while explanation fidelity and user understanding remain unevaluated.

O4 has the strongest new verification evidence: the database-resilience script passed its twenty assertions in an isolated environment. This result is limited to the tested database-manager properties. O5 remains partially achieved through a defined

methodology and instruments; the incident-level and participant experiments must still be conducted.

8.3 Individual Research and Product Contributions

The individual contribution is the Component 4 incident pipeline and its analyst-facing explanation structure, not the internal algorithms of upstream components. The thesis provides a component-specific architecture, an implementation audit, a bounded result interpretation and reproducible evaluation instruments.

The product contribution is a central service and dashboard pattern for relating heterogeneous security evidence. Its future value depends on whether the proposed experiments demonstrate better incident interpretation and manageable false-alert rates under realistic collection conditions. The current evidence is sufficient for a transparent research prototype account, but not for an unqualified production-readiness claim.

8.4 Future Directions

The first priority is a clean-state reproduction of the staged integration failure, including event IDs, signal-family counts, active configuration and state before every request. The second is an incident-labelled evaluation with legitimate-administration negatives, delayed delivery, missing telemetry and ablations. These tasks directly address the most important unresolved research claims.

The next engineering priorities are provenance-aware evidence counting, durable producer delivery, explicit asset identity, correlation-state recovery and authenticated role-based access. Explanation work should add model/output-space metadata, evidence-version linkage and faithfulness checks. Dashboard work should then be evaluated with analysts rather than judged only by visual completeness.

Longer-term work can study adaptive windows, calibrated fusion, drift monitoring and constrained clinical summaries. Commercialisation should follow measured operating limits, supported integrations and clear governance. These directions preserve the individual component boundary while enabling the broader project to integrate reliable, explainable incident reasoning.

REFERENCES

[1] “Component 04 - Copy.rar,” supplied unpublished project archive, including source code, model metadata, database and audit artefacts. Inspected Oct. 9, 2026. Selected hashes and paths are recorded in Appendix A.

[2] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, “HOLMES: Real-time APT detection through correlation of suspicious information flows,” in Proc. IEEE Symposium on Security and Privacy, 2019, pp. 1137–1152. Available: https://arxiv.org/abs/1810.01594.

[3] M. N. Hossain et al., “SLEUTH: Real-time attack scenario reconstruction from COTS audit data,” in Proc. 26th USENIX Security Symposium, 2017, pp. 487–504. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/hossain.

[4] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, “UNICORN: Runtime provenance-based detector for advanced persistent threats,” in Proc. Network and Distributed System Security Symposium, 2020. doi: 10.14722/ndss.2020.24046. Available: https://arxiv.org/abs/2001.01525.

[5] Z. Cheng et al., “KAIROS: Practical intrusion detection and investigation using whole-system provenance,” in Proc. IEEE Symposium on Security and Privacy, 2024. Preprint available: https://arxiv.org/abs/2308.05034.

[6] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems, vol. 30, 2017. Available: https://arxiv.org/abs/1705.07874.

[7] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you?: Explaining the predictions of any classifier,” in Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016. Available: https://arxiv.org/abs/1602.04938.

[8] MITRE, “Remote Services, Technique T1021,” MITRE ATT&CK. Available: https://attack.mitre.org/techniques/T1021/. Accessed: Oct. 9, 2026.

[9] Microsoft, “Sysmon,” Sysinternals documentation. Available: https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon. Accessed: Oct. 9, 2026.

[10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016. doi: 10.1145/2939672.2939785. Available: https://arxiv.org/abs/1603.02754.

[11] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in Proc. IEEE International Conference on Data Mining, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.

[13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002. doi: 10.1613/jair.953.

[14] National Institute of Standards and Technology, “Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile,” NIST SP 800-61 Rev. 3, Apr. 2025. doi: 10.6028/NIST.SP.800-61r3.

[15] P. J. Phillips et al., “Four principles of explainable artificial intelligence,” NISTIR 8312, Sep. 2021. doi: 10.6028/NIST.IR.8312.

[16] “integrationc4.docx,” supplied unpublished integration guidance. Date of original preparation not established; inspected Oct. 9, 2026. Recommendations are distinguished from executed results.

[17] “Formating Standards(1).pdf,” supplied institutional dissertation formatting guidance. Publication date not established; used for manuscript preparation, Oct. 2026.

[18] “Individual Thesis Content(1).docx,” supplied individual-thesis structure template. Publication date not established; eight-chapter body followed in this manuscript.

[19] UNSW, “The UNSW-NB15 dataset,” research dataset documentation. Available: https://research.unsw.edu.au/projects/unsw-nb15-dataset. Accessed: Oct. 9, 2026.

[20] Open Threat Research, “Security Datasets: Introduction,” project documentation. Available: https://securitydatasets.com/introduction.html. Accessed: Oct. 9, 2026.

[21] SQLite, “SQLite foreign key support” and “Write-ahead logging,” official documentation. Available: https://sqlite.org/foreignkeys.html and https://sqlite.org/wal.html. Accessed: Oct. 9, 2026.

GLOSSARY

Term	Definition
Active weight	The share assigned to an available signal after normalisation; distinct from the configured weight.
Asset identity	The mapping between a monitored entity and its identifiers over time.
Baseline relationship	An expected relationship used as context, not an observed attack edge.
Candidate signal	An analytical output that may contribute to an incident if policy permits.
Cold-start gap	An interval after restart in which recent correlation state is not yet reconstructed.
Corroboration	Support from related evidence; independence must be established rather than assumed.
Evidence gate	A policy that decides whether evidence is sufficient before or alongside scoring.
Evidence lineage	The trace from a derived signal or explanation to originating observations.
Forecast support	Prediction context about future risk, distinct from observed malicious activity.
Incident	A bounded investigation object linking a security claim to evidence and status.
Model eligibility	Compatibility of available features with a fitted model input contract.
Provenance	Information about the origin and transformation of data or a conclusion.
Risk score	A prioritisation value; not automatically a calibrated attack probability.
Signal family	A mechanism category such as graph or temporal correlation.
Snapshot	One captured state whose counts must not be merged with unrelated captures.

APPENDIX A: RESEARCH INSTRUMENTS AND EXTENDED RESULTS

This appendix supports independent inspection of Component 4. It contains evidence identifiers, contracts, experimental instruments and selected implementation listings. Proposed test cases are not completed results. Source listings are derived from the supplied archive and are included because the component is an executable research artefact; they do not extend the thesis to other components.

A.1 Evidence Register and Reproducibility Manifest

ID	Artefact	Use and limitation
E1	Source modules	Direct implementation inspection; not a full runtime execution.
E2	models/training_metadata.json	Historical training summaries dated 4 May 2026; not reproduced.
E3	audit/model_audit.json	Fitted feature counts and model settings; archived audit.
E4	integration_failure.txt	Health pass followed by premature-incident failure.
E5	audit/runtime_validation/*.json	Saved state reports; not independent proof of window recovery.
E6	data/database/component4.db	Read-only snapshot: 18 events, one alert and five evidence links.
E7	tests/test_database_resilience.py	Executed in isolation: twenty assertions passed.
E8	integrationc4.docx	Integration recommendations and context; not an execution ledger.

Table A.1. Evidence register.

Table A.1 identifies the evidence used in Chapters 5 and 6. The hashes below identify selected original extracted files. They are not hashes of the reformatted listings, and no source-control commit is inferred.

models/training_metadata.json
SHA-256: 1c765986b4504381b196723c9fa18b4f6c4f4df18f773e523292b09bf1baeeae
audit/model_audit.json
SHA-256: 7c06918bf8cc1cba06b892a4461a01429ebf6f488af16b5576e2c8febfd111b7
integration_failure.txt
SHA-256: 58044664c74342cd6d2b3c255c0ed8432da40e58ecaeb87dbd35aa91800041cb
src/engine/alert_fusion.py
SHA-256: 1f3c1f3d645464e8e9193f229beb37c50ddd831ab92c813e3276673d3c5be2e9

src/engine/correlation_orchestrator.py
SHA-256: 667555856a3edd55197ec3f6674287773170123edcb79394a4ac281e60e9a564
data/database/component4.db
SHA-256: d5610b531d565b1ab96d3e7930ef7218bf7b40d887e676c04749e4bd585c4dc4

A complete future run manifest should additionally record the source revision, resolved dependency environment, model hashes, dataset checksums, scenario stream checksum, configuration, start-state policy, UTC start/end times, hardware resources and all output paths. Without these items, two apparently identical demonstrations may differ in important hidden state.

A.2 Configuration and Feature Contracts

Setting	Recorded default	Interpretation
Correlation window	10 minutes	Bounded graph/temporal context; not a validated optimum.
Memory window	300 seconds	Memory–network association interval.
Teammate window	15 minutes	Recent upstream context.
Maximum graph hops	4	Limits candidate path length.
Graph / temporal cooldown	5 minutes	Duplicate suppression interval.
Memory cooldown	300 seconds	Memory-correlation suppression interval.
Fusion cooldown	10 minutes	Incident deduplication context.
Alert threshold	0.45	Rule threshold, not probability calibration.
Signal fire threshold	0.35	Configured signal policy input.
Minimum fusion families	3	Normal orchestrator gate.
ML / graph / temporal weights	0.30 / 0.25 / 0.25	Configured weights before available-signal normalisation.
Memory / teammate weights	0.15 / 0.05	Configured weights before normalisation.
API host / port	127.0.0.1 / 8000	Local binding in supplied defaults.

Table A.2. Selected configuration values.

Table A.2 records inspected defaults. Environment overrides or later edits can change runtime behaviour. Each experiment must capture resolved values rather than merely citing this table.

Position	Network feature	Contract rule
1	dur	Preserve fitted name, order, preprocessing and units.
2	sbytes	Preserve fitted name, order, preprocessing and units.
3	dbytes	Preserve fitted name, order, preprocessing and units.
4	rate	Preserve fitted name, order, preprocessing and units.
5	spkts	Preserve fitted name, order, preprocessing and units.
6	dpkts	Preserve fitted name, order, preprocessing and units.
7	sttl	Preserve fitted name, order, preprocessing and units.
8	dttl	Preserve fitted name, order, preprocessing and units.
9	sload	Preserve fitted name, order, preprocessing and units.
10	dload	Preserve fitted name, order, preprocessing and units.
11	sloss	Preserve fitted name, order, preprocessing and units.
12	dloss	Preserve fitted name, order, preprocessing and units.
13	sinpkt	Preserve fitted name, order, preprocessing and units.
14	dinpkt	Preserve fitted name, order, preprocessing and units.
15	sjit	Preserve fitted name, order, preprocessing and units.
16	djit	Preserve fitted name, order, preprocessing and units.
17	swin	Preserve fitted name, order, preprocessing and units.
18	dwin	Preserve fitted name, order, preprocessing and units.
19	tcprtt	Preserve fitted name, order, preprocessing and units.

Position	Network feature	Contract rule
20	synack	Preserve fitted name, order, preprocessing and units.
21	ackdat	Preserve fitted name, order, preprocessing and units.
22	smean	Preserve fitted name, order, preprocessing and units.
23	dmean	Preserve fitted name, order, preprocessing and units.
24	ct_srv_src	Preserve fitted name, order, preprocessing and units.
25	ct_state_ttl	Preserve fitted name, order, preprocessing and units.
26	ct_dst_ltm	Preserve fitted name, order, preprocessing and units.
27	ct_src_dport_ltm	Preserve fitted name, order, preprocessing and units.
28	ct_dst_sport_ltm	Preserve fitted name, order, preprocessing and units.
29	ct_dst_src_ltm	Preserve fitted name, order, preprocessing and units.
30	ct_src_ltm	Preserve fitted name, order, preprocessing and units.
31	ct_srv_dst	Preserve fitted name, order, preprocessing and units.
32	is_ftp_login	Preserve fitted name, order, preprocessing and units.
33	ct_ftp_cmd	Preserve fitted name, order, preprocessing and units.
34	ct_flw_http_mthd	Preserve fitted name, order, preprocessing and units.
35	is_sm_ips_ports	Preserve fitted name, order, preprocessing and units.
36	trans_depth	Preserve fitted name, order, preprocessing and units.
37	response_body_len	Preserve fitted name, order, preprocessing and units.

Table A.3. Ordered 37-feature network-model input.

Table A.3 records the ordered network input from the model audit. Repeating the contract rule emphasises that a field name alone does not define its computational

semantics. Dataset documentation and the fitted preprocessing must be consulted before live extraction is considered equivalent.

Position	Memory/host feature
1	eventid_severity
2	is_memory_event
3	is_lateral_event
4	layer_encoded
5	severity_encoded
6	indicator_encoded

Table A.4. Six-feature memory-model input.

Table A.4 is separate from the twelve contextual correlation features. It is also separate from any broader feature-selection experiment mentioned in training metadata. The runtime contract should be validated using a known reference vector and expected model output.

A.3 Persistent Schema Audit

Tables A.5–A.9 describe the actual archived SQLite columns. Defaults are transcribed from SQLite schema metadata. A default value can conceal missing information if the application does not preserve availability separately; for example, zero confidence should not automatically be read as an evaluated negative result.

Column	Type	Required / key	Default
id	INTEGER	PK	—
event_id	TEXT	NOT NULL	—
source_event_id	INTEGER	Nullable	0
timestamp	TEXT	NOT NULL	—
source_component	TEXT	Nullable	''
source_type	TEXT	Nullable	''
event_type	TEXT	Nullable	''
indicator	TEXT	Nullable	''
layer	TEXT	Nullable	'unknown'
source_device	TEXT	Nullable	''

Column	Type	Required / key	Default
destination_device	TEXT	Nullable	''
source_ip	TEXT	Nullable	''
destination_ip	TEXT	Nullable	''
source_device_type	TEXT	Nullable	''
destination_device_type	TEXT	Nullable	''
username	TEXT	Nullable	''
session_id	TEXT	Nullable	''
incident_id	TEXT	Nullable	''
source_process	TEXT	Nullable	''
destination_process	TEXT	Nullable	''
parent_process	TEXT	Nullable	''
process_id	INTEGER	Nullable	0
command_line	TEXT	Nullable	''
protocol	TEXT	Nullable	''
source_port	INTEGER	Nullable	0
destination_port	INTEGER	Nullable	0
bytes_transferred	INTEGER	Nullable	0
auth_type	TEXT	Nullable	''
auth_success	INTEGER	Nullable	0
memory_indicators_json	TEXT	Nullable	'[]'
teammate_source	TEXT	Nullable	''
teammate_confidence	REAL	Nullable	0.0
confidence	REAL	Nullable	0.0
label	TEXT	Nullable	'normal'
attack_category	TEXT	Nullable	''
mitre_technique	TEXT	Nullable	''
severity	TEXT	Nullable	'NONE'
dataset_source	TEXT	Nullable	''
cross_layer_features_json	TEXT	Nullable	'{}'
details_json	TEXT	Nullable	'{}'
raw_json	TEXT	Nullable	'{}'

Column	Type	Required / key	Default
created_at	TEXT	NOT NULL	—

Table A.5. Stored events columns.

Column	Type	Required / key	Default
id	INTEGER	PK	—
alert_id	TEXT	NOT NULL	—
alert_number	INTEGER	Nullable	—
timestamp	TEXT	NOT NULL	—
first_seen	TEXT	NOT NULL	—
last_seen	TEXT	NOT NULL	—
occurrence_count	INTEGER	Nullable	1
alert_type	TEXT	NOT NULL	—
status	TEXT	Nullable	'OPEN'
severity	TEXT	NOT NULL	—
risk_score	REAL	NOT NULL	—
base_risk_score	REAL	Nullable	0.0
agreement_bonus	REAL	Nullable	0.0
source_device	TEXT	Nullable	''
destination_device	TEXT	Nullable	''
lateral_movement_path_json	TEXT	Nullable	'[]'
layers_fired_json	TEXT	Nullable	'[]'
observed_layer_count	INTEGER	Nullable	0
forecast_support	INTEGER	Nullable	0
evidence_count	INTEGER	Nullable	0
score_breakdown_json	TEXT	Nullable	'{}'
signal_weights_json	TEXT	Nullable	'{}'
available_signals_json	TEXT	Nullable	'[]'
mitre_techniques_json	TEXT	Nullable	'[]'
analyst_action	TEXT	Nullable	—
false_positive	INTEGER	Nullable	0
notes	TEXT	Nullable	''
fingerprint	TEXT	Nullable	—

Column	Type	Required / key	Default
created_at	TEXT	NOT NULL	—
updated_at	TEXT	NOT NULL	—

Table A.6. Stored alerts columns.

Column	Type	Required / key	Default
id	INTEGER	PK	—
alert_id	TEXT	NOT NULL	—
event_id	TEXT	NOT NULL	—
evidence_type	TEXT	Nullable	'event'
source_layer	TEXT	Nullable	''
created_at	TEXT	NOT NULL	—

Table A.7. Stored alert_evidence columns.

Column	Type	Required / key	Default
id	INTEGER	PK	—
alert_id	TEXT	NOT NULL	—
model_name	TEXT	Nullable	''
shap_available	INTEGER	Nullable	0
shap_values_json	TEXT	Nullable	'[]'
feature_names_json	TEXT	Nullable	'[]'
top_features_json	TEXT	Nullable	'[]'
mitre_techniques_json	TEXT	Nullable	'[]'
explanation_text	TEXT	Nullable	''
created_at	TEXT	NOT NULL	—
updated_at	TEXT	NOT NULL	—

Table A.8. Stored xai_explanations columns.

Column	Type	Required / key	Default
id	INTEGER	PK	—
alert_id	TEXT	NOT NULL	—
action	TEXT	NOT NULL	—
previous_status	TEXT	Nullable	''
new_status	TEXT	Nullable	''

Column	Type	Required / key	Default
false_positive	INTEGER	Nullable	0
note	TEXT	Nullable	''
analyst	TEXT	Nullable	''
timestamp	TEXT	NOT NULL	—

Table A.9. Stored analyst_feedback columns.

A.4 Evaluation Scenario Catalogue

Table A.10 specifies prospective scenarios. None of these rows is a newly measured detection result. Each scenario must be instantiated with stable event identifiers and independently prepared ground truth. The expected outcome is a test oracle to be reviewed before execution, not a description of observed behaviour.

ID	Scenario	Expected outcome	Property
T01	Legitimate single remote-service connection	No final movement incident solely from a port.	Evidence gate and benign specificity.
T02	Related source activity, remote access and destination execution	One evidence-linked candidate when policy is met.	Incident recall and path support.
T03	Unrelated hosts with similar timestamps	No join solely from temporal proximity.	Entity matching.
T04	Forecast only	Forecast context retained; no observed movement incident.	Observed/forecast separation.
T05	Missing network feature vector	Network model unavailable; supported correlators continue.	Eligibility and partial processing.
T06	Evaluated model score equal to zero	Zero included as available evidence.	Missingness semantics.
T07	Duplicate producer delivery	One logical observation with auditable retry behaviour.	Idempotency.
T08	Same observation triggers three mechanisms	No unsupported independence claim.	Evidence lineage and overlap.
T09	Memory event arrives before network event	Association considered within configured interval.	Arrival-order handling.
T10	Network event arrives before memory event	Equivalent evidence considered under documented policy.	Symmetric association.
T11	Event arrives outside correlation window	No silent extension of the configured window.	Temporal boundary.
T12	Clock skew and unknown timezone	Input rejected or uncertainty explicitly retained.	Time semantics.

ID	Scenario	Expected outcome	Property
T13	One host with multiple role aliases	No fabricated multi-device movement path.	Asset canonicalisation.
T14	Shared or reassigned IP address	Ambiguous identity not silently merged.	Historical asset mapping.
T15	Legitimate image transfer to imaging service	Baseline activity not labelled movement without evidence.	Healthcare-oriented negative case.
T16	Service restart between related observations	Recovery behaviour matches documented state policy.	Window rehydration.
T17	Database unavailable during persistence	Failure visible; no misleading durable-success response.	Transaction/error handling.
T18	Model cannot load	Dependency degraded, remaining paths explicitly reported.	Model resilience.
T19	Stale forecast with fresh observed event	Freshness visible; forecast handled under expiry policy.	Forecast expiry.
T20	Incident receives additional evidence	Stable incident identity and updated explanation lineage.	Lifecycle and deduplication.
T21	Analyst marks false positive	Decision stored without erasing original evidence.	Feedback auditability.
T22	Unknown alert identifier	Explicit not-found result; no unrelated explanation.	API retrieval correctness.
T23	Repeated dashboard refresh during API outage	Cached state labelled and outage visible.	Presentation resilience.
T24	Unauthorised publisher or analyst	Access denied by deployment control.	Security hardening; not established in snapshot.

Table A.10. Prospective functional and robustness scenarios.

A.5 Trial Record and Analysis Worksheet

A trial should be considered reproducible only when another researcher can replay the same stream under the same policy. Table A.11 defines the record to complete for every run. Empty results must remain empty until measured; zero is not an acceptable substitute for an unexecuted trial.

Field	Required content
Run identity	Unique run ID, operator, UTC start and end, source revision.
Environment	Operating system, Python, resolved dependencies, CPU and memory.
Artefact identity	Model and scenario hashes; schema version; configuration snapshot.
Initial state	Empty or restored DB; in-memory state policy; baseline version.

Field	Required content
Ground truth	Scenario labels, asset relationships, time boundaries and adjudicator.
Input trace	Ordered messages, original event time, arrival time and delivery identity.
Decision trace	Eligible models, signal families, evidence IDs, active weights and final score.
Outcome	Matched incidents, FP/FN, duplicates, latency and failed requests.
Explanation audit	Unsupported statements, missing evidence and attribution reconciliation.
Disposition	Pass/fail by predeclared oracle; reason and follow-up issue.

Table A.11. Prospective trial record.

The evaluation dataset should be divided by scenario or campaign where possible, not by randomly splitting highly related records. Development scenarios are used for threshold selection; final scenarios remain untouched until policy is frozen. Legitimate administration cases must not be excluded simply because they are difficult negatives.

For each ablation, retain the same input and ground truth. Disabling graph correlation, for example, should not also remove graph-relevant events from the stream. Otherwise, the comparison changes both the method and its information. Report per-scenario outcomes before aggregating them.

The analysis worksheet should include an error taxonomy: identity error, temporal mismatch, feature incompatibility, insufficient evidence, duplicate evidence, threshold effect, missing telemetry, delivery failure and explanation mismatch. Each error can require a different correction. Aggregate F1 alone cannot reveal which subsystem needs attention.

A.6 Proposed Stakeholder Validation Instrument

This instrument is prospective and has not been administered. Recruitment, consent, participant information and institutional approval must be completed as required before use. Use synthetic or appropriately de-identified incidents and do not ask participants to operate clinical devices.

Task	Success criterion	Record
Identify the initiating asset	Correct asset with supporting event reference.	Answer, evidence chosen, time and confidence.

Task	Success criterion	Record
Explain the movement path	Correctly separates observed edges from inferred relationships.	Path interpretation and unsupported assumptions.
Identify forecast support	Does not describe a forecast as an observed attack.	Classification and explanation.
Inspect model availability	Recognises a skipped model and the feature reason.	Answer and time.
Explain the score	Identifies active weights and distinguishes score from probability.	Interpretation errors.
Review raw evidence	Finds the records behind the incident claim.	Navigation path and evidence IDs.
Record a disposition	Provides a justified action without deleting evidence.	Decision, rationale and task completion.
Restricted clinical summary	Identifies service status and escalation route.	Comprehension without technical terminology.

Table A.12. Prospective stakeholder tasks.

Table A.12 defines observable task outcomes. After each task, a brief questionnaire can ask whether the participant understood the evidence, recognised uncertainty, knew what to do next and found the necessary information. Use a stated response scale and include an open comment. Do not report a standardised instrument score unless that instrument is actually used and scored according to its method.

For a comparison study, counterbalance interface order and use equivalent scenarios to reduce learning effects. Analyse task accuracy and time together: a faster but less accurate decision is not an unqualified improvement. Report participant expertise and uncertainty around results without generalising from a small convenience sample to all clinicians or analysts.

A.7 Completed Database Verification Record

The following record is the actual console output of the isolated database-resilience execution on 9 October 2026. Temporary-path values identify the verification environment and are not production deployment paths. The check was executed before the archived runtime database was separately extracted, explaining the final assertion that no real runtime database was created.

==========

COMPONENT 4 - DATABASE RESILIENCE TEST

==========

======

Real runtime DB : /workspace/scratch/f6196f5f81c6/thesis_work/c4/Component 04 - Copy/data/database/component4.db

Safety : all write/failure tests use temporary SQLite databases

1. Temporary database initialization

PASS - Temporary database does not exist before initialization

[Database] Initializing Component 4 database...

[Database] Database initialized successfully

[Database] Path: /tmp/component4_database_resilience_ry9ofkn4/component4_resilience.db

PASS - Temporary SQLite database created

PASS - Database health check returns healthy

PASS - Required Component 4 tables are present

2. SQLite runtime safety settings

PASS - SQLite foreign-key enforcement is enabled

PASS - SQLite WAL journal mode is enabled

3. Transaction commit behavior

PASS - Successful transaction is committed

4. Transaction rollback on failure

PASS - Intentional transaction failure raised safely

PASS - Failed transaction was rolled back

5. Simulated database-path failure

[Database] Health check failed: unable to open database file

PASS - Database failure is detected without crashing health_check()

PASS - Database health recovers after valid path is restored

6. Close/reopen persistence

PASS - Reopened DatabaseManager is healthy

PASS - Committed data survives DatabaseManager recreation

PASS - Rolled-back data did not reappear after reopen

7. Idempotent schema reinitialization

[Database] Initializing Component 4 database...

[Database] Database initialized successfully

[Database] Path: /tmp/component4_database_resilience_ry9ofkn4/component4_resilience.db

PASS - Database remains healthy after repeated initialize()

PASS - Existing data remains intact after schema reinitialization

8. Fresh database recovery

[Database] Initializing Component 4 database...

[Database] Database initialized successfully

[Database] Path: /tmp/component4_database_resilience_ry9ofkn4/recovered_component4.db

PASS - Fresh recovery database created

PASS - Freshly recovered database is healthy

PASS - Core Component 4 schema restored in fresh database

9. Safety verification - real runtime database

PASS - Test did not create the real runtime database

```
================================================================================
======
TEST RESULT - PASSED
================================================================================
======
All Component 4 database resilience checks passed.
SQLite initialization, health detection, commit/rollback, failure recovery, reopen persistence,
and schema
    recovery worked correctly without modifying the real runtime database.
```

The script exited successfully. The result is limited to the properties asserted by that script; it does not test a running API, model inference or dashboard behaviour. The subsequent read-only archive inspection found 18 events, one alert, five evidence links, one explanation and one feedback record, with an integrity-check result of "ok".

A.8 Archived Staged Integration Failure

This is supplied historical output, not a newly executed API test. It is preserved to make the unresolved acceptance condition reviewable. The original file hash is recorded in A.1. A future successful run should be retained alongside this record rather than replacing it.

```
================================================================================
======
COMPONENT 4 - AUTOMATED API INTEGRATION TEST
================================================================================
======
Target API : http://127.0.0.1:8000
Scenario   : Cross-layer fileless lateral movement
Pre-check - API health
--------------------------------------------------------------------------
  PASS - API reachable at http://127.0.0.1:8000
Stage 1 - SMB lateral network hop
--------------------------------------------------------------------------
================================================================================
======
TEST RESULT - FAILED
================================================================================
======
Stage 1 created a final incident too early.
Expected: False
Actual  : True
```

The failure establishes that the test oracle and observed Stage 1 behaviour disagreed. It does not identify the root cause. A reproduction must log initial state, normalised

events, matched evidence, signal-family counts and the path by which fusion was invoked.

A.9 Typed API Request and Response Contracts

These classes define the inspected service boundary. Field constraints are executable validation rules, while semantic asset identity and source authenticity require additional controls. Read this listing with the route inventory in Table 5.1. Defaults must not be mistaken for measured observations.

Source: src/api/schemas.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
from __future__ import annotations
from typing import Any, Dict, List, Optional
from enum import Enum
from pydantic import BaseModel, Field, field_validator, model_validator
class NetworkEventRequest(BaseModel):
    src_ip: str
    dst_ip: str
    src_port: int = Field(default=0, ge=0, le=65535)
    dst_port: int = Field(ge=1, le=65535)
    protocol: str
    event_type: str = 'NETWORK_CONNECTION'
    indicator: str = 'NETWORK_CONNECTION'
    severity: str = 'NONE'
    label: str = 'normal'
    timestamp: Optional[str] = None
    source_device: Optional[str] = None
    destination_device: Optional[str] = None
    bytes_transferred: int = Field(default=0, ge=0)
    extra_features: Dict[str, Any] = Field(default_factory=dict)
    @field_validator('src_ip', 'dst_ip', 'protocol')
    @classmethod
    def validate_required_network_text(cls, value: str) -> str:
        clean = str(value or '').strip()
        if not clean:
            raise ValueError('Network source, destination, and protocol values cannot be empty.')
        return clean
class MemoryEventRequest(BaseModel):
    device_ip: str
    event_type: str
    indicator: Optional[str] = None
    process_name: str = ''
    parent_name: str = ''
    pid: int = Field(default=0, ge=0)
    command_line: str = ''
    indicators: List[str] = Field(default_factory=list)
    severity: str = 'MEDIUM'
    confidence: float = Field(default=0.0, ge=0.0, le=1.0)
```

```
    timestamp: Optional[str] = None
    remote_ip: str = ''
    remote_port: int = Field(default=0, ge=0, le=65535)
    @field_validator('device_ip', 'event_type')
    @classmethod
    def validate_required_memory_text(cls, value: str) -> str:
        clean = str(value or '').strip()
        if not clean:
            raise ValueError('Memory device and event_type cannot be empty.')
        return clean
class HostEventRequest(BaseModel):
    device_ip: str = ''
    hostname: str = ''
    event_type: str = 'HOST_ACTIVITY'
    indicator: str = ''
    process_name: str = ''
    parent_name: str = ''
    destination_process: str = ''
    pid: int = Field(default=0, ge=0)
    command_line: str = ''
    username: str = ''
    severity: str = 'MEDIUM'
    label: str = 'normal'
    confidence: float = Field(default=0.0, ge=0.0, le=1.0)
    mitre_technique: str = ''
    attack_category: str = ''
    timestamp: Optional[str] = None
    details: Dict[str, Any] = Field(default_factory=dict)
    @field_validator('event_type')
    @classmethod
    def validate_host_event_type(cls, value: str) -> str:
        clean = str(value or '').strip()
        if not clean:
            raise ValueError('Host event_type cannot be empty.')
        return clean
class TeammateAlertRequest(BaseModel):
    source_component: str
    event_type: str
    indicator: Optional[str] = None
    source_device: str
    destination_device: str = ''
    source_ip: str = ''
    destination_ip: str = ''
    username: str = ''
    process_name: str = ''
    parent_process: str = ''
    command_line: str = ''
    confidence: float = Field(default=0.0, ge=0.0, le=1.0)
    severity: str = 'MEDIUM'
    label: Optional[str] = None
    mitre_technique: str = ''
    attack_category: str = ''
    session_id: str = ''
    incident_id: str = ''
```

```
    layer: Optional[str] = None
    timestamp: Optional[str] = None
    details: Dict[str, Any] = Field(default_factory=dict)
    @field_validator('source_component', 'event_type', 'source_device')
    @classmethod
    def validate_required_teammate_text(cls, value: str) -> str:
        clean = str(value or '').strip()
        if not clean:
            raise ValueError('Teammate source_component, event_type, and source_device cannot
be empty.')
        return clean
class AlertStatus(str, Enum):
    OPEN = 'OPEN'
    INVESTIGATING = 'INVESTIGATING'
    CONFIRMED = 'CONFIRMED'
    CONTAINED = 'CONTAINED'
    RESOLVED = 'RESOLVED'
    FALSE_POSITIVE = 'FALSE_POSITIVE'
class AnalystAction(str, Enum):
    ACKNOWLEDGE = 'ACKNOWLEDGE'
    INVESTIGATE = 'INVESTIGATE'
    CONFIRM = 'CONFIRM'
    CONTAIN = 'CONTAIN'
    RESOLVE = 'RESOLVE'
    MARK_FALSE_POSITIVE = 'MARK_FALSE_POSITIVE'
class AnalystFeedbackRequest(BaseModel):
    action: AnalystAction
    new_status: AlertStatus
    analyst: str = ''
    note: str = ''
    false_positive: bool = False
    @model_validator(mode='after')
    def validate_action_status(self):
        valid_transitions = {AnalystAction.ACKNOWLEDGE: AlertStatus.OPEN,
AnalystAction.INVESTIGATE:
            AlertStatus.INVESTIGATING, AnalystAction.CONFIRM: AlertStatus.CONFIRMED,
            AnalystAction.CONTAIN: AlertStatus.CONTAINED, AnalystAction.RESOLVE:
AlertStatus.RESOLVED,
            AnalystAction.MARK_FALSE_POSITIVE: AlertStatus.FALSE_POSITIVE}
        expected_status = valid_transitions[self.action]
        if self.new_status != expected_status:
            raise ValueError(f'Action {self.action.value} requires new_status
{expected_status.value}.')
        if self.action == AnalystAction.MARK_FALSE_POSITIVE:
            self.false_positive = True
        return self
class IngestionResponse(BaseModel):
    success: bool
    source_type: str
    event_id: str
    signal_families: int
    final_alert_created: bool
    final_alert_id: Optional[str] = None
    final_alert: Optional[Dict[str, Any]] = None
```

```
    processing: Dict[str, Any] = Field(default_factory=dict)
class HealthResponse(BaseModel):
    status: str
    component: str
    persistence_enabled: bool
    database_healthy: bool
    ml_available: bool
    network_pipeline_available: bool
    memory_pipeline_available: bool
    xai_available: bool
```

A.10 Fusion, Scoring and Incident Update Logic

This listing exposes the actual score extraction, available-weight normalisation, agreement bonus, sufficiency checks and incident-update behaviour. It is the implementation basis for Section 4.4.6. Pay particular attention to the distinction between the class-level sufficiency rule and the orchestrator gate; direct class calls do not necessarily enforce the same boundary.

Source: src/engine/alert_fusion.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
from __future__ import annotations
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Any, Dict, List, Optional, Set, Tuple
import hashlib
import json
import sys
import uuid
if __package__ in {None, ''}:
    PROJECT_ROOT = Path(__file__).resolve().parents[2]
    if str(PROJECT_ROOT) not in sys.path:
        sys.path.insert(0, str(PROJECT_ROOT))
from src.config.settings import settings
class AlertFusion:
    MITRE_DESCRIPTIONS = {'T1003.001': 'LSASS Memory', 'T1021': 'Remote Services',
'T1021.001': 'Remote
        Desktop Protocol', 'T1021.002': 'SMB/Windows Admin Shares', 'T1021.003': 'Distributed
Component
        Object Model', 'T1047': 'Windows Management Instrumentation', 'T1055': 'Process
Injection',
        'T1055.003': 'Thread Execution Hijacking', 'T1055.012': 'Process Hollowing', 'T1059':
'Command and
         Scripting Interpreter', 'T1059.001': 'PowerShell', 'T1078': 'Valid Accounts', 'T1110':
'Brute
        Force', 'T1129': 'Shared Modules', 'T1134': 'Access Token Manipulation', 'T1190': 'Exploit
        Public-Facing Application', 'T1218': 'System Binary Proxy Execution', 'T1499': 'Endpoint
Denial of
         Service', 'T1550.002': 'Pass the Hash', 'T1595': 'Active Scanning'}
    SIGNAL_WEIGHTS = settings.fusion_weights()
    ALERT_THRESHOLD = settings.alert_threshold
    SIGNAL_FIRE_THRESHOLD = settings.signal_fire_threshold
    AGREEMENT_BONUSES = {2: 0.03, 3: 0.07, 4: 0.1, 5: 0.1}
    def __init__(self, alert_threshold: Optional[float]=None, duplicate_cooldown_minutes:
        Optional[int]=None, signal_fire_threshold: Optional[float]=None, signal_weights:
        Optional[Dict[str, float]]=None):
        resolved_alert_threshold = settings.alert_threshold if alert_threshold is None else
            float(alert_threshold)
        resolved_duplicate_cooldown = settings.fusion_duplicate_cooldown_minutes if
            duplicate_cooldown_minutes is None else int(duplicate_cooldown_minutes)
        resolved_signal_fire_threshold = settings.signal_fire_threshold if signal_fire_threshold is
```

```
None
            else float(signal_fire_threshold)
        resolved_signal_weights = dict(settings.fusion_weights() if signal_weights is None else
            signal_weights)
        if not 0.0 <= resolved_alert_threshold <= 1.0:
            raise ValueError('alert_threshold must be between 0 and 1')
        if resolved_duplicate_cooldown < 0:
            raise ValueError('duplicate_cooldown_minutes cannot be negative')
        if not 0.0 <= resolved_signal_fire_threshold <= 1.0:
            raise ValueError('signal_fire_threshold must be between 0 and 1')
        expected_signals = {'ml_score', 'graph_score', 'temporal_score', 'memory_score',
'teammate_score'}
        if set(resolved_signal_weights) != expected_signals:
            missing = sorted(expected_signals - set(resolved_signal_weights))
            unexpected = sorted(set(resolved_signal_weights) - expected_signals)
            raise ValueError(f'signal_weights must contain exactly the expected signals.
                Missing={missing}, unexpected={unexpected}')
        normalized_weights: Dict[str, float] = {}
        for signal_name, weight in resolved_signal_weights.items():
            try:
                numeric_weight = float(weight)
            except (TypeError, ValueError, OverflowError) as exc:
                raise ValueError(f'Invalid weight for {signal_name}: {weight}') from exc
            if numeric_weight < 0.0:
                raise ValueError(f'Weight for {signal_name} cannot be negative')
            normalized_weights[signal_name] = numeric_weight
        if sum(normalized_weights.values()) <= 0.0:
            raise ValueError('At least one fusion signal weight must be positive')
        self.alert_threshold = resolved_alert_threshold
        self.duplicate_cooldown_minutes = resolved_duplicate_cooldown
        self.signal_fire_threshold = resolved_signal_fire_threshold
        self.signal_weights = normalized_weights
        self.alert_history: List[Dict] = []
        self.alert_count = 0
        self.recent_incidents: Dict[str, Dict] = {}
    def fuse(self, ml_result: Optional[Dict]=None, graph_result: Optional[Dict]=None,
temporal_result:
        Optional[Dict]=None, memory_result: Optional[Dict]=None, teammate_alerts:
        Optional[List[Dict]]=None, source_device: str='', raw_event: Optional[Dict]=None) ->
        Optional[Dict]:
        teammate_alerts = teammate_alerts if isinstance(teammate_alerts, list) else []
        raw_event = raw_event if isinstance(raw_event, dict) else {}
        ml_score = self._extract_ml_score(ml_result)
        graph_score = self._extract_graph_score(graph_result)
        temporal_score = self._extract_temporal_score(temporal_result)
        memory_score = self._extract_memory_score(memory_result)
        teammate_score = self._extract_teammate_score(teammate_alerts)
        scores = {'ml_score': ml_score, 'graph_score': graph_score, 'temporal_score':
temporal_score,
            'memory_score': memory_score, 'teammate_score': teammate_score}
        observed_layers = self._which_observed_layers_fired(ml_result=ml_result,
            graph_result=graph_result, temporal_result=temporal_result,
memory_result=memory_result,
            teammate_alerts=teammate_alerts)
```

```
        forecast_support = self._has_forecast_support(teammate_alerts)
        if not self._has_sufficient_observed_evidence(ml_result=ml_result,
graph_result=graph_result,
            temporal_result=temporal_result, memory_result=memory_result,
teammate_alerts=teammate_alerts,
             observed_layers=observed_layers):
            return None
        base_risk_score = self._compute_available_weighted_score(scores)
        agreement_bonus = self._calculate_agreement_bonus(observed_layers)
        risk_score = min(base_risk_score + agreement_bonus, 1.0)
        risk_score = round(risk_score, 4)
        if risk_score < self.alert_threshold:
            return None
        source_device = source_device.strip() if source_device else
            self._derive_source_device(raw_event=raw_event, graph_result=graph_result,
            temporal_result=temporal_result, memory_result=memory_result,
teammate_alerts=teammate_alerts)
        destination_device = self._derive_destination_device(raw_event=raw_event,
            graph_result=graph_result, temporal_result=temporal_result,
memory_result=memory_result,
            teammate_alerts=teammate_alerts)
        lateral_path = []
        if graph_result and isinstance(graph_result.get('path'), list):
            lateral_path = list(graph_result['path'])
        mitre_techniques = self._collect_mitre(ml_result=ml_result, graph_result=graph_result,
            temporal_result=temporal_result, memory_result=memory_result,
teammate_alerts=teammate_alerts)
        shap_values, feature_names, top_features = self._extract_xai(ml_result)
        evidence_event_ids = self._collect_evidence_event_ids(raw_event=raw_event,
            graph_result=graph_result, temporal_result=temporal_result,
memory_result=memory_result,
            teammate_alerts=teammate_alerts)
        layers_fired = list(observed_layers)
        if forecast_support:
            layers_fired.append('FORECAST_SUPPORT')
        fingerprint = self._create_incident_fingerprint(source_device=source_device,
            destination_device=destination_device, lateral_path=lateral_path,
            temporal_result=temporal_result, memory_result=memory_result,
teammate_alerts=teammate_alerts)
        existing_alert = self._find_recent_incident(fingerprint)
        if existing_alert is not None:
            return self._update_existing_alert(alert=existing_alert, risk_score=risk_score,
                base_risk_score=base_risk_score, agreement_bonus=agreement_bonus, scores=scores,
                layers_fired=layers_fired, evidence_event_ids=evidence_event_ids,
ml_result=ml_result,
                graph_result=graph_result, temporal_result=temporal_result,
memory_result=memory_result,
                teammate_alerts=teammate_alerts, mitre_techniques=mitre_techniques,
                lateral_path=lateral_path, shap_values=shap_values, feature_names=feature_names,
                top_features=top_features)
        severity = self._classify_severity(risk_score)
        self.alert_count += 1
        now = datetime.now(timezone.utc).isoformat()
        alert = {'alert_id': self._generate_alert_id(), 'alert_number': self.alert_count, 'timestamp':
```

```
            now, 'first_seen': now, 'last_seen': now, 'occurrence_count': 1, 'alert_type':
            'CROSS_LAYER_LATERAL_MOVEMENT', 'status': 'OPEN', 'severity': severity,
'risk_score':
            risk_score, 'base_risk_score': round(base_risk_score, 4), 'agreement_bonus':
            round(agreement_bonus, 4), 'source_device': source_device, 'destination_device':
            destination_device, 'score_breakdown': self._build_score_breakdown(scores),
            'available_signals': [name for name, value in scores.items() if value is not None],
            'signal_weights': self._get_effective_weights(scores), 'lateral_movement_path':
lateral_path,
            'mitre_techniques': mitre_techniques, 'layers_fired': layers_fired,
'observed_layer_count':
            len(observed_layers), 'forecast_support': forecast_support, 'evidence_event_ids':
            evidence_event_ids, 'evidence_count': len(evidence_event_ids), 'shap_values':
shap_values,
            'feature_names': feature_names, 'top_features': top_features, 'shap_available':
            bool(shap_values), 'ml_result': ml_result, 'graph_result': graph_result,
'temporal_result':
            temporal_result, 'memory_result': memory_result, 'teammate_alerts': teammate_alerts,
            'analyst_action': None, 'false_positive': False, 'notes': '', 'fingerprint': fingerprint,
            'is_update': False}
        self.alert_history.append(alert)
        self.recent_incidents[fingerprint] = {'alert_id': alert['alert_id'], 'last_seen':
            datetime.now(timezone.utc)}
        print(f'[Fusion] NEW ALERT #{self.alert_count} | {severity} | Risk={risk_score:.4f} |
            Device={source_device} | Layers={layers_fired}')
        return alert
    def _compute_available_weighted_score(self, scores: Dict[str, Optional[float]]) -> float:
        weighted_total = 0.0
        active_weight = 0.0
        for signal_name, value in scores.items():
            if value is None:
                continue
            weight = self.signal_weights[signal_name]
            weighted_total += value * weight
            active_weight += weight
        if active_weight <= 0.0:
            return 0.0
        return round(weighted_total / active_weight, 4)
    def _get_effective_weights(self, scores: Dict[str, Optional[float]]) -> Dict[str, float]:
        active_signals = [signal_name for signal_name, value in scores.items() if value is not None]
        active_weight = sum((self.signal_weights[signal_name] for signal_name in active_signals))
        if active_weight <= 0:
            return {}
        return {signal_name: round(self.signal_weights[signal_name] / active_weight, 4) for
signal_name in
             active_signals}
    def _extract_ml_score(self, ml_result: Optional[Dict]) -> Optional[float]:
        if not ml_result:
            return None
        status = str(ml_result.get('status', '')).lower()
        if status in {'unavailable', 'no_compatible_ml_model'}:
            return None
        probability = ml_result.get('probability')
        if probability is not None:
```

```
            return self._clamp_score(probability)
        anomaly_score = ml_result.get('normalized_anomaly_score')
        if anomaly_score is not None:
            return self._clamp_score(anomaly_score)
        return None
    def _extract_graph_score(self, graph_result: Optional[Dict]) -> Optional[float]:
        if not graph_result:
            return None
        value = graph_result.get('graph_score')
        if value is None:
            return None
        return self._clamp_score(value)
    def _extract_temporal_score(self, temporal_result: Optional[Dict]) -> Optional[float]:
        if not temporal_result:
            return None
        value = temporal_result.get('temporal_score')
        if value is None:
            return None
        return self._clamp_score(value)
    def _extract_memory_score(self, memory_result: Optional[Dict]) -> Optional[float]:
        if not memory_result:
            return None
        value = memory_result.get('memory_score', memory_result.get('correlation_score'))
        if value is None:
            return None
        return self._clamp_score(value)
    def _extract_teammate_score(self, teammate_alerts: List[Dict]) -> Optional[float]:
        if not teammate_alerts:
            return None
        observed_scores = []
        forecast_scores = []
        for alert in teammate_alerts:
            if not isinstance(alert, dict):
                continue
            confidence = self._safe_float(alert.get('confidence', alert.get('teammate_confidence',
0.0)))
            confidence = self._clamp_score(confidence)
            if self._is_forecast_alert(alert):
                forecast_scores.append(confidence)
            else:
                observed_scores.append(confidence)
        if observed_scores:
            return max(observed_scores)
        if forecast_scores:
            return round(max(forecast_scores) * 0.5, 4)
        return None
    def _has_sufficient_observed_evidence(self, ml_result: Optional[Dict], graph_result:
Optional[Dict],
        temporal_result: Optional[Dict], memory_result: Optional[Dict], teammate_alerts:
List[Dict],
        observed_layers: List[str]) -> bool:
        if graph_result:
            return True
        if temporal_result:
```

```
            return True
        if memory_result:
            return True
        independent_signals = 0
        if ml_result and bool(ml_result.get('is_malicious')):
            independent_signals += 1
        if self._has_observed_teammate_alert(teammate_alerts):
            independent_signals += 1
        return independent_signals >= 2 or len(observed_layers) >= 2
    def _which_observed_layers_fired(self, ml_result: Optional[Dict], graph_result:
Optional[Dict],
        temporal_result: Optional[Dict], memory_result: Optional[Dict], teammate_alerts:
List[Dict]) ->
        List[str]:
        fired: List[str] = []
        if ml_result and bool(ml_result.get('is_malicious')):
            ml_layer = str(ml_result.get('layer', 'unknown')).upper()
            fired.append(f'ML_{ml_layer}')
        if graph_result:
            fired.append('GRAPH')
        if temporal_result:
            fired.append('TEMPORAL')
        if memory_result:
            fired.append('MEMORY_NETWORK')
        if self._has_observed_teammate_alert(teammate_alerts):
            fired.append('TEAMMATE_OBSERVED')
        return list(dict.fromkeys(fired))
    def _calculate_agreement_bonus(self, observed_layers: List[str]) -> float:
        count = len(set(observed_layers))
        if count >= 4:
            return 0.1
        if count == 3:
            return 0.07
        if count == 2:
            return 0.03
        return 0.0
    def _has_forecast_support(self, teammate_alerts: List[Dict]) -> bool:
        return any((self._is_forecast_alert(alert) for alert in teammate_alerts if isinstance(alert,
            dict)))
    def _has_observed_teammate_alert(self, teammate_alerts: List[Dict]) -> bool:
        return any((not self._is_forecast_alert(alert) for alert in teammate_alerts if isinstance(alert,
            dict)))
    @staticmethod
    def _is_forecast_alert(alert: Dict) -> bool:
        layer = str(alert.get('layer', '')).strip().lower()
        source = str(alert.get('source_component', alert.get('from', ''))).strip().lower()
        event_type = str(alert.get('event_type', alert.get('alert_type', ''))).strip().upper()
        return bool(layer == 'forecast' or source in {'component_3', 'threat_forecaster'} or
'FORECAST' in
            event_type)
    def _collect_mitre(self, ml_result: Optional[Dict], graph_result: Optional[Dict],
temporal_result:
        Optional[Dict], memory_result: Optional[Dict], teammate_alerts: List[Dict]) -> List[Dict]:
        techniques: Set[str] = set()
```

```
        if temporal_result:
            mitre = temporal_result.get('mitre')
            if mitre:
                techniques.add(str(mitre))
        if graph_result:
            techniques.add('T1021')
            edge_details = graph_result.get('edge_details', [])
            for edge in edge_details:
                if not isinstance(edge, dict):
                    continue
                ports = edge.get('ports', [])
                if 445 in ports:
                    techniques.add('T1021.002')
                if 3389 in ports:
                    techniques.add('T1021.001')
                if 135 in ports:
                    techniques.add('T1021.003')
        if memory_result:
            indicators = {str(indicator).upper() for indicator in memory_result.get('indicators', [])}
            if 'CODE_INJECTION' in indicators:
                techniques.add('T1055.003')
            if 'PROCESS_HOLLOWING' in indicators:
                techniques.add('T1055.012')
            if 'CREDENTIAL_MEMORY_ACCESS' in indicators:
                techniques.add('T1003.001')
        for alert in teammate_alerts:
            if not isinstance(alert, dict):
                continue
            mitre = str(alert.get('mitre_technique', '')).strip()
            if mitre:
                techniques.add(mitre)
        if ml_result:
            mitre = str(ml_result.get('mitre_technique', '')).strip()
            if mitre:
                techniques.add(mitre)
        return [{'technique_id': technique, 'description':
self.MITRE_DESCRIPTIONS.get(technique, 'MITRE
            ATT&CK technique')} for technique in sorted(techniques)]
    def _extract_xai(self, ml_result: Optional[Dict]) -> Tuple[List[float], List[str], List[Dict]]:
        if not ml_result:
            return ([], [], [])
        shap_values = ml_result.get('shap_values', []) or []
        feature_names = ml_result.get('features_used', []) or []
        top_features = ml_result.get('top_features', []) or []
        xgb_result = ml_result.get('xgboost')
        if isinstance(xgb_result, dict):
            if not shap_values:
                shap_values = xgb_result.get('shap_values', []) or []
            if not feature_names:
                feature_names = xgb_result.get('features_used', []) or []
            if not top_features:
                top_features = xgb_result.get('top_features', []) or []
        return (list(shap_values), list(feature_names), list(top_features))
    def _collect_evidence_event_ids(self, raw_event: Dict, graph_result: Optional[Dict],
```

```
temporal_result:
        Optional[Dict], memory_result: Optional[Dict], teammate_alerts: List[Dict]) -> List[str]:
        evidence_ids: List[str] = []
        def add(value: Any) -> None:
            text = str(value or '').strip()
            if text and text not in evidence_ids:
                evidence_ids.append(text)
        add(raw_event.get('event_id'))
        if graph_result:
            for event_id in graph_result.get('matched_event_ids', []):
                add(event_id)
        if temporal_result:
            for event_id in temporal_result.get('matched_event_ids', []):
                add(event_id)
        if memory_result:
            for event_id in memory_result.get('matched_event_ids', []):
                add(event_id)
        for alert in teammate_alerts:
            if not isinstance(alert, dict):
                continue
            add(alert.get('event_id', alert.get('alert_id')))
        return evidence_ids
    @staticmethod
    def _derive_source_device(raw_event: Dict, graph_result: Optional[Dict], temporal_result:
        Optional[Dict], memory_result: Optional[Dict], teammate_alerts: List[Dict]) -> str:
        source = str(raw_event.get('source_device', raw_event.get('source_ip', '')) or '').strip()
        if source:
            return source
        if memory_result:
            source = str(memory_result.get('source_device', '') or '').strip()
            if source:
                return source
        if graph_result:
            path = graph_result.get('path', [])
            if path:
                return str(path[0])
        if temporal_result:
            devices = temporal_result.get('devices', [])
            if devices:
                return str(devices[0])
        for alert in teammate_alerts:
            if not isinstance(alert, dict):
                continue
            source = str(alert.get('source_device', alert.get('device', '')) or '').strip()
            if source:
                return source
        return 'unknown'
    @staticmethod
    def _derive_destination_device(raw_event: Dict, graph_result: Optional[Dict],
temporal_result:
        Optional[Dict], memory_result: Optional[Dict], teammate_alerts: List[Dict]) -> str:
        destination = str(raw_event.get('destination_device', raw_event.get('destination_ip', '')) or
            '').strip()
        if destination:
```

```
            return destination
        if graph_result:
            path = graph_result.get('path', [])
            if len(path) >= 2:
                return str(path[-1])
        if temporal_result:
            destinations = temporal_result.get('destination_devices', [])
            if destinations:
                return str(destinations[-1])
        if memory_result:
            network_events = memory_result.get('network_events', [])
            if network_events:
                destination = str(network_events[-1].get('destination_device',
                    network_events[-1].get('destination_ip', '')) or '').strip()
                if destination:
                    return destination
        for alert in teammate_alerts:
            if not isinstance(alert, dict):
                continue
            destination = str(alert.get('destination_device', '') or '').strip()
            if destination:
                return destination
        return ''
    def _create_incident_fingerprint(self, source_device: str, destination_device: str, lateral_path:
        List[str], temporal_result: Optional[Dict], memory_result: Optional[Dict],
teammate_alerts:
        List[Dict]) -> str:
        temporal_pattern = ''
        if temporal_result:
            temporal_pattern = str(temporal_result.get('pattern_name', ''))
        memory_indicators = []
        if memory_result:
            memory_indicators = sorted((str(indicator).upper() for indicator in
                memory_result.get('strong_indicators', memory_result.get('indicators', []))))
        teammate_sources = sorted({str(alert.get('source_component', alert.get('from', ''))).lower()
for
            alert in teammate_alerts if isinstance(alert, dict) and (not self._is_forecast_alert(alert))})
        data = {'source_device': str(source_device).lower(), 'destination_device':
            str(destination_device).lower(), 'path': [str(device).lower() for device in lateral_path],
            'temporal_pattern': temporal_pattern, 'memory_indicators': memory_indicators,
            'teammate_sources': teammate_sources}
        payload = json.dumps(data, sort_keys=True)
        return hashlib.sha256(payload.encode('utf-8')).hexdigest()
    def _find_recent_incident(self, fingerprint: str) -> Optional[Dict]:
        record = self.recent_incidents.get(fingerprint)
        if not record:
            return None
        last_seen = record.get('last_seen')
        if not isinstance(last_seen, datetime):
            return None
        cooldown = timedelta(minutes=self.duplicate_cooldown_minutes)
        if datetime.now(timezone.utc) - last_seen > cooldown:
            self.recent_incidents.pop(fingerprint, None)
            return None
```

```
        alert_id = record.get('alert_id')
        for alert in reversed(self.alert_history):
            if alert.get('alert_id') == alert_id:
                return alert
        return None
    def _update_existing_alert(self, alert: Dict, risk_score: float, base_risk_score: float,
        agreement_bonus: float, scores: Dict, layers_fired: List[str], evidence_event_ids: List[str],
        ml_result: Optional[Dict], graph_result: Optional[Dict], temporal_result: Optional[Dict],
        memory_result: Optional[Dict], teammate_alerts: List[Dict], mitre_techniques: List[Dict],
        lateral_path: List[str], shap_values: List[float], feature_names: List[str], top_features:
        List[Dict]) -> Dict:
        alert['occurrence_count'] = int(alert.get('occurrence_count', 1)) + 1
        alert['last_seen'] = datetime.now(timezone.utc).isoformat()
        alert['risk_score'] = max(float(alert.get('risk_score', 0.0)), risk_score)
        alert['severity'] = self._classify_severity(alert['risk_score'])
        alert['base_risk_score'] = round(base_risk_score, 4)
        alert['agreement_bonus'] = round(agreement_bonus, 4)
        alert['score_breakdown'] = self._build_score_breakdown(scores)
        alert['signal_weights'] = self._get_effective_weights(scores)
        existing_layers = list(alert.get('layers_fired', []))
        for layer in layers_fired:
            if layer not in existing_layers:
                existing_layers.append(layer)
        alert['layers_fired'] = existing_layers
        existing_ids = list(alert.get('evidence_event_ids', []))
        for event_id in evidence_event_ids:
            if event_id not in existing_ids:
                existing_ids.append(event_id)
        alert['evidence_event_ids'] = existing_ids
        alert['evidence_count'] = len(existing_ids)
        if lateral_path:
            alert['lateral_movement_path'] = lateral_path
        alert['mitre_techniques'] = self._merge_mitre(alert.get('mitre_techniques', []),
mitre_techniques)
        if shap_values:
            alert['shap_values'] = shap_values
            alert['feature_names'] = feature_names
            alert['top_features'] = top_features
            alert['shap_available'] = True
        if ml_result:
            alert['ml_result'] = ml_result
        if graph_result:
            alert['graph_result'] = graph_result
        if temporal_result:
            alert['temporal_result'] = temporal_result
        if memory_result:
            alert['memory_result'] = memory_result
        if teammate_alerts:
            alert['teammate_alerts'] = teammate_alerts
        alert['is_update'] = True
        fingerprint = alert.get('fingerprint')
        if fingerprint:
            self.recent_incidents[fingerprint] = {'alert_id': alert['alert_id'], 'last_seen':
                datetime.now(timezone.utc)}
```

```
        print(f"[Fusion] UPDATED ALERT {alert['alert_id']} |
Occurrences={alert['occurrence_count']} |
            Risk={alert['risk_score']:.4f}")
        return alert
    @staticmethod
    def _safe_float(value: Any, default: float=0.0) -> float:
        try:
            return float(value if value is not None else default)
        except (TypeError, ValueError, OverflowError):
            return default
    @classmethod
    def _clamp_score(cls, value: Any) -> float:
        number = cls._safe_float(value)
        return max(0.0, min(number, 1.0))
    @staticmethod
    def _generate_alert_id() -> str:
        return 'ALERT-' + str(uuid.uuid4())[:8].upper()
    @staticmethod
    def _classify_severity(risk_score: float) -> str:
        if risk_score >= 0.85:
            return 'CRITICAL'
        if risk_score >= 0.65:
            return 'HIGH'
        if risk_score >= 0.45:
            return 'MEDIUM'
        return 'LOW'
    @staticmethod
    def _build_score_breakdown(scores: Dict[str, Optional[float]]) -> Dict[str, Optional[float]]:
        return {signal_name: round(value, 4) if value is not None else None for signal_name, value
in
            scores.items()}
    @staticmethod
    def _merge_mitre(existing: List[Dict], new: List[Dict]) -> List[Dict]:
        merged: Dict[str, Dict] = {}
        for item in existing + new:
            if not isinstance(item, dict):
                continue
            technique_id = str(item.get('technique_id', ''))
            if technique_id:
                merged[technique_id] = item
        return [merged[technique_id] for technique_id in sorted(merged)]
    def get_alert_history(self) -> List[Dict]:
        return list(self.alert_history)
    def get_recent_alerts(self, last_n: int=10) -> List[Dict]:
        if last_n <= 0:
            return []
        return self.alert_history[-last_n:]
    def get_alert(self, alert_id: str) -> Optional[Dict]:
        for alert in self.alert_history:
            if alert.get('alert_id') == alert_id:
                return alert
        return None
    def get_summary(self) -> Dict:
        if not self.alert_history:
```

```
            return {'total_alerts': 0, 'critical_alerts': 0, 'high_alerts': 0, 'medium_alerts': 0,
                'low_alerts': 0, 'avg_risk_score': 0.0}
        severities = [alert.get('severity', 'LOW') for alert in self.alert_history]
        average_risk = sum((float(alert.get('risk_score', 0.0)) for alert in self.alert_history)) /
            len(self.alert_history)
        return {'total_alerts': len(self.alert_history), 'critical_alerts':
severities.count('CRITICAL'),
            'high_alerts': severities.count('HIGH'), 'medium_alerts': severities.count('MEDIUM'),
            'low_alerts': severities.count('LOW'), 'avg_risk_score': round(average_risk, 4)}
```

A.11 Orchestrator and Model Eligibility

This listing shows event processing, compatible-model routing, related-signal lookup, observed-family counting, cleanup and persistence coordination. It is relevant to the premature-alert investigation because it identifies the normal entry path to fusion. Source existence is not evidence that every branch has been exercised.

Source: src/engine/correlation_orchestrator.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
from __future__ import annotations
from collections import deque
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Any, Deque, Dict, List, Optional, Set
import sys
PROJECT_ROOT = Path(__file__).resolve().parents[2]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
from src.preprocessing.normalizer import EventNormalizer
from src.preprocessing.feature_extraction import FeatureExtractor
from src.engine.ml_detector import MLDetector
from src.engine.graph_engine import DeviceGraphEngine
from src.engine.temporal_correlator import TemporalCorrelator
from src.engine.memory_correlator import MemoryCorrelator
from src.engine.alert_fusion import AlertFusion
from src.database.database import DatabaseManager
from src.database.repositories import Component4Repository
from src.config.settings import settings
class CorrelationOrchestrator:
    def __init__(self, models_path: Optional[str]=None, correlation_window_minutes:
Optional[int]=None,
        memory_window_seconds: Optional[int]=None, teammate_window_minutes:
Optional[int]=None,
        minimum_fusion_signal_families: Optional[int]=None, database_path: Optional[str]=None,
        enable_persistence: Optional[bool]=None):
        resolved_models_path = models_path if models_path is not None else
str(settings.models_path)
        resolved_database_path = database_path if database_path is not None else
            str(settings.database_path)
        resolved_correlation_window_minutes = int(correlation_window_minutes if
correlation_window_minutes
             is not None else settings.correlation_window_minutes)
        resolved_memory_window_seconds = int(memory_window_seconds if
memory_window_seconds is not None
            else settings.memory_window_seconds)
        resolved_teammate_window_minutes = int(teammate_window_minutes if
teammate_window_minutes is not
            None else settings.teammate_window_minutes)
        resolved_minimum_fusion_signal_families = int(minimum_fusion_signal_families if
            minimum_fusion_signal_families is not None else
settings.minimum_fusion_signal_families)
```

```
        resolved_enable_persistence = bool(enable_persistence) if enable_persistence is not None
else
            bool(settings.enable_persistence)
        if resolved_correlation_window_minutes <= 0:
            raise ValueError('correlation_window_minutes must be greater than 0')
        if resolved_memory_window_seconds <= 0:
            raise ValueError('memory_window_seconds must be greater than 0')
        if resolved_teammate_window_minutes <= 0:
            raise ValueError('teammate_window_minutes must be greater than 0')
        if resolved_minimum_fusion_signal_families < 1:
            raise ValueError('minimum_fusion_signal_families must be at least 1')
        self.models_path = resolved_models_path
        self.correlation_window_minutes = resolved_correlation_window_minutes
        self.memory_window_seconds = resolved_memory_window_seconds
        self.teammate_window_minutes = resolved_teammate_window_minutes
        self.minimum_fusion_signal_families = resolved_minimum_fusion_signal_families
        self.enable_persistence = resolved_enable_persistence
        self.database = None
        self.repository = None
        if self.enable_persistence:
            self.database = DatabaseManager(database_path=resolved_database_path)
            self.repository = Component4Repository(self.database)
            print('[Orchestrator] SQLite persistence enabled')
        else:
            print('[Orchestrator] SQLite persistence disabled')
        self.normalizer = EventNormalizer()
        self.feature_extractor = FeatureExtractor()
        self.ml_detector = MLDetector(models_path=resolved_models_path)
        self.graph_engine =
DeviceGraphEngine(window_minutes=resolved_correlation_window_minutes,
            max_path_hops=settings.graph_max_path_hops,
            duplicate_cooldown_minutes=settings.graph_duplicate_cooldown_minutes)
        self.temporal_correlator =
TemporalCorrelator(window_minutes=resolved_correlation_window_minutes,
            duplicate_cooldown_minutes=settings.temporal_duplicate_cooldown_minutes)
        self.memory_correlator =
            MemoryCorrelator(correlation_window_seconds=resolved_memory_window_seconds,
            duplicate_cooldown_seconds=settings.memory_duplicate_cooldown_seconds)
        self.alert_fusion = AlertFusion(alert_threshold=settings.alert_threshold,
            duplicate_cooldown_minutes=settings.fusion_duplicate_cooldown_minutes)
        self.event_history: Deque[Dict] = deque(maxlen=10000)
        self.teammate_alert_buffer: Deque[Dict] = deque(maxlen=2000)
        self.recent_ml_results: Deque[Dict] = deque(maxlen=2000)
        self.recent_graph_results: Deque[Dict] = deque(maxlen=1000)
        self.recent_temporal_results: Deque[Dict] = deque(maxlen=1000)
        self.recent_memory_results: Deque[Dict] = deque(maxlen=1000)
        self.events_processed = 0
        self.final_alerts_generated = 0
        print('[Orchestrator] Component 4 engine initialized')
    def ingest_event(self, raw_event: Dict, source_type: str) -> Dict:
        if not isinstance(raw_event, dict):
            raise TypeError('raw_event must be a dictionary')
        self.events_processed += 1
        unified_event = self.normalizer.normalize(raw_event, source_type)
```

```
        event = unified_event.to_dict()
        self.event_history.append(event)
        event_time = self._parse_timestamp(event.get('timestamp'))
        feature_values = self.feature_extractor.extract_single_named(event)
        event['cross_layer_features'] = feature_values
        if self.enable_persistence and self.repository:
            self.repository.save_event(event)
        if source_type == 'teammate_alert':
            self.teammate_alert_buffer.append(event)
        ml_result = self._run_compatible_ml(event)
        if self._is_valid_ml_result(ml_result):
            self._store_signal_result(buffer=self.recent_ml_results, result=ml_result, event=event,
                timestamp=event_time)
        graph_result = None
        if event.get('layer') == 'network':
            graph_result = self.graph_engine.add_event(event)
            if graph_result:
                self._store_signal_result(buffer=self.recent_graph_results, result=graph_result,
                    event=event, timestamp=event_time, extra_entities=graph_result.get('path', []))
            self._enrich_network_event_with_graph_novelty(event=event,
graph_result=graph_result)
        temporal_result = self.temporal_correlator.add_event(event)
        if temporal_result:
            extra_entities = []
            extra_entities.extend(temporal_result.get('devices', []))
            extra_entities.extend(temporal_result.get('destination_devices', []))
            self._store_signal_result(buffer=self.recent_temporal_results, result=temporal_result,
                event=event, timestamp=event_time, extra_entities=extra_entities)
        memory_result = self.memory_correlator.process_event(event)
        if memory_result:
            extra_entities = [memory_result.get('source_device', '')]
            self._store_signal_result(buffer=self.recent_memory_results, result=memory_result,
                event=event, timestamp=event_time, extra_entities=extra_entities)
        self._cleanup_recent_state()
        event_entities = self._get_event_entities(event)
        related_ml = ml_result if self._is_valid_ml_result(ml_result) else
            self._find_latest_related_signal(self.recent_ml_results, event_entities)
        related_graph = graph_result or self._find_latest_related_signal(self.recent_graph_results,
            event_entities)
        related_temporal = temporal_result or
            self._find_latest_related_signal(self.recent_temporal_results, event_entities)
        related_memory = memory_result or
self._find_latest_related_signal(self.recent_memory_results,
            event_entities)
        related_teammates = self._find_related_teammate_alerts(event)
        signal_families = self._count_observed_signal_families(ml_result=related_ml,
            graph_result=related_graph, temporal_result=related_temporal,
memory_result=related_memory,
            teammate_alerts=related_teammates)
        final_alert = None
        if signal_families >= self.minimum_fusion_signal_families:
            final_alert = self.alert_fusion.fuse(ml_result=related_ml, graph_result=related_graph,
                temporal_result=related_temporal, memory_result=related_memory,
                teammate_alerts=related_teammates, source_device=event.get('source_device', ''),
```

```
                raw_event=event)
        if final_alert:
            if not final_alert.get('is_update', False):
                self.final_alerts_generated += 1
            if self.enable_persistence and self.repository:
                self._persist_final_alert(final_alert)
        return {'processing_number': self.events_processed, 'source_type': source_type, 'event':
event,
            'cross_layer_features': feature_values, 'ml_result': ml_result, 'graph_result':
graph_result,
            'temporal_result': temporal_result, 'memory_result': memory_result,
'related_ml_result':
            related_ml, 'related_graph_result': related_graph, 'related_temporal_result':
            related_temporal, 'related_memory_result': related_memory, 'teammate_alerts':
            related_teammates, 'signal_families': signal_families, 'final_alert': final_alert}
    @staticmethod
    def _enrich_network_event_with_graph_novelty(event: Dict, graph_result: Optional[Dict]) ->
None:
        if not isinstance(event, dict):
            return
        if str(event.get('layer', '')).strip().lower() != 'network':
            return
        if not isinstance(graph_result, dict):
            return
        event_id = str(event.get('event_id', '')).strip()
        if not event_id:
            return
        edge_details = graph_result.get('edge_details', [])
        if not isinstance(edge_details, list):
            return
        for detail in edge_details:
            if not isinstance(detail, dict):
                continue
            detail_event_id = str(detail.get('event_id', '')).strip()
            if detail_event_id != event_id:
                continue
            if 'is_new' not in detail:
                return
            novelty = detail.get('is_new')
            if novelty is None:
                return
            event['service_is_new'] = bool(novelty)
            event['novelty_source'] = 'graph_engine'
            return
    def _run_compatible_ml(self, event: Dict) -> Optional[Dict]:
        layer = str(event.get('layer', 'unknown')).strip().lower()
        if layer == 'memory':
            return self.ml_detector.predict_memory_event(event)
        if layer == 'network':
            ml_event = self._prepare_network_ml_event(event)
            if not self._has_complete_unsw_features(ml_event):
                return {'model': None, 'prediction': None, 'is_malicious': None, 'probability': None,
                    'confidence': None, 'layer': 'network', 'status': 'insufficient_network_features',
                    'error': 'Event does not contain all 37 UNSW-NB15 model features.', 'shap_values':
```

```
[],
                'top_features': []}
            return self.ml_detector.predict_network_ensemble(ml_event)
        return {'model': None, 'prediction': None, 'is_malicious': None, 'probability': None,
            'confidence': None, 'layer': layer, 'status': 'no_compatible_ml_model', 'error': None,
            'shap_values': [], 'top_features': []}
    def _prepare_network_ml_event(self, event: Dict) -> Dict:
        ml_event = {}
        raw = event.get('raw', {})
        if isinstance(raw, dict):
            ml_event.update(raw)
        ml_event.update(event)
        return ml_event
    def _has_complete_unsw_features(self, event: Dict) -> bool:
        for feature_name in self.ml_detector.UNSW_FEATURES:
            if feature_name not in event:
                return False
            value = event.get(feature_name)
            if value is None:
                return False
        return True
    def _store_signal_result(self, buffer: Deque[Dict], result: Dict, event: Dict, timestamp:
datetime,
        extra_entities: Optional[List[Any]]=None) -> None:
        entities = self._get_event_entities(event)
        if extra_entities:
            for entity in extra_entities:
                normalized = self._normalize_entity(entity)
                if normalized:
                    entities.add(normalized)
        buffer.append({'timestamp': timestamp, 'entities': entities, 'result': result})
    @staticmethod
    def _normalize_entity(value: Any) -> str:
        return str(value or '').strip().lower()
    def _get_event_entities(self, event: Dict) -> Set[str]:
        entities: Set[str] = set()
        for key in ['source_device', 'destination_device', 'source_ip', 'destination_ip']:
            value = self._normalize_entity(event.get(key))
            if value:
                entities.add(value)
        return entities
    def _find_latest_related_signal(self, buffer: Deque[Dict], event_entities: Set[str]) ->
        Optional[Dict]:
        if not event_entities:
            return None
        for record in reversed(buffer):
            entities = record.get('entities', set())
            if event_entities.intersection(entities):
                return record.get('result')
        return None
    def _find_related_teammate_alerts(self, event: Dict) -> List[Dict]:
        event_entities = self._get_event_entities(event)
        if not event_entities:
            return []
```

```
        event_time = self._parse_timestamp(event.get('timestamp'))
        window = timedelta(minutes=self.teammate_window_minutes)
        related: List[Dict] = []
        for alert in self.teammate_alert_buffer:
            alert_entities = self._get_event_entities(alert)
            if not event_entities.intersection(alert_entities):
                continue
            alert_time = self._parse_timestamp(alert.get('timestamp'))
            if abs((event_time - alert_time).total_seconds()) > window.total_seconds():
                continue
            related.append(alert)
        return related
    def _count_observed_signal_families(self, ml_result: Optional[Dict], graph_result:
Optional[Dict],
        temporal_result: Optional[Dict], memory_result: Optional[Dict], teammate_alerts:
List[Dict]) ->
        int:
        families: Set[str] = set()
        if ml_result and ml_result.get('status') == 'success' and bool(ml_result.get('is_malicious')):
            families.add('ml')
        if graph_result:
            families.add('graph')
        if temporal_result:
            families.add('temporal')
        if memory_result:
            families.add('memory')
        for alert in teammate_alerts:
            if not isinstance(alert, dict):
                continue
            if not self._is_forecast_teammate(alert):
                families.add('teammate')
                break
        return len(families)
    @staticmethod
    def _is_forecast_teammate(alert: Dict) -> bool:
        layer = str(alert.get('layer', '')).strip().lower()
        source = str(alert.get('source_component', alert.get('teammate_source', ''))).strip().lower()
        event_type = str(alert.get('event_type', '')).upper()
        return layer == 'forecast' or source in {'component_3', 'threat_forecaster'} or
'FORECAST' in
            event_type
    @staticmethod
    def _is_valid_ml_result(result: Optional[Dict]) -> bool:
        if not result:
            return False
        return result.get('status') == 'success'
    def _cleanup_recent_state(self) -> None:
        cutoff = datetime.now(timezone.utc) -
timedelta(minutes=max(self.correlation_window_minutes,
            self.teammate_window_minutes))
        self.recent_ml_results = self._clean_signal_buffer(self.recent_ml_results, cutoff)
        self.recent_graph_results = self._clean_signal_buffer(self.recent_graph_results, cutoff)
        self.recent_temporal_results = self._clean_signal_buffer(self.recent_temporal_results,
cutoff)
```

```
    self.recent_memory_results = self._clean_signal_buffer(self.recent_memory_results, cutoff)
    teammate_maxlen = self.teammate_alert_buffer.maxlen
    valid_teammates = []
    for alert in self.teammate_alert_buffer:
        try:
            timestamp = self._parse_timestamp(alert.get('timestamp'))
        except Exception:
            continue
        if timestamp >= cutoff:
            valid_teammates.append(alert)
    self.teammate_alert_buffer = deque(valid_teammates, maxlen=teammate_maxlen)
@staticmethod
def _clean_signal_buffer(buffer: Deque[Dict], cutoff: datetime) -> Deque[Dict]:
    maxlen = buffer.maxlen
    retained = [record for record in buffer if isinstance(record.get('timestamp'), datetime) and
        record['timestamp'] >= cutoff]
    return deque(retained, maxlen=maxlen)
@staticmethod
def _parse_timestamp(value: Any) -> datetime:
    if isinstance(value, datetime):
        parsed = value
    else:
        text = str(value or '').strip()
        if not text:
            return datetime.now(timezone.utc)
        parsed = datetime.fromisoformat(text.replace('Z', '+00:00'))
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=timezone.utc)
    return parsed.astimezone(timezone.utc)
def _persist_final_alert(self, alert: Dict) -> None:
    if not (self.enable_persistence and self.repository):
        return
    alert_id = self.repository.save_alert(alert)
    evidence_ids = alert.get('evidence_event_ids', [])
    if isinstance(evidence_ids, list):
        self.repository.save_alert_evidence(alert_id=alert_id, event_ids=evidence_ids,
            evidence_type='correlation')
    explanation_text = self._build_xai_explanation_text(alert)
    self.repository.save_xai_explanation(alert, explanation_text=explanation_text)
@staticmethod
def _build_xai_explanation_text(alert: Dict) -> str:
    parts = []
    risk_score = alert.get('risk_score')
    severity = str(alert.get('severity', '')).strip()
    if risk_score is not None:
        try:
            parts.append(f'Final {severity.lower()} cross-layer risk score was
                {float(risk_score):.2f}.')
        except (TypeError, ValueError):
            pass
    layers = alert.get('layers_fired', [])
    if isinstance(layers, list) and layers:
        parts.append('Correlated evidence was observed from ' + ', '.join((str(layer) for layer in
            layers)) + '.')
```

```
        path = alert.get('lateral_movement_path', [])
        if isinstance(path, list) and len(path) >= 2:
            parts.append('Observed lateral movement path: ' + ' -> '.join((str(device) for device in
                path)) + '.')
        top_features = alert.get('top_features', [])
        if isinstance(top_features, list) and top_features:
            feature_names = [str(feature.get('feature', '')) for feature in top_features[:3] if
                isinstance(feature, dict) and feature.get('feature')]
            if feature_names:
                parts.append('The strongest XGBoost SHAP features were ' + ', '.join(feature_names)
+ '.')
        mitre = alert.get('mitre_techniques', [])
        if isinstance(mitre, list) and mitre:
            technique_ids = [str(item.get('technique_id', '')) for item in mitre if isinstance(item, dict)
                 and item.get('technique_id')]
            if technique_ids:
                parts.append('Mapped MITRE ATT&CK techniques: ' + ', '.join(technique_ids) + '.')
        return ' '.join(parts)
    def get_database_stats(self) -> Dict:
        if not (self.enable_persistence and self.repository):
            return {'persistence_enabled': False}
        stats = self.repository.get_stats()
        stats['persistence_enabled'] = True
        stats['database_path'] = str(self.database.database_path)
        return stats
    def build_graph_baseline(self, normal_events: List[Dict]) -> None:
        normalized_events = []
        for event in normal_events:
            if isinstance(event, dict):
                normalized_events.append(event)
        self.graph_engine.build_baseline(normalized_events)
    def register_device(self, device: str, device_type: str) -> None:
        self.graph_engine.register_device(device, device_type)
    def get_summary(self) -> Dict:
        return {'events_processed': self.events_processed, 'final_alerts_generated':
            self.final_alerts_generated, 'events_in_memory': len(self.event_history),
            'teammate_alerts_buffered': len(self.teammate_alert_buffer), 'ml':
            self.ml_detector.get_model_status(), 'graph': self.graph_engine.get_graph_summary(),
            'temporal': self.temporal_correlator.get_summary(), 'memory':
            self.memory_correlator.get_summary(), 'fusion': self.alert_fusion.get_summary(),
'database':
            self.get_database_stats()}
    def get_recent_events(self, last_n: int=50) -> List[Dict]:
        if last_n <= 0:
            return []
        return list(self.event_history)[-last_n:]
    def get_recent_alerts(self, last_n: int=10) -> List[Dict]:
        return self.alert_fusion.get_recent_alerts(last_n)
```

A.12 Graph Construction and Path Evaluation

This listing provides the graph baseline, event insertion, candidate-path search, path evaluation and graph-output logic. It supports inspection of the relation between expected topology and observed activity. A path is an analytical representation whose correctness still requires scenario-level validation.

Source: src/engine/graph_engine.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
from __future__ import annotations
from datetime import datetime, timedelta, timezone
from typing import Any, Dict, List, Optional, Set, Tuple
import hashlib
import json
import networkx as nx
class DeviceGraphEngine:
    CRITICAL_DEVICE_TYPES = {'ehr_server', 'pacs', 'pacs_server', 'infusion_pump',
'ventilator',
        'patient_monitor', 'medical_gateway', 'dicom_server'}
    LATERAL_PORTS = {445, 3389, 5985, 5986, 22, 135, 139, 5900}
    def __init__(self, window_minutes: int=10, max_path_hops: int=4,
duplicate_cooldown_minutes: int=5):
        if window_minutes <= 0:
            raise ValueError('window_minutes must be greater than zero')
        if max_path_hops < 2:
            raise ValueError('max_path_hops must be at least 2')
        self.window_minutes = int(window_minutes)
        self.max_path_hops = int(max_path_hops)
        self.duplicate_cooldown_minutes = int(duplicate_cooldown_minutes)
        self.live_graph = nx.DiGraph()
        self.baseline_graph = nx.DiGraph()
        self.detected_paths: List[Dict] = []
        self.device_registry: Dict[str, str] = {}
        self.recent_path_fingerprints: Dict[str, datetime] = {}
        self.event_count = 0
    @staticmethod
    def _parse_timestamp(value: Any) -> datetime:
        if isinstance(value, datetime):
            parsed = value
        else:
            text = str(value or '').strip()
            if not text:
                return datetime.now(timezone.utc)
            parsed = datetime.fromisoformat(text.replace('Z', '+00:00'))
        if parsed.tzinfo is None:
            parsed = parsed.replace(tzinfo=timezone.utc)
        return parsed.astimezone(timezone.utc)
    @classmethod
    def _normalize_timestamp(cls, value: Any) -> str:
        try:
            parsed = cls._parse_timestamp(value)
```

```
            except (TypeError, ValueError, OverflowError):
                parsed = datetime.now(timezone.utc)
            return parsed.isoformat()
        @staticmethod
        def _safe_int(value: Any, default: int=0) -> int:
            try:
                return int(float(value if value is not None else default))
            except (TypeError, ValueError, OverflowError):
                return default
        @staticmethod
        def _normalize_device(value: Any) -> str:
            return str(value or '').strip()
        def register_device(self, device: str, device_type: str) -> None:
            device = self._normalize_device(device)
            if not device:
                return
            self.device_registry[device] = str(device_type or 'unknown').strip().lower()
        def build_baseline(self, normal_events: List[Dict]) -> None:
            if not isinstance(normal_events, list):
                raise TypeError('normal_events must be a list')
            for event in normal_events:
                if not isinstance(event, dict):
                    continue
                src = self._normalize_device(event.get('source_device', event.get('source_ip', '')))
                dst = self._normalize_device(event.get('destination_device', event.get('destination_ip',
'')))
                if not src or not dst or src == dst:
                    continue
                timestamp = self._normalize_timestamp(event.get('timestamp'))
                protocol = str(event.get('protocol', '')).strip().lower()
                port = self._safe_int(event.get('destination_port', event.get('dst_port', 0)))
                if self.baseline_graph.has_edge(src, dst):
                    edge = self.baseline_graph[src][dst]
                    edge['count'] += 1
                    edge['last_seen'] = timestamp
                    if protocol:
                        edge['protocols'].add(protocol)
                    if port:
                        edge['ports'].add(port)
                    edge.setdefault('services', set())
                    if protocol or port:
                        edge['services'].add((protocol, port))
                else:
                    protocols: Set[str] = set()
                    ports: Set[int] = set()
                    if protocol:
                        protocols.add(protocol)
                    if port:
                        ports.add(port)
                    services: Set[Tuple[str, int]] = set()
                    if protocol or port:
                        services.add((protocol, port))
                    self.baseline_graph.add_edge(src, dst, count=1, protocols=protocols, ports=ports,
                        services=services, first_seen=timestamp, last_seen=timestamp)
```

```
        print(f'[Graph] Baseline built: {self.baseline_graph.number_of_nodes()} devices,
            {self.baseline_graph.number_of_edges()} normal connections learned')
    def _is_baseline_service_known(self, src: str, dst: str, protocol: str, destination_port: int) ->
        bool:
        if not self.baseline_graph.has_edge(src, dst):
            return False
        edge = self.baseline_graph[src][dst]
        protocol = str(protocol or '').strip().lower()
        destination_port = self._safe_int(destination_port, 0)
        services = {(str(service_protocol or '').strip().lower(), self._safe_int(service_port, 0)) for
            service_protocol, service_port in edge.get('services', set())}
        if not services:
            baseline_protocols = {str(item).strip().lower() for item in edge.get('protocols', set())}
            baseline_ports = {self._safe_int(item, 0) for item in edge.get('ports', set())}
            if protocol and destination_port:
                return protocol in baseline_protocols and destination_port in baseline_ports
            if protocol:
                return protocol in baseline_protocols
            if destination_port:
                return destination_port in baseline_ports
            return True
        if protocol and destination_port:
            return (protocol, destination_port) in services
        if protocol:
            return any((service_protocol == protocol for service_protocol, _ in services))
        if destination_port:
            return any((service_port == destination_port for _, service_port in services))
        return True
    def add_event(self, event: Dict) -> Optional[Dict]:
        if not isinstance(event, dict):
            raise TypeError('event must be a dictionary')
        self.event_count += 1
        src = self._normalize_device(event.get('source_device', event.get('source_ip', '')))
        dst = self._normalize_device(event.get('destination_device', event.get('destination_ip', '')))
        if not src or not dst or src == dst:
            return None
        event_id = str(event.get('event_id', f'graph-event-{self.event_count}'))
        timestamp = self._normalize_timestamp(event.get('timestamp'))
        protocol = str(event.get('protocol', '')).strip().lower()
        destination_port = self._safe_int(event.get('destination_port', event.get('dst_port', 0)))
        layer = str(event.get('layer', 'network')).strip().lower()
        label = str(event.get('label', 'unknown')).strip().lower()
        baseline_service_known = self._is_baseline_service_known(src=src, dst=dst,
protocol=protocol,
            destination_port=destination_port)
        is_new = not baseline_service_known
        if self.live_graph.has_edge(src, dst):
            edge = self.live_graph[src][dst]
            edge['count'] += 1
            edge['last_seen'] = timestamp
            edge['is_new'] = bool(edge.get('is_new', False) or is_new)
            edge['event_ids'].append(event_id)
            edge['timestamps'].append(timestamp)
            edge.setdefault('observation_protocols', []).append(protocol)
```

```
            edge.setdefault('observation_ports', []).append(destination_port)
            edge.setdefault('observation_is_new', []).append(is_new)
            if protocol:
                edge['protocols'].add(protocol)
            if destination_port:
                edge['ports'].add(destination_port)
            edge['latest_label'] = label
        else:
            protocols: Set[str] = set()
            ports: Set[int] = set()
            if protocol:
                protocols.add(protocol)
            if destination_port:
                ports.add(destination_port)
            self.live_graph.add_edge(src, dst, count=1, first_seen=timestamp, last_seen=timestamp,
                layer=layer, protocols=protocols, ports=ports, event_ids=[event_id],
                timestamps=[timestamp], observation_protocols=[protocol],
                observation_ports=[destination_port], observation_is_new=[is_new], is_new=is_new,
                latest_label=label)
        self._cleanup_fingerprints()
        candidates = self._find_candidate_chains(src=src, dst=dst, triggering_event_id=event_id)
        if not candidates:
            return None
        candidates.sort(key=lambda chain: (chain['graph_score'], chain['hop_count']),
reverse=True)
        best_chain = candidates[0]
        fingerprint = best_chain['fingerprint']
        if self._is_duplicate_path(fingerprint):
            return None
        self.recent_path_fingerprints[fingerprint] = datetime.now(timezone.utc)
        self.detected_paths.append(best_chain)
        print(f"[Graph] LATERAL MOVEMENT: {best_chain['path_str']} | Severity:
{best_chain['severity']} |
            Score: {best_chain['graph_score']:.2f}")
        return best_chain
    def _find_candidate_chains(self, src: str, dst: str, triggering_event_id: str) -> List[Dict]:
        candidates: List[Dict] = []
        possible_starts: Set[str] = {src}
        try:
            ancestors = nx.ancestors(self.live_graph, src)
            possible_starts.update(ancestors)
        except nx.NetworkXError:
            pass
        possible_targets: Set[str] = {dst}
        try:
            descendants = nx.descendants(self.live_graph, dst)
            possible_targets.update(descendants)
        except nx.NetworkXError:
            pass
        for start in possible_starts:
            for target in possible_targets:
                if start == target or start not in self.live_graph or target not in self.live_graph:
                    continue
                try:
```

```
                paths = nx.all_simple_paths(self.live_graph, source=start, target=target,
                cutoff=self.max_path_hops)
                for path in paths:
                    hop_count = len(path) - 1
                    if hop_count < 2:
                        continue
                    chain = self._evaluate_path(path=path, triggering_event_id=triggering_event_id)
                    if chain:
                        candidates.append(chain)
            except (nx.NetworkXNoPath, nx.NodeNotFound):
                continue
        unique: Dict[str, Dict] = {}
        for chain in candidates:
            unique[chain['fingerprint']] = chain
        return list(unique.values())
    def _evaluate_path(self, path: List[str], triggering_event_id: str) -> Optional[Dict]:
        if len(path) < 3:
            return None
        edge_details: List[Dict] = []
        selected_event_ids: List[str] = []
        selected_times: List[datetime] = []
        previous_time: Optional[datetime] = None
        for index in range(len(path) - 1):
            source = path[index]
            target = path[index + 1]
            edge = self.live_graph[source][target]
            event_ids = list(edge.get('event_ids', []))
            timestamps = list(edge.get('timestamps', []))
            observation_protocols = list(edge.get('observation_protocols', []))
            observation_ports = list(edge.get('observation_ports', []))
            observation_is_new = list(edge.get('observation_is_new', []))
            while len(observation_protocols) < len(event_ids):
                observation_protocols.append('')
            while len(observation_ports) < len(event_ids):
                observation_ports.append(0)
            while len(observation_is_new) < len(event_ids):
                observation_is_new.append(bool(edge.get('is_new', False)))
            observations: List[Tuple[datetime, str, str, int, bool]] = []
            for timestamp, event_id, event_protocol, event_port, event_is_new in zip(timestamps,
                event_ids, observation_protocols, observation_ports, observation_is_new):
                try:
                    parsed = self._parse_timestamp(timestamp)
                except (ValueError, TypeError, OverflowError):
                    continue
                observations.append((parsed, str(event_id), str(event_protocol or '').strip().lower(),
                    self._safe_int(event_port, 0), bool(event_is_new)))
            observations.sort(key=lambda item: item[0])
            chosen = None
            for observation_time, observation_id, observation_protocol, observation_port,
                observation_is_new in observations:
                if previous_time is None or observation_time >= previous_time:
                    chosen = (observation_time, observation_id, observation_protocol,
observation_port,
                    observation_is_new)
```

```
                break
            if chosen is None:
                return None
            chosen_time, chosen_id, chosen_protocol, chosen_port, chosen_is_new = chosen
            selected_times.append(chosen_time)
            selected_event_ids.append(chosen_id)
            previous_time = chosen_time
            edge_details.append({'source': source, 'target': target, 'event_id': chosen_id,
'timestamp':
                chosen_time.isoformat(), 'is_new': bool(chosen_is_new), 'protocol': chosen_protocol,
                'destination_port': chosen_port, 'protocols': [chosen_protocol] if chosen_protocol else
                [], 'ports': [chosen_port] if chosen_port else [], 'count': int(edge.get('count', 1))})
        if triggering_event_id not in selected_event_ids:
            return None
        first_time = selected_times[0]
        last_time = selected_times[-1]
        duration_seconds = (last_time - first_time).total_seconds()
        if duration_seconds < 0:
            return None
        if duration_seconds > self.window_minutes * 60:
            return None
        new_edges = [detail for detail in edge_details if detail['is_new']]
        new_paths = [f"{detail['source']} → {detail['target']}" for detail in new_edges]
        lateral_service_edges = 0
        for detail in edge_details:
            if any((port in self.LATERAL_PORTS for port in detail['ports'])):
                lateral_service_edges += 1
                continue
            protocols = {protocol.lower() for protocol in detail['protocols']}
            if protocols.intersection({'smb', 'rdp', 'winrm', 'ssh', 'rpc', 'vnc'}):
                lateral_service_edges += 1
        device_types = [self.device_registry.get(device, 'unknown') for device in path]
        graph_score = self._compute_graph_score(path=path, edge_details=edge_details,
            lateral_service_edges=lateral_service_edges)
        severity = self._score_to_severity(graph_score)
        fingerprint = self._create_path_fingerprint(path=path, event_ids=selected_event_ids)
        return {'alert_type': 'LATERAL_MOVEMENT_CHAIN', 'path': path.copy(), 'path_str': '
→ '.join(path),
            'hop_count': len(path) - 1, 'new_paths': new_paths, 'new_edge_count': len(new_edges),
            'lateral_service_edges': lateral_service_edges, 'severity': severity, 'device_types':
            device_types, 'edge_details': edge_details, 'matched_event_ids': selected_event_ids,
            'sequence_start': first_time.isoformat(), 'sequence_end': last_time.isoformat(),
            'sequence_duration_seconds': round(duration_seconds, 3), 'window_minutes':
            self.window_minutes, 'detected_at': datetime.now(timezone.utc).isoformat(),
'graph_score':
            graph_score, 'fingerprint': fingerprint}
    def _compute_graph_score(self, path: List[str], edge_details: List[Dict],
lateral_service_edges: int)
        -> float:
        score = 0.4
        new_edge_count = sum((1 for detail in edge_details if detail.get('is_new', False)))
        score += min(new_edge_count * 0.15, 0.3)
        score += min(lateral_service_edges * 0.07, 0.14)
        hop_count = len(path) - 1
```

```
        if hop_count >= 3:
            score += 0.05
        if hop_count >= 4:
            score += 0.03
        critical_count = 0
        for device in path:
            device_type = str(self.device_registry.get(device, '')).strip().lower()
            if device_type in self.CRITICAL_DEVICE_TYPES:
                critical_count += 1
        score += min(critical_count * 0.04, 0.12)
        return round(min(score, 1.0), 4)
    @staticmethod
    def _score_to_severity(score: float) -> str:
        if score >= 0.85:
            return 'CRITICAL'
        if score >= 0.7:
            return 'HIGH'
        if score >= 0.5:
            return 'MEDIUM'
        return 'LOW'
    @staticmethod
    def _create_path_fingerprint(path: List[str], event_ids: List[str]) -> str:
        data = json.dumps({'path': path, 'event_ids': event_ids}, sort_keys=True)
        return hashlib.sha256(data.encode('utf-8')).hexdigest()
    def _is_duplicate_path(self, fingerprint: str) -> bool:
        detected_at = self.recent_path_fingerprints.get(fingerprint)
        if detected_at is None:
            return False
        cooldown = timedelta(minutes=self.duplicate_cooldown_minutes)
        return datetime.now(timezone.utc) - detected_at <= cooldown
    def _cleanup_fingerprints(self) -> None:
        now = datetime.now(timezone.utc)
        expiry = timedelta(minutes=max(self.window_minutes, self.duplicate_cooldown_minutes) *
2)
        expired = [fingerprint for fingerprint, detected_at in self.recent_path_fingerprints.items() if
            now - detected_at > expiry]
        for fingerprint in expired:
            self.recent_path_fingerprints.pop(fingerprint, None)
    def get_device_risk_scores(self) -> Dict[str, float]:
        scores: Dict[str, float] = {}
        for node in self.live_graph.nodes():
            out_edges = list(self.live_graph.out_edges(node, data=True))
            in_edges = list(self.live_graph.in_edges(node, data=True))
            all_edges = out_edges + in_edges
            if not all_edges:
                scores[node] = 0.0
                continue
            new_count = sum((1 for _, _, data in all_edges if data.get('is_new', False)))
            lateral_count = 0
            for _, _, data in all_edges:
                ports = data.get('ports', set())
                if any((port in self.LATERAL_PORTS for port in ports)):
                    lateral_count += 1
            total = len(all_edges)
```

```
            new_ratio = new_count / total
            lateral_ratio = lateral_count / total
            score = new_ratio * 0.65 + lateral_ratio * 0.35
            device_type = str(self.device_registry.get(node, '')).lower()
            if device_type in self.CRITICAL_DEVICE_TYPES:
                score += 0.1
            scores[node] = round(min(score, 1.0), 3)
        return dict(sorted(scores.items(), key=lambda item: item[1], reverse=True))
    def get_graph_summary(self) -> Dict:
        return {'total_devices': self.live_graph.number_of_nodes(), 'total_connections':
            self.live_graph.number_of_edges(), 'baseline_devices':
self.baseline_graph.number_of_nodes(),
            'baseline_connections': self.baseline_graph.number_of_edges(), 'detected_chains':
            len(self.detected_paths), 'new_connections': sum((1 for _, _, data in
            self.live_graph.edges(data=True) if data.get('is_new', False))), 'total_events_seen':
            self.event_count, 'window_minutes': self.window_minutes, 'max_path_hops':
self.max_path_hops}
    def get_recent_paths(self, last_n: int=10) -> List[Dict]:
        if last_n <= 0:
            return []
        return self.detected_paths[-last_n:]
    def get_cytoscape_elements(self) -> List[Dict]:
        elements: List[Dict] = []
        risk_scores = self.get_device_risk_scores()
        all_nodes = set(self.device_registry.keys())
        all_nodes.update(self.baseline_graph.nodes())
        all_nodes.update(self.live_graph.nodes())
        for node in sorted(all_nodes):
            elements.append({'data': {'id': node, 'label': node, 'device_type':
                self.device_registry.get(node, 'unknown'), 'risk_score': risk_scores.get(node, 0.0),
                'in_baseline': node in self.baseline_graph, 'in_live_graph': node in self.live_graph}})
        baseline_pairs = set(self.baseline_graph.edges())
        live_pairs = set(self.live_graph.edges())
        all_pairs = baseline_pairs | live_pairs
        for source, target in sorted(all_pairs):
            baseline_data = self.baseline_graph.get_edge_data(source, target, default={}) or {}
            live_data = self.live_graph.get_edge_data(source, target, default={}) or {}
            baseline_protocols = sorted((str(item) for item in baseline_data.get('protocols', set()) if
                str(item)))
            baseline_ports = sorted((self._safe_int(item, 0) for item in baseline_data.get('ports', set())
                 if self._safe_int(item, 0)))
            live_protocols = sorted((str(item) for item in live_data.get('protocols', set()) if
                str(item)))
            live_ports = sorted((self._safe_int(item, 0) for item in live_data.get('ports', set()) if
                self._safe_int(item, 0)))
            is_baseline = bool(baseline_data)
            is_live = bool(live_data)
            is_new = bool(live_data.get('is_new', False))
            display_protocols = live_protocols if live_protocols else baseline_protocols
            display_ports = live_ports if live_ports else baseline_ports
            if is_live and is_new:
                edge_kind = 'new_live'
            elif is_live:
                edge_kind = 'known_live'
```

```
            else:
                edge_kind = 'baseline'
            source_data = live_data if is_live else baseline_data
            baseline_services = []
            for service_protocol, service_port in baseline_data.get('services', set()):
                baseline_services.append({'protocol': str(service_protocol or ''), 'destination_port':
                    self._safe_int(service_port, 0)})
            baseline_services.sort(key=lambda item: (item['protocol'], item['destination_port']))
            elements.append({'data': {'id': f'edge::{source}::{target}', 'source': source, 'target':
                target, 'edge_kind': edge_kind, 'is_baseline': is_baseline, 'is_live': is_live, 'is_new':
                is_new, 'protocol': ', '.join(display_protocols), 'protocols': display_protocols, 'ports':
                 display_ports, 'baseline_protocols': baseline_protocols, 'baseline_ports':
                baseline_ports, 'baseline_services': baseline_services, 'live_protocols': live_protocols,
                'live_ports': live_ports, 'count': int(source_data.get('count', 1)), 'first_seen':
                source_data.get('first_seen', ''), 'last_seen': source_data.get('last_seen', ''),
                'latest_label': live_data.get('latest_label', 'normal' if is_baseline else 'unknown')}})
        return elements
```

A.13 Temporal Pattern Correlation

This listing shows entity grouping, timestamp handling, bounded event retention, sequence matching, duplicate suppression and temporal scoring. The key evaluation questions are whether unrelated entities remain separate and whether delivery order changes the result. The code is reproduced for inspection, not as evidence that those tests passed.

Source: src/engine/temporal_correlator.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
from __future__ import annotations
from collections import defaultdict, deque
from datetime import datetime, timedelta, timezone
from typing import Deque, Dict, List, Optional, Tuple, Any
import hashlib
import json
class TemporalCorrelator:
    ATTACK_PATTERNS = {'pass_the_hash': {'sequence': ['AUTH_FAILURE',
'AUTH_SUCCESS',
        'NETWORK_CONNECTION'], 'description': 'Failed authentication followed by successful
authentication
         and a network connection.', 'mitre': 'T1550.002', 'severity': 'CRITICAL'},
        'fileless_lateral_move': {'sequence': ['CODE_INJECTION',
'NETWORK_CONNECTION'], 'description':
        'Code injection followed by network lateral movement.', 'mitre': 'T1055', 'severity':
'CRITICAL'},
         'credential_then_lateral': {'sequence': ['CREDENTIAL_MEMORY_ACCESS',
'EXPLICIT_CRED_USE',
        'NETWORK_CONNECTION'], 'description': 'Credential memory access followed by
explicit credential
        use and network movement.', 'mitre': 'T1003.001', 'severity': 'CRITICAL'},
        'process_inject_lateral': {'sequence': ['PROCESS_CREATION', 'CODE_INJECTION',
        'NETWORK_CONNECTION'], 'description': 'Process creation followed by code injection
and network
        movement.', 'mitre': 'T1055.003', 'severity': 'HIGH'}, 'lolbin_lateral': {'sequence':
        ['PROCESS_CREATION', 'NETWORK_CONNECTION'], 'description': 'Process
creation followed by a network
         connection. This pattern should be interpreted with process context such as LOLBin
usage.',
        'mitre': 'T1218', 'severity': 'HIGH'}, 'recon_then_exploit': {'sequence':
['AUTH_FAILURE',
        'AUTH_FAILURE', 'AUTH_SUCCESS'], 'description': 'Repeated authentication failures
followed by a
        successful authentication.', 'mitre': 'T1110', 'severity': 'HIGH'}, 'hollow_then_move':
        {'sequence': ['PROCESS_HOLLOWING', 'NETWORK_CONNECTION'], 'description':
'Process hollowing
        followed by network lateral movement.', 'mitre': 'T1055.012', 'severity': 'CRITICAL'}}
    SEVERITY_RANK = {'CRITICAL': 4, 'HIGH': 3, 'MEDIUM': 2, 'LOW': 1, 'NONE': 0}
    def __init__(self, window_minutes: int=10, max_events_per_entity: int=1000,
        duplicate_cooldown_minutes: int=5):
```

```
        if window_minutes <= 0:
            raise ValueError('window_minutes must be greater than zero')
        if max_events_per_entity <= 0:
            raise ValueError('max_events_per_entity must be greater than zero')
        self.window_minutes = int(window_minutes)
        self.max_events_per_entity = int(max_events_per_entity)
        self.duplicate_cooldown_minutes = int(duplicate_cooldown_minutes)
        self.entity_buffers: Dict[str, Deque[Dict]] = defaultdict(lambda:
            deque(maxlen=self.max_events_per_entity))
        self.matched_patterns: List[Dict] = []
        self.event_count = 0
        self.recent_match_fingerprints: Dict[str, datetime] = {}
    def add_event(self, event: Dict) -> Optional[Dict]:
        if not isinstance(event, dict):
            raise TypeError('event must be a dictionary')
        self.event_count += 1
        normalized_event = dict(event)
        if not normalized_event.get('event_id'):
            normalized_event['event_id'] = f'temporal-event-{self.event_count}'
        normalized_event['timestamp'] =
self._normalize_timestamp(normalized_event.get('timestamp'))
        normalized_event['indicator'] = self._get_primary_indicator(normalized_event)
        if not normalized_event['indicator']:
            return None
        entity_key = self._build_entity_key(normalized_event)
        normalized_event['_temporal_entity_key'] = entity_key
        buffer = self.entity_buffers[entity_key]
        buffer.append(normalized_event)
        self._flush_old_events(entity_key)
        self._cleanup_fingerprints()
        return self._match_patterns(entity_key, triggering_event_id=normalized_event['event_id'])
    def _build_entity_key(self, event: Dict) -> str:
        incident_id = str(event.get('incident_id', '') or '').strip()
        if incident_id:
            return f'incident:{incident_id}'
        session_id = str(event.get('session_id', '') or '').strip()
        if session_id:
            return f'session:{session_id}'
        source_device = str(event.get('source_device', '') or '').strip().lower()
        source_ip = str(event.get('source_ip', '') or '').strip().lower()
        username = str(event.get('username', '') or '').strip().lower()
        if source_device and username:
            return f'device:{source_device}|user:{username}'
        if source_device:
            return f'device:{source_device}'
        if source_ip:
            return f'ip:{source_ip}'
        return 'unknown'
    @staticmethod
    def _parse_timestamp(value: Any) -> datetime:
        if isinstance(value, datetime):
            parsed = value
        else:
            text = str(value or '').strip()
```

```
        if not text:
            return datetime.now(timezone.utc)
        parsed = datetime.fromisoformat(text.replace('Z', '+00:00'))
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=timezone.utc)
    return parsed.astimezone(timezone.utc)
@classmethod
def _normalize_timestamp(cls, value: Any) -> str:
    try:
        parsed = cls._parse_timestamp(value)
    except (ValueError, TypeError, OverflowError):
        parsed = datetime.now(timezone.utc)
    return parsed.isoformat()
def _flush_old_events(self, entity_key: str) -> None:
    buffer = self.entity_buffers.get(entity_key)
    if not buffer:
        return
    now = datetime.now(timezone.utc)
    cutoff = now - timedelta(minutes=self.window_minutes)
    retained: List[Dict] = []
    for event in buffer:
        try:
            event_time = self._parse_timestamp(event.get('timestamp'))
        except (ValueError, TypeError, OverflowError):
            continue
        if event_time >= cutoff:
            retained.append(event)
    new_buffer = deque(retained, maxlen=self.max_events_per_entity)
    self.entity_buffers[entity_key] = new_buffer
@staticmethod
def _get_primary_indicator(event: Dict) -> str:
    direct_indicator = str(event.get('indicator', '') or '').strip()
    if direct_indicator:
        return direct_indicator.upper()
    event_type = str(event.get('event_type', '') or '').strip()
    if event_type:
        return event_type.upper()
    memory_indicators = event.get('memory_indicators', [])
    if isinstance(memory_indicators, list):
        for indicator in memory_indicators:
            text = str(indicator).strip()
            if text:
                return text.upper()
    return ''
def _match_patterns(self, entity_key: str, triggering_event_id: str) -> Optional[Dict]:
    buffer = list(self.entity_buffers.get(entity_key, []))
    if not buffer:
        return None
    candidates: List[Dict] = []
    for pattern_name, pattern_info in self.ATTACK_PATTERNS.items():
        sequence = pattern_info['sequence']
        matched_events = self._find_sequence_events(sequence, buffer)
        if not matched_events:
            continue
```

```
            matched_event_ids = [str(event.get('event_id', '')) for event in matched_events]
            if triggering_event_id not in matched_event_ids:
                continue
            first_time = self._parse_timestamp(matched_events[0]['timestamp'])
            last_time = self._parse_timestamp(matched_events[-1]['timestamp'])
            duration_seconds = (last_time - first_time).total_seconds()
            if duration_seconds < 0:
                continue
            if duration_seconds > self.window_minutes * 60:
                continue
            devices = sorted({str(event.get('source_device', '')).strip() for event in matched_events if
                str(event.get('source_device', '')).strip()})
            destination_devices = sorted({str(event.get('destination_device', '')).strip() for event in
                matched_events if str(event.get('destination_device', '')).strip()})
            fingerprint = self._create_match_fingerprint(entity_key=entity_key,
pattern_name=pattern_name,
                 event_ids=matched_event_ids)
            if self._is_duplicate_match(fingerprint):
                continue
            temporal_score = self._compute_score(severity=pattern_info['severity'],
                matched_events=matched_events)
            candidate = {'alert_type': 'TEMPORAL_PATTERN_MATCH', 'pattern_name':
pattern_name,
                'description': pattern_info['description'], 'sequence': sequence.copy(), 'mitre':
                pattern_info['mitre'], 'severity': pattern_info['severity'], 'entity_key': entity_key,
                'devices': devices, 'destination_devices': destination_devices, 'events_in_window':
                len(buffer), 'matched_event_count': len(matched_events), 'matched_event_ids':
                matched_event_ids, 'matched_events': [self._summarize_event(event) for event in
                matched_events], 'window_minutes': self.window_minutes,
'sequence_duration_seconds':
                round(duration_seconds, 3), 'detected_at': datetime.now(timezone.utc).isoformat(),
                'temporal_score': temporal_score, 'fingerprint': fingerprint}
            candidates.append(candidate)
        if not candidates:
            return None
        candidates.sort(key=lambda item: (self.SEVERITY_RANK.get(item['severity'], 0),
            item['temporal_score']), reverse=True)
        best_match = candidates[0]
        self.recent_match_fingerprints[best_match['fingerprint']] = datetime.now(timezone.utc)
        self.matched_patterns.append(best_match)
        print(f"[Temporal] PATTERN MATCH: {best_match['pattern_name']} | Entity:
            {best_match['entity_key']} | Severity: {best_match['severity']} | Score:
            {best_match['temporal_score']:.2f}")
        return best_match
    def _find_sequence_events(self, sequence: List[str], buffer: List[Dict]) -> Optional[List[Dict]]:
        if not sequence:
            return None
        matched: List[Dict] = []
        sequence_index = 0
        previous_time: Optional[datetime] = None
        sortable_events: List[Tuple[datetime, Dict]] = []
        for event in buffer:
            try:
                event_time = self._parse_timestamp(event.get('timestamp'))
```

```
            except (ValueError, TypeError, OverflowError):
                continue
            sortable_events.append((event_time, event))
        sortable_events.sort(key=lambda item: item[0])
        for event_time, event in sortable_events:
            if sequence_index >= len(sequence):
                break
            indicator = self._get_primary_indicator(event)
            expected = str(sequence[sequence_index]).upper()
            if indicator != expected:
                continue
            if previous_time is not None and event_time < previous_time:
                continue
            matched.append(event)
            previous_time = event_time
            sequence_index += 1
        if sequence_index == len(sequence):
            return matched
        return None
    @staticmethod
    def _create_match_fingerprint(entity_key: str, pattern_name: str, event_ids: List[str]) -> str:
        fingerprint_source = json.dumps({'entity_key': entity_key, 'pattern_name': pattern_name,
            'event_ids': event_ids}, sort_keys=True)
        return hashlib.sha256(fingerprint_source.encode('utf-8')).hexdigest()
    def _is_duplicate_match(self, fingerprint: str) -> bool:
        previous_time = self.recent_match_fingerprints.get(fingerprint)
        if previous_time is None:
            return False
        now = datetime.now(timezone.utc)
        cooldown = timedelta(minutes=self.duplicate_cooldown_minutes)
        return now - previous_time <= cooldown
    def _cleanup_fingerprints(self) -> None:
        if not self.recent_match_fingerprints:
            return
        now = datetime.now(timezone.utc)
        expiry = timedelta(minutes=max(self.window_minutes, self.duplicate_cooldown_minutes) *
2)
        expired = [fingerprint for fingerprint, detected_at in self.recent_match_fingerprints.items()
if
            now - detected_at > expiry]
        for fingerprint in expired:
            self.recent_match_fingerprints.pop(fingerprint, None)
    def _compute_score(self, severity: str, matched_events: List[Dict]) -> float:
        base = {'CRITICAL': 0.85, 'HIGH': 0.7, 'MEDIUM': 0.5, 'LOW': 0.3}.get(severity, 0.4)
        layer_count = len({str(event.get('layer', 'unknown')).lower() for event in matched_events})
        event_bonus = min(max(len(matched_events) - 1, 0) * 0.025, 0.05)
        layer_bonus = min(max(layer_count - 1, 0) * 0.04, 0.08)
        has_remote_destination = any((bool(str(event.get('destination_device', '')).strip() or
            str(event.get('destination_ip', '')).strip()) for event in matched_events))
        destination_bonus = 0.03 if has_remote_destination else 0.0
        score = base + event_bonus + layer_bonus + destination_bonus
        return round(min(score, 1.0), 4)
    @staticmethod
    def _summarize_event(event: Dict) -> Dict:
```

```
        return {'event_id': event.get('event_id'), 'timestamp': event.get('timestamp'), 'indicator':
            event.get('indicator'), 'event_type': event.get('event_type'), 'layer': event.get('layer'),
            'source_device': event.get('source_device'), 'destination_device':
            event.get('destination_device'), 'username': event.get('username')}
    def get_summary(self) -> Dict:
        active_buffers = {entity_key: len(buffer) for entity_key, buffer in self.entity_buffers.items()
if
             buffer}
        return {'active_entities': len(active_buffers), 'events_in_window':
sum(active_buffers.values()),
            'events_per_entity': active_buffers, 'total_events_seen': self.event_count,
            'patterns_matched': len(self.matched_patterns), 'window_minutes':
self.window_minutes,
            'duplicate_cooldown_minutes': self.duplicate_cooldown_minutes, 'pattern_names':
            [pattern['pattern_name'] for pattern in self.matched_patterns]}
    def get_recent_matches(self, last_n: int=10) -> List[Dict]:
        if last_n <= 0:
            return []
        return self.matched_patterns[-last_n:]
    def get_entity_events(self, entity_key: str) -> List[Dict]:
        return list(self.entity_buffers.get(entity_key, []))
```

A.14 Memory–network Correlation

This listing shows association in both arrival directions, entity matching, novelty context, scoring and duplicate handling. Its evidence granularity should be examined before describing an association as process-level causation. The proposed tests in Table A.10 cover the main boundary conditions.

Source: src/engine/memory_correlator.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
from __future__ import annotations
from collections import deque
from datetime import datetime, timedelta, timezone
from typing import Any, Deque, Dict, List, Optional, Set
import hashlib
import json
import threading
class MemoryCorrelator:
    STRONG_MEMORY_INDICATORS = {'CODE_INJECTION',
'PROCESS_HOLLOWING', 'CREDENTIAL_MEMORY_ACCESS',
        'EXECUTABLE_MEMORY_REGION', 'SUSPICIOUS_PROCESS_CHAIN'}
    SUPPORTING_INDICATORS = {'LOLBIN_USAGE', 'NEW_PROCESS',
'FILELESS_NETWORK', 'LATERAL_PORT_CONNECTION'}
    LATERAL_PORTS = {445, 3389, 5985, 5986, 22, 135, 139, 5900, 4444}
    LATERAL_PROTOCOLS = {'smb', 'smb2', 'smb3', 'rdp', 'winrm', 'ssh', 'rpc', 'dcom', 'vnc'}
    def __init__(self, correlation_window_seconds: int=300, max_memory_events: int=2000,
        max_network_events: int=5000, duplicate_cooldown_seconds: int=300):
        if correlation_window_seconds <= 0:
            raise ValueError('correlation_window_seconds must be greater than zero')
        self.correlation_window_seconds = int(correlation_window_seconds)
        self.duplicate_cooldown_seconds = int(duplicate_cooldown_seconds)
        self.memory_event_buffer: Deque[Dict] = deque(maxlen=max_memory_events)
        self.network_event_buffer: Deque[Dict] = deque(maxlen=max_network_events)
        self.correlated_alerts: List[Dict] = []
        self.recent_fingerprints: Dict[str, datetime] = {}
        self._lock = threading.RLock()
        self.event_count = 0
    @staticmethod
    def _parse_timestamp(value: Any) -> datetime:
        if isinstance(value, datetime):
            parsed = value
        else:
            text = str(value or '').strip()
            if not text:
                return datetime.now(timezone.utc)
            parsed = datetime.fromisoformat(text.replace('Z', '+00:00'))
        if parsed.tzinfo is None:
            parsed = parsed.replace(tzinfo=timezone.utc)
        return parsed.astimezone(timezone.utc)
    @classmethod
    def _normalize_timestamp(cls, value: Any) -> str:
        try:
```

```
            return cls._parse_timestamp(value).isoformat()
        except (TypeError, ValueError, OverflowError):
            return datetime.now(timezone.utc).isoformat()
    @staticmethod
    def _safe_int(value: Any, default: int=0) -> int:
        try:
            return int(float(value if value is not None else default))
        except (TypeError, ValueError, OverflowError):
            return default
    @staticmethod
    def _normalize_device(value: Any) -> str:
        return str(value or '').strip().lower()
    @staticmethod
    def _normalize_indicator(value: Any) -> str:
        return str(value or '').strip().upper()
    @staticmethod
    def _get_network_novelty(event: Dict) -> Optional[bool]:
        if not isinstance(event, dict):
            return None
        for key in ['service_is_new', 'graph_is_new', 'baseline_is_new', 'is_new']:
            if key not in event:
                continue
            value = event.get(key)
            if value is None:
                continue
            if isinstance(value, bool):
                return value
            if isinstance(value, (int, float)):
                return bool(value)
            text = str(value).strip().lower()
            if text in {'true', '1', 'yes', 'new', 'unusual'}:
                return True
            if text in {'false', '0', 'no', 'known', 'baseline', 'normal'}:
                return False
        return None
    def _extract_indicators(self, event: Dict) -> List[str]:
        indicators: List[str] = []
        candidate_lists = [event.get('memory_indicators'), event.get('indicators')]
        for candidate in candidate_lists:
            if isinstance(candidate, (list, tuple, set)):
                for value in candidate:
                    indicator = self._normalize_indicator(value)
                    if indicator and indicator not in indicators:
                        indicators.append(indicator)
        for key in ['indicator', 'event_type']:
            indicator = self._normalize_indicator(event.get(key))
            if indicator and indicator not in indicators:
                indicators.append(indicator)
        return indicators
    def _get_event_entities(self, event: Dict) -> Set[str]:
        entities: Set[str] = set()
        for key in ['source_device', 'destination_device', 'source_ip', 'destination_ip']:
            value = self._normalize_device(event.get(key))
            if value:
```

```
            entities.add(value)
    return entities
def _events_share_entity(self, first: Dict, second: Dict) -> bool:
    first_entities = self._get_event_entities(first)
    second_entities = self._get_event_entities(second)
    if not first_entities or not second_entities:
        return False
    return bool(first_entities.intersection(second_entities))
def ingest_network_event(self, event: Dict) -> Optional[Dict]:
    if not isinstance(event, dict):
        raise TypeError('event must be a dictionary')
    normalized = dict(event)
    normalized['timestamp'] = self._normalize_timestamp(normalized.get('timestamp'))
    normalized['_memory_corr_type'] = 'network'
    with self._lock:
        self.network_event_buffer.append(normalized)
        self._flush_old_events()
    return self._correlate_network_with_memory(normalized)
def ingest_memory_event(self, event: Dict) -> Optional[Dict]:
    if not isinstance(event, dict):
        raise TypeError('event must be a dictionary')
    normalized = dict(event)
    normalized['timestamp'] = self._normalize_timestamp(normalized.get('timestamp'))
    normalized['_memory_corr_type'] = 'memory'
    indicators = self._extract_indicators(normalized)
    normalized['_memory_corr_indicators'] = indicators
    with self._lock:
        self.memory_event_buffer.append(normalized)
        self._flush_old_events()
    return self._correlate_with_network(normalized)
def process_event(self, event: Dict) -> Optional[Dict]:
    layer = str(event.get('layer', 'unknown')).strip().lower()
    if layer == 'network':
        return self.ingest_network_event(event)
    if layer in {'memory', 'host'}:
        indicators = self._extract_indicators(event)
        if not indicators:
            return None
        return self.ingest_memory_event(event)
    return None
def _flush_old_events(self) -> None:
    cutoff = datetime.now(timezone.utc) - timedelta(seconds=self.correlation_window_seconds)
    self.memory_event_buffer = deque([event for event in self.memory_event_buffer if
        self._event_is_recent(event, cutoff)], maxlen=self.memory_event_buffer.maxlen)
    self.network_event_buffer = deque([event for event in self.network_event_buffer if
        self._event_is_recent(event, cutoff)], maxlen=self.network_event_buffer.maxlen)
def _event_is_recent(self, event: Dict, cutoff: datetime) -> bool:
    try:
        timestamp = self._parse_timestamp(event.get('timestamp'))
    except (TypeError, ValueError, OverflowError):
        return False
    return timestamp >= cutoff
def _correlate_with_network(self, memory_event: Dict) -> Optional[Dict]:
    indicators = self._extract_indicators(memory_event)
```

```
        if not indicators:
            return None
        strong_indicators = set(indicators).intersection(self.STRONG_MEMORY_INDICATORS)
        if not strong_indicators:
            return None
        memory_time = self._parse_timestamp(memory_event.get('timestamp'))
        with self._lock:
            candidates = list(self.network_event_buffer)
        related_network: List[Dict] = []
        for network_event in candidates:
            if not self._events_share_entity(memory_event, network_event):
                continue
            network_time = self._parse_timestamp(network_event.get('timestamp'))
            difference = abs((network_time - memory_time).total_seconds())
            if difference > self.correlation_window_seconds:
                continue
            if not self._is_lateral_network_event(network_event):
                continue
            related_network.append(network_event)
        if not related_network:
            return None
        return self._create_correlation_alert(memory_event=memory_event,
network_events=related_network)
    def _correlate_network_with_memory(self, network_event: Dict) -> Optional[Dict]:
        if not self._is_lateral_network_event(network_event):
            return None
        network_time = self._parse_timestamp(network_event.get('timestamp'))
        with self._lock:
            candidates = list(self.memory_event_buffer)
        possible_alerts: List[Dict] = []
        for memory_event in candidates:
            if not self._events_share_entity(memory_event, network_event):
                continue
            indicators = self._extract_indicators(memory_event)
            if not set(indicators).intersection(self.STRONG_MEMORY_INDICATORS):
                continue
            memory_time = self._parse_timestamp(memory_event.get('timestamp'))
            difference = abs((network_time - memory_time).total_seconds())
            if difference > self.correlation_window_seconds:
                continue
            alert = self._create_correlation_alert(memory_event=memory_event,
                network_events=[network_event])
            if alert:
                possible_alerts.append(alert)
        if not possible_alerts:
            return None
        possible_alerts.sort(key=lambda alert: alert['correlation_score'], reverse=True)
        return possible_alerts[0]
    def _is_lateral_network_event(self, event: Dict) -> bool:
        destination_port = self._safe_int(event.get('destination_port', event.get('dst_port', 0)))
        protocol = str(event.get('protocol', '')).strip().lower()
        is_lateral_port = destination_port in self.LATERAL_PORTS
        is_lateral_protocol = protocol in self.LATERAL_PROTOCOLS
        novelty = self._get_network_novelty(event)
```

```
        is_new = novelty is True
        label = str(event.get('label', '')).strip().lower()
        suspicious_label = label in {'malicious', 'suspicious', 'attack', '1'}
        return bool(is_lateral_port or is_lateral_protocol or (is_new and suspicious_label))
    def _create_correlation_alert(self, memory_event: Dict, network_events: List[Dict]) ->
Optional[Dict]:
        indicators = self._extract_indicators(memory_event)
        strong_indicators =
sorted(set(indicators).intersection(self.STRONG_MEMORY_INDICATORS))
        if not strong_indicators:
            return None
        related_network = self._deduplicate_network_events(network_events)
        if not related_network:
            return None
        fingerprint = self._create_fingerprint(memory_event, related_network)
        self._cleanup_fingerprints()
        if self._is_duplicate(fingerprint):
            return None
        score = self._compute_correlation_score(memory_event=memory_event,
network_events=related_network)
        severity = self._score_to_severity(score)
        source_device = str(memory_event.get('source_device', memory_event.get('source_ip',
'unknown')))
        network_event_ids = [str(event.get('event_id', '')) for event in related_network if
            event.get('event_id')]
        memory_event_id = str(memory_event.get('event_id', ''))
        matched_event_ids = []
        if memory_event_id:
            matched_event_ids.append(memory_event_id)
        matched_event_ids.extend(network_event_ids)
        alert = {'alert_type': 'CROSS_LAYER_CORRELATION', 'correlation_type':
'MEMORY_NETWORK',
            'source_device': source_device, 'memory_event':
self._summarize_memory_event(memory_event),
            'network_events': [self._summarize_network_event(event) for event in related_network],
            'indicators': indicators, 'strong_indicators': strong_indicators, 'matched_event_ids':
            matched_event_ids, 'network_event_count': len(related_network), 'severity': severity,
            'description': self._build_description(source_device=source_device,
            strong_indicators=strong_indicators, network_events=related_network), 'detected_at':
            datetime.now(timezone.utc).isoformat(), 'correlation_score': score, 'memory_score':
score,
            'fingerprint': fingerprint}
        self.recent_fingerprints[fingerprint] = datetime.now(timezone.utc)
        with self._lock:
            self.correlated_alerts.append(alert)
        print(f"[Memory] CROSS-LAYER CORRELATION: {source_device} | {',
'.join(strong_indicators)} |
            Network events: {len(related_network)} | Score: {score:.2f}")
        return alert
    def _compute_correlation_score(self, memory_event: Dict, network_events: List[Dict]) ->
float:
        indicators = set(self._extract_indicators(memory_event))
        score = 0.45
        if 'CODE_INJECTION' in indicators:
```

```
            score += 0.14
        if 'PROCESS_HOLLOWING' in indicators:
            score += 0.18
        if 'CREDENTIAL_MEMORY_ACCESS' in indicators:
            score += 0.16
        if 'EXECUTABLE_MEMORY_REGION' in indicators:
            score += 0.12
        if 'SUSPICIOUS_PROCESS_CHAIN' in indicators:
            score += 0.08
        lateral_ports = 0
        suspicious_network_events = 0
        new_connections = 0
        for event in network_events:
            destination_port = self._safe_int(event.get('destination_port', event.get('dst_port', 0)))
            if destination_port in self.LATERAL_PORTS:
                lateral_ports += 1
            novelty = self._get_network_novelty(event)
            if novelty is True:
                new_connections += 1
            label = str(event.get('label', '')).strip().lower()
            if label in {'malicious', 'suspicious', 'attack', '1'}:
                suspicious_network_events += 1
        if lateral_ports:
            score += min(lateral_ports * 0.05, 0.1)
        if new_connections:
            score += min(new_connections * 0.04, 0.08)
        if suspicious_network_events:
            score += min(suspicious_network_events * 0.03, 0.06)
        if len(network_events) >= 2:
            score += 0.04
        return round(min(score, 1.0), 4)
    @staticmethod
    def _score_to_severity(score: float) -> str:
        if score >= 0.85:
            return 'CRITICAL'
        if score >= 0.7:
            return 'HIGH'
        if score >= 0.5:
            return 'MEDIUM'
        return 'LOW'
    @staticmethod
    def _create_fingerprint(memory_event: Dict, network_events: List[Dict]) -> str:
        data = {'memory_event_id': str(memory_event.get('event_id', '')), 'network_event_ids':
            sorted((str(event.get('event_id', '')) for event in network_events)), 'indicators':
            sorted((str(indicator) for indicator in memory_event.get('memory_indicators',
            memory_event.get('indicators', [])))))}
        encoded = json.dumps(data, sort_keys=True)
        return hashlib.sha256(encoded.encode('utf-8')).hexdigest()
    def _is_duplicate(self, fingerprint: str) -> bool:
        previous = self.recent_fingerprints.get(fingerprint)
        if previous is None:
            return False
        return (datetime.now(timezone.utc) - previous).total_seconds() <=
self.duplicate_cooldown_seconds
```

```
    def _cleanup_fingerprints(self) -> None:
        now = datetime.now(timezone.utc)
        expiry_seconds = max(self.correlation_window_seconds, self.duplicate_cooldown_seconds)
* 2
        expired = [fingerprint for fingerprint, timestamp in self.recent_fingerprints.items() if (now
-
            timestamp).total_seconds() > expiry_seconds]
        for fingerprint in expired:
            self.recent_fingerprints.pop(fingerprint, None)
    @staticmethod
    def _deduplicate_network_events(events: List[Dict]) -> List[Dict]:
        unique: Dict[str, Dict] = {}
        for index, event in enumerate(events):
            event_id = str(event.get('event_id', ''))
            if event_id:
                key = f'id:{event_id}'
            else:
                key = json.dumps({'src': event.get('source_device'), 'dst':
                    event.get('destination_device'), 'port': event.get('destination_port'), 'timestamp':
                    event.get('timestamp'), 'index': index}, sort_keys=True, default=str)
            unique[key] = event
        return list(unique.values())
    @staticmethod
    def _build_description(source_device: str, strong_indicators: List[str], network_events:
List[Dict])
        -> str:
        indicators = ', '.join(strong_indicators)
        destinations = sorted({str(event.get('destination_device', event.get('destination_ip', ''))) for
            event in network_events if event.get('destination_device') or event.get('destination_ip')})
        destination_text = ', '.join(destinations) if destinations else 'a remote system'
        return f'Strong memory/host evidence ({indicators}) on {source_device} was correlated
with
            lateral-movement-relevant network activity involving {destination_text}.'
    @staticmethod
    def _summarize_memory_event(event: Dict) -> Dict:
        return {'event_id': event.get('event_id'), 'timestamp': event.get('timestamp'),
'source_device':
            event.get('source_device'), 'source_ip': event.get('source_ip'), 'source_process':
            event.get('source_process', event.get('process_name')), 'parent_process':
            event.get('parent_process', event.get('parent_name')), 'indicators':
            event.get('_memory_corr_indicators', event.get('memory_indicators',
event.get('indicators',
            [']))), 'severity': event.get('severity')}
    def _summarize_network_event(self, event: Dict) -> Dict:
        novelty = self._get_network_novelty(event)
        if novelty is True:
            novelty_status = 'new'
        elif novelty is False:
            novelty_status = 'known'
        else:
            novelty_status = 'unknown'
        return {'event_id': event.get('event_id'), 'timestamp': event.get('timestamp'),
'source_device':
            event.get('source_device'), 'destination_device': event.get('destination_device'),
```

```
            'source_ip': event.get('source_ip'), 'destination_ip': event.get('destination_ip'),
            'protocol': event.get('protocol'), 'destination_port': event.get('destination_port',
            event.get('dst_port')), 'is_new': novelty, 'novelty_status': novelty_status, 'label':
            event.get('label')}
    def get_recent_events(self, last_n: int=50) -> List[Dict]:
        if last_n <= 0:
            return []
        with self._lock:
            return list(self.memory_event_buffer)[-last_n:]
    def get_correlated_alerts(self) -> List[Dict]:
        with self._lock:
            return list(self.correlated_alerts)
    def get_summary(self) -> Dict:
        with self._lock:
            return {'memory_events_tracked': len(self.memory_event_buffer),
'network_events_tracked':
                len(self.network_event_buffer), 'correlated_alerts': len(self.correlated_alerts),
                'correlation_window_seconds': self.correlation_window_seconds,
                'duplicate_cooldown_seconds': self.duplicate_cooldown_seconds}
```

A.15 Database Manager and Transaction Boundary

This listing contains the database-manager implementation exercised by the isolated resilience test. It should be read with the schema audit and actual console output. Successful manager-level transactions do not automatically establish atomicity of every higher-level ingestion operation.

Source: src/database/database.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
from __future__ import annotations
import sqlite3
from contextlib import contextmanager
from pathlib import Path
from typing import Generator, List
import sys
PROJECT_ROOT = Path(__file__).resolve().parents[2]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
from src.config.settings import settings
from src.database.schema import ALL_TABLES, CREATE_INDEXES
class DatabaseManager:
    def __init__(self, database_path: str | Path | None=None):
        resolved_database_path = database_path if database_path is not None else
settings.database_path
        self.database_path = Path(resolved_database_path).expanduser().resolve()
        self.database_path.parent.mkdir(parents=True, exist_ok=True)
    def connect(self) -> sqlite3.Connection:
        connection = sqlite3.connect(str(self.database_path), timeout=30)
        connection.row_factory = sqlite3.Row
        connection.execute('PRAGMA foreign_keys = ON;')
        connection.execute('PRAGMA journal_mode = WAL;')
        return connection
    @contextmanager
    def get_connection(self) -> Generator[sqlite3.Connection, None, None]:
        connection = self.connect()
        try:
            yield connection
            connection.commit()
        except Exception:
            connection.rollback()
            raise
        finally:
            connection.close()
    def initialize(self) -> None:
        print('[Database] Initializing Component 4 database...')
        with self.get_connection() as connection:
            for statement in ALL_TABLES:
                connection.execute(statement)
            for statement in CREATE_INDEXES:
                connection.execute(statement)
        print('[Database] Database initialized successfully')
```

```
    print(f'[Database] Path: {self.database_path}')
def health_check(self) -> bool:
    try:
        with self.get_connection() as connection:
            result = connection.execute('SELECT 1;').fetchone()
        return bool(result and result[0] == 1)
    except Exception as exc:
        print(f'[Database] Health check failed: {exc}')
        return False
def get_tables(self) -> List[str]:
    with self.get_connection() as connection:
        rows = connection.execute("\n            SELECT name\n            FROM sqlite_master\n
                    WHERE type = 'table'\n            AND name NOT LIKE 'sqlite_%'\n
                    ORDER BY name;\n            ").fetchall()
    return [row['name'] for row in rows]
def get_table_columns(self, table_name: str) -> List[str]:
    allowed_tables = {'events', 'alerts', 'alert_evidence', 'xai_explanations', 'analyst_feedback'}
    if table_name not in allowed_tables:
        raise ValueError(f'Unknown table: {table_name}')
    with self.get_connection() as connection:
        rows = connection.execute(f'PRAGMA table_info({table_name});').fetchall()
    return [row['name'] for row in rows]
def get_row_count(self, table_name: str) -> int:
    allowed_tables = {'events', 'alerts', 'alert_evidence', 'xai_explanations', 'analyst_feedback'}
    if table_name not in allowed_tables:
        raise ValueError(f'Unknown table: {table_name}')
    with self.get_connection() as connection:
        row = connection.execute(f'\n            SELECT COUNT(*) AS count\n            FROM
            {table_name};\n            ').fetchone()
    return int(row['count'])
def get_database_info(self) -> dict:
    return {'database_path': str(self.database_path), 'exists': self.database_path.exists(),
        'healthy': self.health_check(), 'tables': self.get_tables() if self.database_path.exists()
        else []}
```

A.16 SHAP Model-explanation Implementation

This listing records the supplied SHAP explanation path. It must be evaluated against the fitted model, feature order, background data and output space. It does not explain graph joins or temporal rules by itself, and its presence does not establish explanation fidelity or runtime compatibility.

Source: src/xai/shap_explainer.py. Reformatting: Python AST unparse; comments, docstrings and standalone demonstration harnesses omitted; long display lines wrapped. The authoritative source is the supplied archive.

```
import numpy as np
import json
import joblib
from pathlib import Path
from typing import Dict, List, Optional
class SHAPExplainer:
    def __init__(self, models_path: str=None):
        if models_path is None:
            models_path = str(Path(__file__).parent.parent.parent / 'models')
        self.models_path = Path(models_path)
        self.explainer = None
        self.feature_names = []
        self._load_explainer()
    def _load_explainer(self) -> None:
        try:
            self.explainer = joblib.load(self.models_path / 'shap_explainer_xgb.pkl')
            metadata_path = self.models_path / 'training_metadata.json'
            if metadata_path.exists():
                with open(metadata_path, 'r') as f:
                    metadata = json.load(f)
            self.feature_names = ['dur', 'sbytes', 'dbytes', 'rate', 'spkts', 'dpkts', 'sttl', 'dttl',
                'sload', 'dload', 'sloss', 'dloss', 'sinpkt', 'dinpkt', 'sjit', 'djit', 'swin', 'dwin',
                'tcprtt', 'synack', 'ackdat', 'smean', 'dmean', 'ct_srv_src', 'ct_state_ttl',
                'ct_dst_ltm', 'ct_src_dport_ltm', 'ct_dst_sport_ltm', 'ct_dst_src_ltm', 'ct_src_ltm',
                'ct_srv_dst', 'is_ftp_login', 'ct_ftp_cmd', 'ct_flw_http_mthd', 'is_sm_ips_ports',
                'trans_depth', 'response_body_len']
            print('[SHAP] Explainer loaded successfully')
        except FileNotFoundError as e:
            print(f'[SHAP] Explainer not found: {e}')
    def explain_alert(self, alert: Dict) -> Dict:
        shap_values = alert.get('shap_values', [])
        feature_names = alert.get('feature_names', self.feature_names)
        top_features = alert.get('top_features', [])
        explanation = self._generate_explanation(alert, top_features)
        recommendation = self._generate_recommendation(alert)
        return {'alert_id': alert.get('alert_id'), 'risk_score': alert.get('risk_score'), 'severity':
            alert.get('severity'), 'explanation': explanation, 'recommendation': recommendation,
            'top_features': top_features, 'shap_values': shap_values, 'feature_names':
feature_names,
            'lateral_path': alert.get('lateral_movement_path', []), 'mitre_techniques':
            alert.get('mitre_techniques', []), 'layers_fired': alert.get('layers_fired', []),
            'score_breakdown': alert.get('score_breakdown', {}), 'dashboard_data':
```

```
            self._build_dashboard_data(alert, top_features)}
    def _generate_explanation(self, alert: Dict, top_features: List[Dict]) -> str:
        risk = alert.get('risk_score', 0)
        device = alert.get('source_device', 'Unknown')
        path = alert.get('lateral_movement_path', [])
        layers = alert.get('layers_fired', [])
        mitre = alert.get('mitre_techniques', [])
        lines = []
        lines.append(f'The engine detected lateral movement with {risk:.1%} confidence
originating from
            device {device}.')
        if path and len(path) >= 2:
            path_str = ' → '.join(path)
            lines.append(f'The attack traversal path was identified as: {path_str}.')
        if layers:
            layer_str = ', '.join(layers)
            lines.append(f'Evidence was detected across {len(layers)} monitoring layers:
{layer_str}.')
        if top_features:
            top = top_features[0]
            lines.append(f"The strongest indicator was '{top['feature']}' with a SHAP value of
                {top['shap_value']:+.4f}, which {top['direction'].replace('_', ' ')}.")
        if mitre:
            techs = ', '.join([m['technique_id'] for m in mitre])
            lines.append(f'MITRE ATT&CK techniques identified: {techs}.')
        return ' '.join(lines)
    def _generate_recommendation(self, alert: Dict) -> str:
        severity = alert.get('severity', 'LOW')
        path = alert.get('lateral_movement_path', [])
        device = alert.get('source_device', 'Unknown')
        if severity == 'CRITICAL':
            action = f'IMMEDIATE ACTION REQUIRED: Isolate device {device} from the
network. Initiate
                incident response procedure. Preserve memory forensics before remediation.'
        elif severity == 'HIGH':
            action = f'URGENT: Investigate device {device} within the next 30 minutes. Monitor all
                outbound connections. Consider network isolation.'
        elif severity == 'MEDIUM':
            action = f'INVESTIGATE: Review recent activity on device {device}. Check
authentication logs
                and running processes.'
        else:
            action = f'MONITOR: Keep device {device} under observation. No immediate action
required.'
        if path and len(path) > 1:
            action += f" Also investigate connected devices: {', '.join(path[1:])}."
        return action
    def _build_dashboard_data(self, alert: Dict, top_features: List[Dict]) -> Dict:
        score_breakdown = alert.get('score_breakdown', {})
        path = alert.get('lateral_movement_path', [])
        graph_elements = []
        if path:
            for device in path:
                graph_elements.append({'data': {'id': device, 'label': device}})
```

```
            for i in range(len(path) - 1):
                graph_elements.append({'data': {'source': path[i], 'target': path[i + 1]}})
        shap_chart = [{'feature': f['feature'], 'shap_value': f['shap_value'], 'color': '#e74c3c' if
            f['shap_value'] > 0 else '#3498db'} for f in top_features]
        score_gauge = {'value': alert.get('risk_score', 0) * 100, 'max': 100, 'color': '#e74c3c' if
            alert.get('severity') == 'CRITICAL' else '#e67e22'}
        return {'graph_elements': graph_elements, 'shap_chart': shap_chart, 'score_gauge':
score_gauge,
            'score_breakdown': [{'name': k, 'value': round(v * 100, 1)} for k, v in
            score_breakdown.items()], 'mitre_list': alert.get('mitre_techniques', []), 'layers_fired':
            alert.get('layers_fired', [])}
    def explain_from_shap_values(self, shap_values: List[float], feature_names: List[str], top_n:
int=10)
        -> List[Dict]:
        pairs = list(zip(feature_names, shap_values))
        pairs.sort(key=lambda x: abs(x[1]), reverse=True)
        return [{'rank': i + 1, 'feature': name, 'shap_value': round(val, 4), 'direction':
            'increases_risk' if val > 0 else 'decreases_risk', 'magnitude': 'strong' if abs(val) > 1.0
            else 'moderate' if abs(val) > 0.3 else 'weak'} for i, (name, val) in
enumerate(pairs[:top_n])]
```

APPENDIX B: ACADEMIC INTEGRITY AND AI-USE DISCLOSURE

This appendix records how this draft was prepared. It is part of the manuscript and should remain accurate in the final submission. The student must check the institution's current rules and obtain any required supervisor guidance; this document does not provide institutional approval.

B.1 Assistance Used

OpenAI ChatGPT/Codex was used to inspect the supplied formatting and structure files, retrieve project artefacts, examine Component 4 source and metadata, search primary technical literature, draft and organise the manuscript, generate diagrams, create the Word document and check pagination. The assistance was substantial and includes original prose generation rather than spelling correction alone.

A Python script generated the document. The source archive was inspected without executing the trained model binaries. One existing database-resilience script was executed using temporary databases, and the archived SQLite file was inspected read-only. These activities are disclosed as manuscript-preparation verification, not retroactively attributed to experiments conducted by the student.

B.2 Evidence and Authorship Boundaries

Historical model scores, saved runtime reports and the archived failed integration test are attributed to supplied project artefacts. Proposed detection, robustness and participant studies are explicitly marked as prospective. No participant data, ethics approval, supervisor certification, institutional endorsement or hospital deployment result has been invented.

The student must confirm ownership of the individual contribution and accurately acknowledge shared code or assistance. The component boundary excludes the internal development and evaluation of upstream components. Source availability alone does not establish authorship, and the final declaration must not be signed without review.

B.3 Required Author Review Before Submission

Review item	Required action
Identity and degree details	Replace bracketed title-page fields with verified official details.
Contribution ownership	Confirm which design, implementation and experiments are the student's work.
Results	Retain the evidence categories; add completed experiments only with logs and manifests.
References	Read and verify cited sources and confirm citation accuracy.
Outstanding validation	Resolve or discuss the staged failure and complete required research evaluation.
Institutional rules	Confirm that the disclosed AI assistance is permitted and acknowledged appropriately.
Supervisor certification	Obtain genuine review and signature; do not treat this draft as approval.
Final pagination	Update contents if editing changes layout and recheck the required formatting.

Table B.1. Final author-review responsibilities.

Table B.1 is a submission-readiness checklist, not a record of completed approvals. The current document is an evidence-grounded thesis draft. Academic acceptability ultimately depends on the author's verified contribution, completed research requirements and institutional review.